CodeFusionAI

PLACEMENT PREPARATION GUIDE

AI-Powered Real-Time Collaborative Code Editor

The whole project, from first principles to system design: architecture, every technology choice and its alternatives, the real implementation, diagrams, failure scenarios, and a full interview question bank with model answers.

React 19 + Vite · Monaco Editor · Yjs CRDT · Socket.IO · Node.js + Express 5 · MongoDB + Mongoose · Redis · Amazon S3 · Firebase Auth + JWT · Google Gemini · WebContainers · GitHub Actions

What's inside
31 chapters · 28 diagrams
150+ interview questions

How it's written
Grounded in the actual source code; unverifiable items are flagged

Edition
Interview edition
October 2026

Contents

FOUNDATIONS 6

1 How to Use This Guide 6
- 1.1 Who this guide is for 7
- 1.2 How each chapter is organised 7
- 1.3 The legend: implemented, partial, planned 7
- 1.4 What was verified, and what could not be 7

2 CodeFusionAI at a Glance 8
- 2.1 The one-paragraph pitch 9
- 2.2 The problem it solves 9
- 2.3 Feature inventory with honest status 9
- 2.4 The tech stack in one table 10

3 Fundamentals Primer 12
- 3.1 Client, server and the request–response model 13
- 3.2 HTTP, REST and JSON 13
- 3.3 WebSockets and Socket.IO 14
- 3.4 Authentication, authorisation and JWTs 15
- 3.5 Databases: document store, key-value cache, object storage 15
- 3.6 Concurrency, race conditions and locks 15
- 3.7 CRDTs in one page 16
- 3.8 The Node.js event loop 16
- 3.9 Single-page apps, bundlers and browser sandboxes 17

ARCHITECTURE & TECHNOLOGY 18

4 System Architecture 18
- 4.1 The overall picture 19
- 4.2 Client–server responsibilities 20
- 4.3 Frontend–backend communication 20
- 4.4 Backend layering 21
- 4.5 Repository map 22
- 4.6 Key architectural decisions (ADR summary) 22

5 Technology Choices and Alternatives 24
- 5.1 React 19 (UI library) 25
- 5.2 Vite 8 (build tool and dev server) 25
- 5.3 Monaco Editor (code editor) 25
- 5.4 Yjs + y-monaco (CRDT) 26
- 5.5 Socket.IO (real-time transport) 26
- 5.6 Node.js 22 + Express 5 (server) 27
- 5.7 MongoDB + Mongoose (metadata store) 27
- 5.8 Redis + ioredis (cache and coordination) 28
- 5.9 Amazon S3 (object storage) 28
- 5.10 Firebase Authentication + jose 29
- 5.11 Google Gemini (AI) 29
- 5.12 WebContainers + xterm.js (in-browser execution) 29
- 5.13 Tailwind CSS, Vitest, node:test, GitHub Actions 30
- 5.14 Comparison cheat table 30

IMPLEMENTATION DEEP DIVE 32

6 Backend Walkthrough 32
- 6.1 server.js: the composition root 33
- 6.2 Configuration modules 34
- 6.3 Error handling: AppError, asyncHandler, errorHandler 35
- 6.4 Validation middleware 36
- 6.5 Utilities: Redis key registry 36

7 API Design and Request Lifecycle 37
- 7.1 Endpoint reference 38
- 7.2 Socket.IO event reference 39
- 7.3 The life of a REST request 39
- 7.4 API design principles used (and where they bend) 40

8 Database Design 41
- 8.1 Schema and relationships 42
- 8.2 The Mongoose models in code 42
- 8.3 Query patterns and why each index exists 43
- 8.4 Design decisions and normalisation 44

9 Authentication and Authorisation 45
- 9.1 The complete flow 46
- 9.2 The verification code, line by line 47
- 9.3 Authorisation model 47
- 9.4 Session management 48

10 Real-Time Collaboration: Socket.IO and Yjs 49
- 10.1 Joining a room 50
- 10.2 The server-side CRDT service 51
- 10.3 A keystroke's journey 52
- 10.4 Write-back: from CRDT to Redis 53
- 10.5 Structural changes: create, delete, rename, upload 53
- 10.6 Presence and live cursors 54
- 10.7 Correctness: why CRDT merging is safe 54
- 10.8 Offline and reconnection behaviour 55

11 Caching, Locking and Persistence 56
- 11.1 Redis data model 57
- 11.2 Reading: the cache-aside waterfall 57
- 11.3 Writing: the full persistence pipeline 58
- 11.4 The token bucket, explained 60
- 11.5 The distributed persist lock 62
- 11.6 Failure-mode table 63

12 Amazon S3 Storage 64
- 12.1 Bucket layout and data flow 65
- 12.2 Why S3 and not MongoDB for contents 65
- 12.3 Security and operations 66

13 Version History 67
- 13.1 How a version is recorded 68
- 13.2 Restoring a version 69
- 13.3 The History UI 69

14 AI Integration 70
- 14.1 Request and response flow 71
- 14.2 Prompt construction 72
- 14.3 Applying AI code safely 72

14.4 Backend service and error mapping 73
14.5 The full-page Assistant 73
14.6 Cost, safety and privacy considerations 74

15 Frontend Walkthrough 75
15.1 Routing and component tree 76
15.2 The Room page (the workspace) 76
15.3 Monaco setup 78
15.4 State management choices 78
15.5 Dashboard, login and design system 79

16 Running Code: WebContainers and /api/execute 79
16.1 In-browser execution with WebContainers 80
16.2 Server-side execution: /api/execute 81

17 End-to-End Data Flow 82
17.1 A · Open the site and authenticate 83
17.2 B · Create or join a room 83
17.3 C · Edit and collaborate 84
17.4 D · Run and ask AI 84
17.5 E · Save, restore and leave 84

PRODUCTION ENGINEERING 86

18 Security 86
18.1 Defence in depth 87
18.2 Authentication versus authorisation recap 87
18.3 Secrets and environment variables 87
18.4 Input validation and injection 88
18.5 CORS 88
18.6 Rate limiting 89
18.7 File access permissions and data security 89
18.8 Security headers and transport 89

19 Testing and Continuous Integration 90
19.1 What is tested 91
19.2 CI pipeline 91
19.3 Testing gaps and what to add 92

20 Deployment 93
20.1 Deployment architecture 94
20.2 Environment variables 94
20.3 Production checklist and challenges 95

21 Scalability, Reliability and Monitoring 96
21.1 Where the current design stops scaling 97
21.2 Back-of-the-envelope capacity (estimates, not measurements) 97
21.3 Horizontal scaling design 97
21.4 Reliability and fault tolerance 98
21.5 Monitoring and observability 99

22 Troubleshooting, Failure Scenarios and Real Bugs 100
22.1 Bug 1: concurrent edits overwrote each other 101
22.2 Bug 2: the token bucket never throttled 101
22.3 Bug 3: “crossOriginIsolated” error in the terminal 101
22.4 Bug 4: editor context-menu actions did nothing 101
22.5 Bug 5: AI answered about the wrong file 101
22.6 Bug 6: AI returned “failed to generate” 102
22.7 Other bugs and edge cases worth knowing 102

22.8 Failure scenarios: what the system does 102
22.9 Debugging toolkit 103

INTERVIEW PREPARATION 104

23 Answer Framework and Project Pitch 104
23.1 The 30-second, 2-minute and 5-minute pitches 105
23.2 How to structure any technical answer 105
23.3 The STAR format for behavioural and project stories 105
23.4 Phrases that signal seniority 106
23.5 Red flags to avoid 106
24 Question Bank: Beginner 106
25 Question Bank: Intermediate 110
26 Question Bank: Advanced and System Design 116
27 Tough Follow-Ups, Answered in Depth 122
28 Mock Interview Rounds 127
28.1 Round 1: Project walkthrough (30 minutes, campus or first technical round) 128
28.2 Round 2: Deep dive (45 minutes, product-company technical round) 128
28.3 Round 3: System design (45–60 minutes, senior or SDE-2 style) 128
28.4 Rapid-fire round (answer each in one breath) 129

REVISION 131

29 Quick Reference and Cheat Sheets 131
29.1 Architecture in ten lines 132
29.2 Numbers to remember 132
29.3 Comparison tables 133
29.4 What is implemented versus proposed (memorise) 134
29.5 FAQ: the twelve most likely questions 134
30 Glossary 135
31 Final Checklist and References 137
31.1 The night before 138
31.2 Demo script (5 minutes) 138
31.3 References 138

PART I

Foundations

What CodeFusionAI is, why it exists, what is actually built, and the basic concepts you need before the deep dives.

CHAPTER 1

How to Use This Guide

This guide is your main study resource for CodeFusionAI interviews. It starts from first principles, then walks through the real implementation file by file, and ends with question banks, mock rounds and revision sheets.

1.1 Who this guide is for

It assumes you know some JavaScript and have used React, but it does **not** assume you already understand WebSockets, CRDTs, Redis, S3, JWTs or system design. Every term is defined before it is used, in a blue *Definition* box. If a chapter feels easy, skim the definitions and jump to its *Interview kit*.

1.2 How each chapter is organised

- **Concept first.** A plain-language explanation of the idea (for example, "what is a distributed lock").
- **Then the real code.** Snippets come from the actual repository with the file path in the title bar. Snippets are shortened where noted, but never altered in meaning.
- **Then diagrams.** Every diagram has numbered steps that the text explains one by one.
- **Then the interview kit.** Each major topic ends with six parts: a simple explanation, a technical explanation, a spoken answer you can say out loud, a CodeFusionAI example, likely follow-up questions with model answers, and common mistakes to avoid.

STUDY PLAN (7 DAYS)

Day 1: Part I and Chapter 4 (architecture). **Day 2:** Chapter 5 (technology choices). **Day 3:** Chapters 6–9 (backend, APIs, database, auth). **Day 4:** Chapters 10–13 (real time, caching, S3, history). **Day 5:** Chapters 14–17 plus Part IV. **Day 6:** Part V question banks, answered out loud. **Day 7:** mock rounds, cheat sheet and final checklist.

1.3 The legend: implemented, partial, planned

Interviewers punish exaggeration. This guide labels features honestly, using these badges everywhere:

Badge	Meaning
Implemented	The code exists in the repository and was traced end to end while writing this guide.
Partial	Something exists, but with a known gap (for example, an endpoint exists but has no ownership check, or a feature has no UI).
Proposed	A design you can *describe* as a future improvement. It is not in the code; never say it is.
Not verifiable	Depends on things outside the repository (hosting, production settings, real traffic numbers).

1.4 What was verified, and what could not be

Everything in this guide was checked against the source code in the repository (backend `Backend/src`, frontend `Frontend/src`, the configuration files, the tests and the CI workflow). These items **cannot** be verified from the files, and the guide flags them wherever they come up:

- **Production hosting of the backend.** The repository contains `vercel.json` and `netlify.toml` for the frontend, but nothing that pins where the Node.js backend runs. A Docker setup was added in commit `ccfdc28` and removed again in commit `495f18d`, so it is not part of the current tree.
- **Redis durability settings** (RDB/AOF persistence) of whatever Redis you deploy. These decide how much data a Redis crash can lose.
- **Load numbers.** No load test exists in the repository, so any "handles N users" claim must be presented as an estimate with assumptions, never as a measured fact.
- **MongoDB Atlas, S3 bucket and Firebase project configuration** (backups, IAM policies, bucket policies, authorised domains).

GOLDEN RULE FOR THE INTERVIEW

Say "we implemented X" only for green items. For the rest, say "the current version does Y; the next step I would take is X, because…". Being specific about limitations reads as seniority, not weakness.

CHAPTER 2

CodeFusionAI at a Glance

2.1 The one-paragraph pitch

CodeFusionAI is a browser-based, real-time collaborative code editor with an AI pair programmer. Several people open the same *room* and edit the same files at the same time, and see each other's cursors live. They can run Node.js projects inside the browser through WebContainers, ask a Gemini-powered assistant about the open file or a selection, and restore any earlier saved version of the workspace. The backend is Node.js with Express and Socket.IO. Live edits are merged with the Yjs CRDT. Hot state sits in Redis, and is flushed to Amazon S3 (file contents) and MongoDB (metadata and version history) under a token-bucket policy with a Redis lock.

2.2 The problem it solves

Pair programming and code reviews usually involve screen sharing, where only one person types, or pushing to Git and waiting. Tools like VS Code Live Share or Replit solve this, but they are large products. CodeFusionAI shows the core engineering behind such tools:

- **Simultaneous editing without overwrites:** two people typing in the same file must both keep their changes.
- **Low latency:** changes appear on other screens in milliseconds, not after a page refresh.
- **Durability without hammering storage:** every keystroke cannot become a cloud write, yet work must not be lost.
- **AI help in context:** the assistant should see the file you are actually looking at.
- **Running code safely:** projects execute in the user's own browser sandbox, not on the shared server.

2.3 Feature inventory with honest status

Feature	Status	Where it lives / notes
Sign in (Google, email + password, reset)	Implemented	Firebase Auth in `firebase/firebase.js`; display name captured on sign-up.
Server-side token verification	Implemented	`middleware/auth.js` verifies Firebase ID tokens with `jose` for REST and Socket.IO.
Create / join / close / reopen / delete rooms	Implemented	Socket events and `/api/rooms`; owner-only checks for close, reopen and delete.
Private rooms	Partial	`accessType` hides a room from the public hub, but `join-room` does not check it. Anyone signed in who knows the room ID can join.
Real-time same-file co-editing	Implemented	Yjs CRDT: `crdt.service.js` (server) + `y-monaco` (client).
Live cursors and presence	Implemented	`cursor-change` / `room-users` socket events, name tags rendered as Monaco content widgets.

Feature	Status	Where it lives / notes
File tree: create, delete, rename, upload folder	Implemented	Structural changes sent as a full file list on `code-change`, reconciled into the CRDT.
Redis hot cache + S3/Mongo persistence	Implemented	`workspace.service.js`, `redis.service.js`; token bucket and `SET NX EX` lock.
Version history, compare and restore	Implemented	`history.service.js`, Snapshot model, History panel and DiffEditor compare view.
AI panel in the room (file/selection aware)	Implemented	`AIPanel.jsx` → `POST /api/ai/chat` → Gemini 2.5 Flash.
Full-page AI assistant with chat history	Implemented	`pages/Assistant.jsx`; history kept in `localStorage` (per browser, not synced).
AI review / explain / optimise endpoints	Partial	Backend routes exist (`/api/ai/review`, etc.); the current UI uses `/chat` with suggested prompts instead.
Run Node.js projects in the browser	Implemented	WebContainers + xterm.js terminal + preview iframe; requires COOP/COEP headers.
Server-side execution (`/api/execute`)	Partial	Exists and is rate-limited, but runs code with `child_process.exec` on the host with no sandbox. The UI does not call it. Treat it as a known risk.
Rate limiting	Implemented	Redis fixed-window limiter on AI, files, execute, profile, history and delete routes.
Presigned S3 upload/download API	Partial	`/api/files/*` routes exist; no room-membership check, and the main editor flow does not use them.
Automated tests and CI	Implemented	16 backend tests (`node --test`), 7 frontend tests (Vitest), GitHub Actions on push and PR.
Horizontal scaling (many backend nodes)	Proposed	Needs the Socket.IO Redis adapter and a shared or routed CRDT store. See Chapter 21.
Containerised deployment	Not in tree	Docker files existed in history but were removed; backend hosting is not defined in the repository.

2.4 The tech stack in one table

Layer	Technology (version in package.json)	Job in CodeFusionAI
UI framework	React 19, React Router 7	Single-page app: landing, login, dashboard, assistant, docs, room.
Build tool	Vite 8	Dev server with hot reload, production bundling, Monaco web workers.
Editor	Monaco Editor 0.55 via `@monaco-editor/react`	The VS Code editor component: syntax highlighting, IntelliSense, diff view.
Real-time sync	Yjs 13.6, y-monaco	CRDT that merges concurrent edits; binds a Y.Text to a Monaco model.

Layer	Technology (version in package.json)	Job in CodeFusionAI
Transport	Socket.IO 4.8 (server and client)	WebSocket-based events: join, CRDT updates, cursors, presence, save, restore.
API server	Node.js 22, Express 5	REST endpoints, middleware pipeline, error handling.
Metadata DB	MongoDB with Mongoose 9	Rooms, file metadata, version snapshots, user profiles.
Cache / coordination	Redis via ioredis 5 (in-memory fallback)	Workspace hash, dirty set, token buckets, locks, rate-limit counters.
Blob storage	Amazon S3 (AWS SDK v3)	File contents and snapshot JSON; presigned URLs.
Identity	Firebase Auth 12 + `jose` 5	Sign-in on the client; JWT verification on the server.
AI	Google Gemini 2.5 Flash via `@google/generative-ai`	Chat, code explanation, bug finding, optimisation, tests.
In-browser runtime	WebContainers API 1.6, xterm 5	Run `npm install` / `npm run dev` and a shell inside the browser tab.
Styling	Tailwind CSS 4 + design tokens	Light-only design system (`tokens.css`).
Quality	ESLint 10, Vitest 5, `node:test` , GitHub Actions	Linting, unit tests, CI on every push and pull request.

Interview kit · "Tell me about your project"

SIMPLE EXPLANATION

It is like Google Docs for code, with an AI helper built in. Several people edit the same files live, run them in the browser, and ask an AI about the code.

TECHNICAL EXPLANATION

A React + Monaco single-page app talks to a Node.js server over REST (Express) and WebSockets (Socket.IO). Concurrent edits are merged with the Yjs CRDT, with an authoritative document per room on the server. Hot workspace state lives in Redis. A background flusher, throttled by a per-room token bucket and protected by a Redis `SET NX EX` lock, persists file contents to S3 and metadata to MongoDB, and records de-duplicated snapshots for version history. Identity is Firebase Auth, verified server-side with `jose`. AI calls go through the backend to Gemini so the key never reaches the browser.

SAY IT LIKE THIS (ABOUT 60 SECONDS)

> *"CodeFusionAI is a real-time collaborative code editor with an AI assistant. The interesting part is the collaboration engine. Instead of sending whole files and letting the last writer win, every keystroke becomes a small Yjs CRDT update over Socket.IO, so two people typing in the same line both keep their changes. On the server I keep one Yjs document per room, write it back to Redis every few hundred milliseconds, and flush Redis to S3 and MongoDB in the background. A token bucket stops a busy room from spamming S3, and a Redis lock makes sure only one flush per room runs at a time. Every flush also records a version, de-duplicated by a SHA-256 hash, so users can compare and restore. Authentication is Firebase on the client with JWT verification on the server, and the AI assistant sees the file or selection you're working on."*

CODEFUSIONAI EXAMPLE

Alice and Bob both edit `src/App.jsx`. Alice adds an import on line 1 while Bob renames a variable on line 20. Both changes appear on both screens within milliseconds, both survive, and about 10 seconds after they stop typing the room is flushed to S3 as version `v12`.

LIKELY FOLLOW-UPS

- **Why build this?** To learn the hard parts of real-time systems, such as concurrency, caching and persistence trade-offs, in a product people actually understand.
- **What was the hardest part?** Making concurrent edits correct (moving from whole-file last-writer-wins to a CRDT), and balancing durability against S3 write cost.
- **What would you improve first?** Room-level authorisation for private rooms and file APIs, removing the unsandboxed execute endpoint, and the horizontal-scaling story (Chapter 21).

MISTAKES TO AVOID

- Listing technologies without saying what problem each one solves.
- Claiming it “scales to millions”; there is no load test. Say what limits it today and how you would remove them.
- Saying files are “stored in MongoDB”. Contents are in S3; MongoDB holds metadata.

CHAPTER 3

Fundamentals Primer

Read this chapter if any of these words feel fuzzy: client, server, HTTP, REST, WebSocket, JSON, JWT, cache, CRDT, event loop. Each section ends with how CodeFusionAI uses the idea.

3.1 Client, server and the request–response model

DEFINITION: CLIENT AND SERVER

A **client** is a program that asks for something; in our case, the React app running in a browser tab. A **server** is a program that waits for requests and answers them; in our case, the Node.js process on port 5000. They talk over a network using agreed rules called **protocols**.

In classic web apps the client always starts the conversation: it sends a *request*, the server sends back a *response*, and the conversation ends. That is fine for "load my rooms", but bad for "tell me the moment Bob types something", because the server cannot speak first. That limitation is why CodeFusionAI also uses WebSockets.

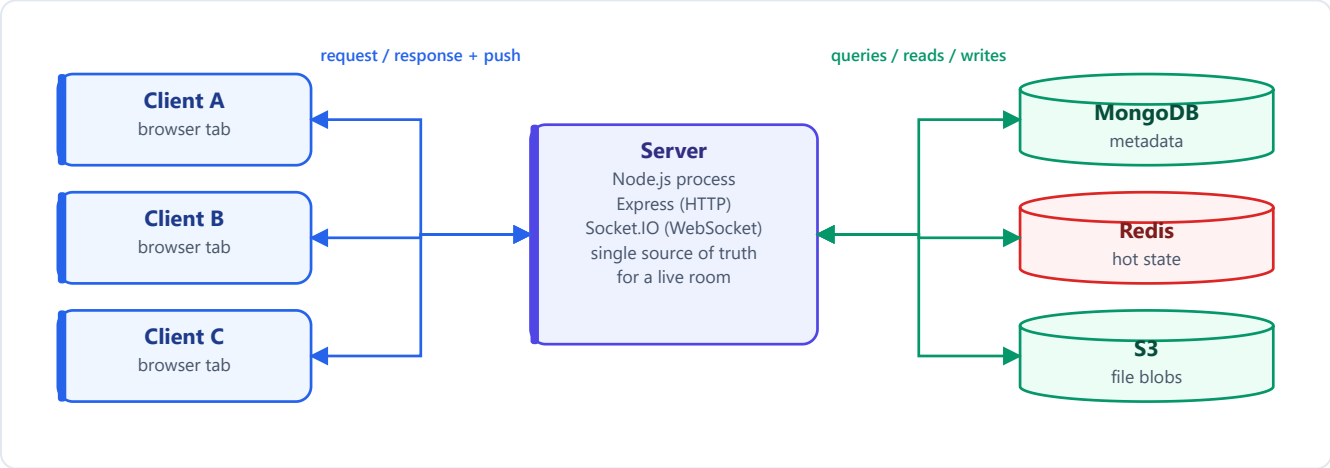


Figure 1. Client–server model in CodeFusionAI: many browser clients share one server process, which in turn talks to three storage systems.

Reading the figure: each client keeps two channels open to the server, normal HTTP requests and one long-lived WebSocket. The server is the only component that touches MongoDB, Redis and S3. Clients never see database credentials, which is a key security property.

3.2 HTTP, REST and JSON

DEFINITION: HTTP

HyperText Transfer Protocol: the request–response protocol of the web. A request has a **method** (GET reads, POST creates/acts, PUT replaces, PATCH updates, DELETE removes), a **URL**, **headers** (metadata such as `Authorization`) and an optional **body**. A response has a **status code** (2xx success, 4xx client error, 5xx server error), headers and a body. HTTPS is HTTP encrypted with TLS.

DEFINITION: REST AND JSON

REST is a style of designing HTTP APIs around *resources* (rooms, files, users) addressed by URLs, manipulated with the standard methods, and **stateless**: each request carries everything the server needs, such as the auth token. **JSON** (JavaScript Object Notation) is the text format used for the bodies, for example `{"roomId":"cfai-ab12-cd34"}`.

CodeFusionAI exposes REST endpoints under `/api`: `GET /api/rooms/user/:userId` lists your rooms, `POST /api/rooms/:roomId/close` closes one, `POST /api/ai/chat` asks the AI, and so on. Chapter 7 lists all of them.

3.3 WebSockets and Socket.IO

DEFINITION: WEBSOCKET

A protocol (RFC 6455) that starts as an HTTP request with an `Upgrade: websocket` header. If the server agrees (status `101 Switching Protocols`), the same TCP connection stays open and **both sides can send messages at any time**, with tiny per-message overhead.

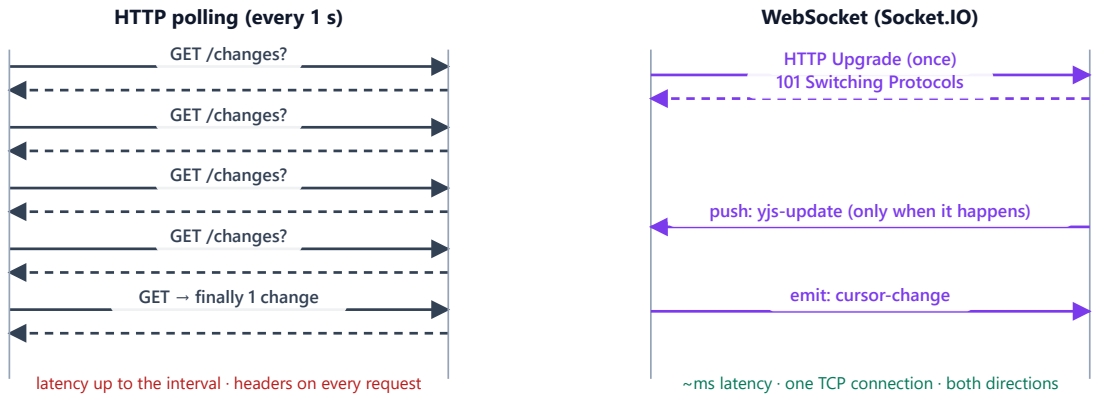


Figure 2. HTTP polling versus a WebSocket. Polling wastes requests and adds latency; a WebSocket pushes changes the moment they happen.

Reading the figure: on the left the client keeps asking "anything new?", so most responses are empty and a change waits up to one polling interval. On the right there is one upgrade handshake, after which the server pushes `yjs-update` as soon as Bob types, and the client emits `cursor-change` whenever the cursor moves.

DEFINITION: SOCKET.IO

A library built on top of WebSockets (with HTTP long-polling as a fallback) that adds named **events** (`socket.emit("join-room", data)`), **rooms** (broadcast groups), **acknowledgements** (request/response over the socket), automatic **reconnection**, and **binary** support. Its wire format is its own protocol, so you need the Socket.IO client, not a raw WebSocket.

3.4 Authentication, authorisation and JWTs

DEFINITION: AUTHENTICATION VERSUS AUTHORISATION

Authentication (AuthN) answers "who are you?", for example proving you are user `uid_123`. **Authorisation (AuthZ)** answers "are you allowed to do this?", for example "only the owner may delete this room". A 401 status means not authenticated; 403 means authenticated but not allowed.

DEFINITION: JWT

A JSON Web Token is three base64url parts joined by dots: `header.payload.signature`. The payload holds *claims* such as `sub` (user id), `iss` (issuer), `aud` (audience) and `exp` (expiry). The signature is made with the issuer's private key; anyone holding the matching **public key** can verify that nobody changed the token, without calling the issuer. Firebase ID tokens are RS256-signed JWTs that live for one hour.

3.5 Databases: document store, key-value cache, object storage

Kind	Example	Mental model	Used for here
Document database	MongoDB	Collections of JSON-like documents; flexible schema; secondary indexes; queries by any field.	Rooms, file metadata, snapshots, profiles.
In-memory data store	Redis	A giant hash map in RAM with rich types (strings, hashes, sets) and atomic commands; microsecond latency; optional persistence.	Hot workspace, dirty set, token buckets, locks, rate-limit counters.
Object storage	Amazon S3	A flat key → bytes store. Effectively unlimited, very durable, cheap per GB, but each request costs latency and money.	File contents, snapshot JSON.

DEFINITION: CACHE, TTL, CACHE HIT/MISS

A **cache** is a fast copy of data that also lives somewhere slower. A **hit** means the data was in the cache; a **miss** means we fall back to the slow source and usually fill the cache. **TTL** (time to live) makes a key delete itself after N seconds.

3.6 Concurrency, race conditions and locks

DEFINITION: RACE CONDITION

A bug where the result depends on the timing of two operations. The classic case is **lost update**: A reads value 5, B reads 5, A writes 6, B writes 6, and one increment is lost. In an editor, “last writer wins” on a whole file is a lost update for someone's typing.

DEFINITION: LOCK / MUTEX AND DISTRIBUTED LOCK

A **lock** ensures only one actor does something at a time. A **distributed lock** does this across processes or machines, using a shared store such as Redis: whoever creates the key first holds the lock. A **TTL** on the key makes sure a crashed holder cannot block everyone forever.

3.7 CRDTs in one page

DEFINITION: CRDT

A Conflict-free Replicated Data Type is a data structure that many replicas can edit independently and still merge to the **same result regardless of the order** in which they receive each other's changes. It needs no central lock and no “transform” step. Yjs implements a sequence CRDT (based on the YATA algorithm) for text.

The key trick: instead of saying “insert ‘r’ at index 2” (indexes shift when others type), a CRDT says “insert ‘r’ with unique ID `(client A, clock 1)` right after the character whose ID is `(origin, 7)`”. IDs never shift, so every replica can place every character deterministically.

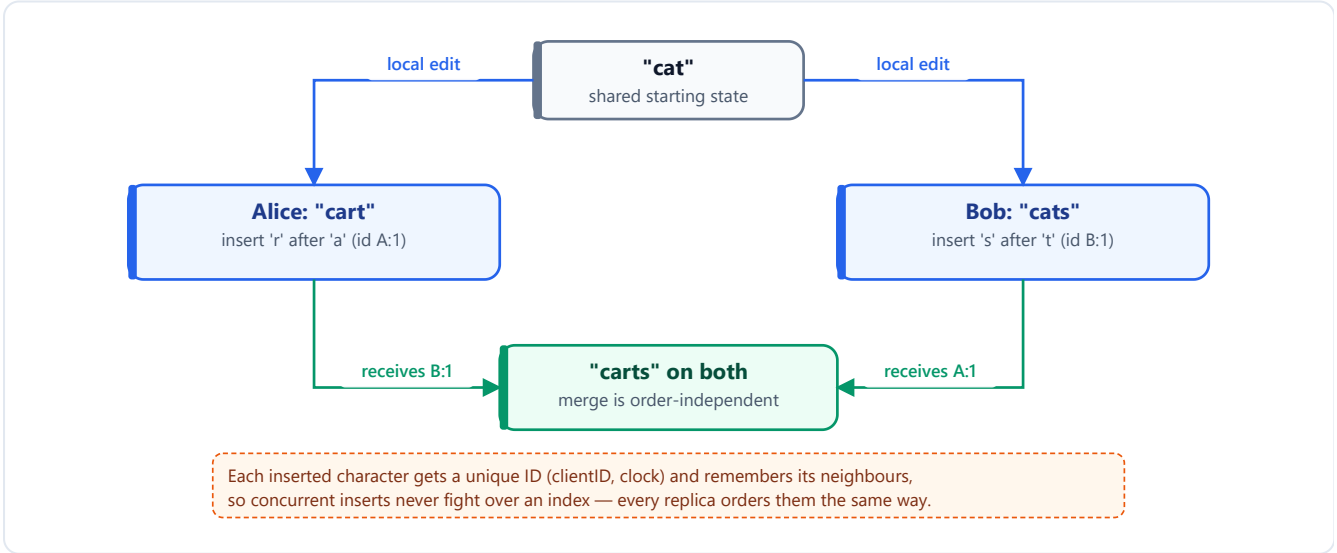


Figure 3. Two concurrent edits to “cat”. Each replica applies the other's operation and both converge to “carts”.

3.8 The Node.js event loop

DEFINITION: EVENT LOOP

Node.js runs your JavaScript on **one thread**. Slow I/O (network, disk, database) is handed to the operating system, and a callback or promise continuation runs when it finishes. One process can therefore hold thousands of idle WebSocket connections cheaply, but a CPU-heavy task (for example hashing a huge file) blocks *everyone* until it finishes.

This shapes CodeFusionAI's design. The server mostly forwards small Yjs updates (cheap), writes back to Redis on a 400 ms debounce, and does the heavy S3/Mongo flush asynchronously, without blocking typing.

3.9 Single-page apps, bundlers and browser sandboxes

DEFINITION: SPA, BUNDLER, WEB WORKER

A **single-page application** loads one HTML page and swaps views with JavaScript (React Router). A **bundler** (Vite) turns many source modules into optimised files for the browser. A **web worker** runs JavaScript on a background thread; Monaco runs its language services in workers so typing stays smooth.

DEFINITION: CROSS-ORIGIN ISOLATION (COOP/COEP)

Two response headers, `Cross-Origin-Opener-Policy: same-origin` and `Cross-Origin-Embedder-Policy: require-corp`, put a page in an isolated process. Only then do browsers enable `SharedArrayBuffer`, which WebContainers need to run Node.js inside the tab.

PART II

Architecture & Technology

The big picture: how the pieces fit, how they talk to each other, and why each technology was chosen over its alternatives.

CHAPTER 4

System Architecture

CodeFusionAI is a classic three-tier web system: a browser client, one Node.js application server, and managed data services. Two features make it unusual: the client runs a CRDT replica and a full Node.js runtime, and the server keeps a live in-memory document per room.

4.1 The overall picture

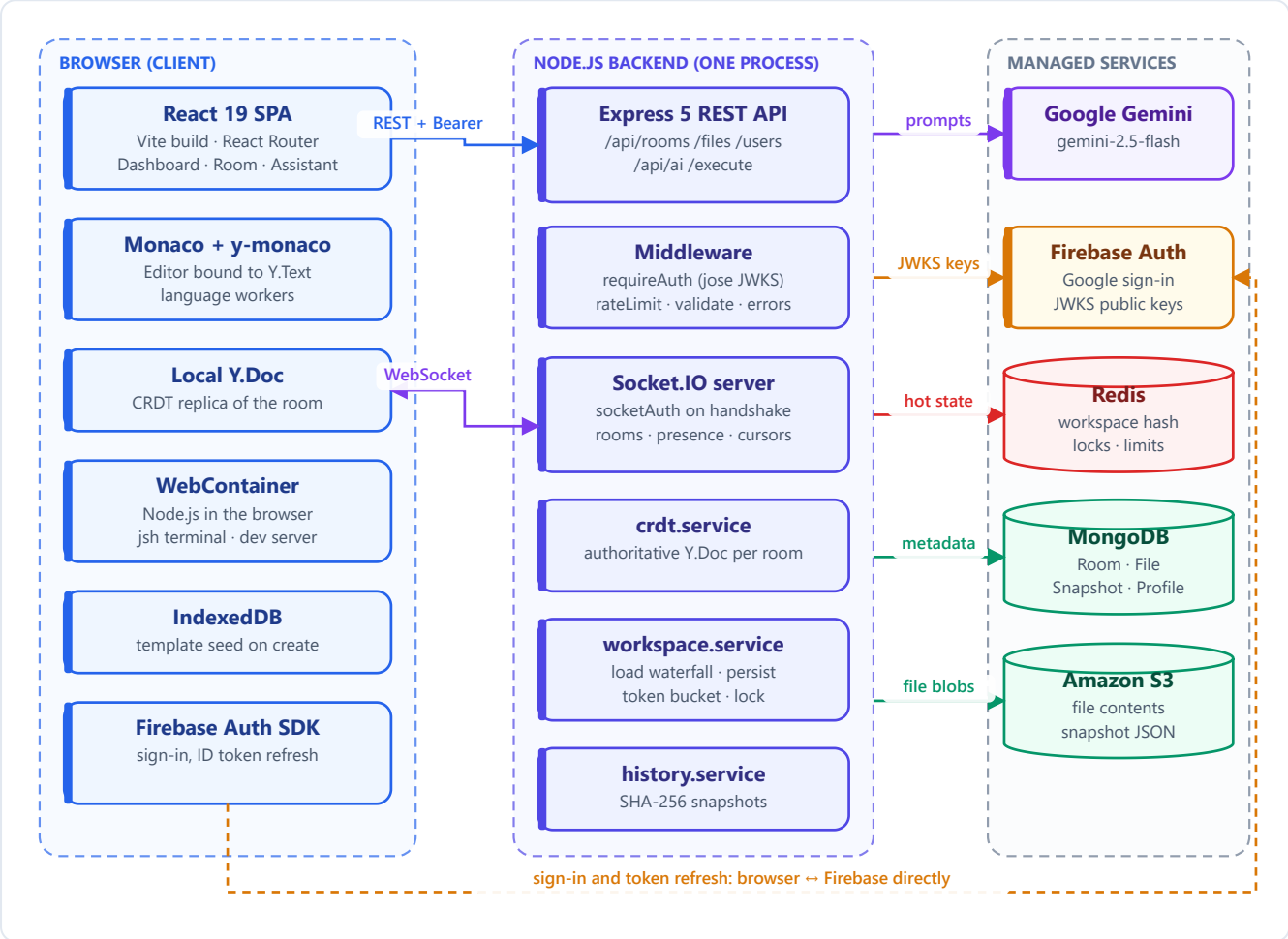


Figure 4. Overall system architecture. The browser owns rendering, editing, a CRDT replica and code execution; the backend owns identity checks, the authoritative CRDT, persistence and AI access; managed services hold data.

Step by step, from left to right:

1. **Browser tier.** The React SPA renders every screen. In a room, Monaco is bound to a local `Y.Doc` (the CRDT replica) through `y-monaco`. WebContainers run the project's Node.js code inside the tab, and IndexedDB holds the template files written when a room is created, so the first paint is instant. The Firebase SDK signs the user in and refreshes their ID token.
2. **Two channels to the backend.** Normal CRUD and AI go over *REST + Bearer token*. Collaboration (join, CRDT updates, cursors, presence, save, restore) goes over one *Socket.IO WebSocket*, authenticated once at the handshake.

3. **Backend tier.** A single Node.js process runs Express (REST) and Socket.IO on the same HTTP server and port. Middleware enforces auth, rate limits and validation. The services layer holds the logic: `crdt.service` keeps the authoritative Y.Doc, `workspace.service` loads and persists workspaces, and `history.service` records versions.
4. **Managed services.** Gemini answers prompts. Firebase publishes the public keys (JWKS) used to verify tokens. Redis holds hot state. MongoDB holds metadata. S3 holds the bytes.
5. **Sign-in never touches our backend.** The dashed orange path shows the browser talking to Firebase directly. Our server only *verifies* the resulting token, so it never sees a password.

4.2 Client–server responsibilities

Concern	Client (browser)	Server (Node.js)
Rendering and UX	All UI, routing, toasts, editor, diff view	None (pure API)
Identity	Signs in with Firebase, attaches the ID token	Verifies the token on every request and socket
Editing	Local CRDT replica; edits apply instantly (optimistic)	Authoritative CRDT per room; relays updates; validates membership
Persistence	IndexedDB seed only	Redis → S3 + MongoDB, version history
Code execution	WebContainers (sandboxed in the tab)	`/api/execute` exists but is unsandboxed and unused by the UI
AI	Builds the prompt from the live editor	Holds the Gemini key; rate-limits; calls Gemini

4.3 Frontend–backend communication

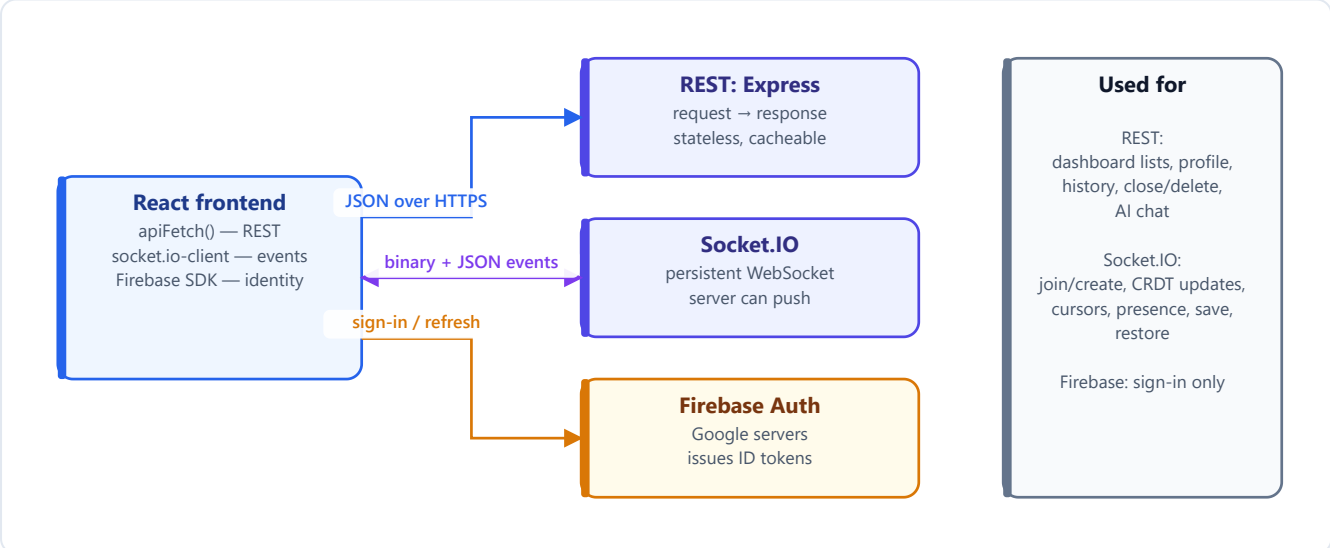


Figure 5. The three channels the frontend uses, and what each carries.

Why two channels? REST is ideal for request/response operations that are naturally one-off and benefit from HTTP semantics: status codes, caching, simple retries, and easy testing with curl. Collaboration needs *server push* and very small, very frequent messages, which is exactly what a persistent WebSocket is for. Both

channels carry the same Firebase identity: REST in the `Authorization` header, Socket.IO in `handshake.auth.token`.

Frontend/src/services/api.js

```
export async function apiFetch(url, options = {}) {
  const token = await getIdToken();               // Firebase refreshes it when it is near expiry
  const headers = new Headers(options.headers || {});
  if (token) headers.set("Authorization", `Bearer ${token}`);
  if (options.body && !headers.has("Content-Type")) headers.set("Content-Type",
"application/json");
  return fetch(url.startsWith("http") ? url : `${API_BASE}${url}`, { ...options, headers });
}
```

Frontend/src/services/socket.js

```
export const socket = io(URL, {
  autoConnect: false,            // connect only on Dashboard/Room, not on the login page
  reconnectionAttempts: 5,
  reconnectionDelay: 1000,
  // auth as a callback: a fresh token is fetched on every (re)connection
  auth: (cb) => { getIdToken().then((token) => cb({ token })).catch(() => cb({})); },
});
```

The `auth` callback detail matters. If it were a static object, a reconnect after an hour would present an *expired* token and be rejected. Because it is a function, Socket.IO asks for a fresh token each time it connects.

4.4 Backend layering

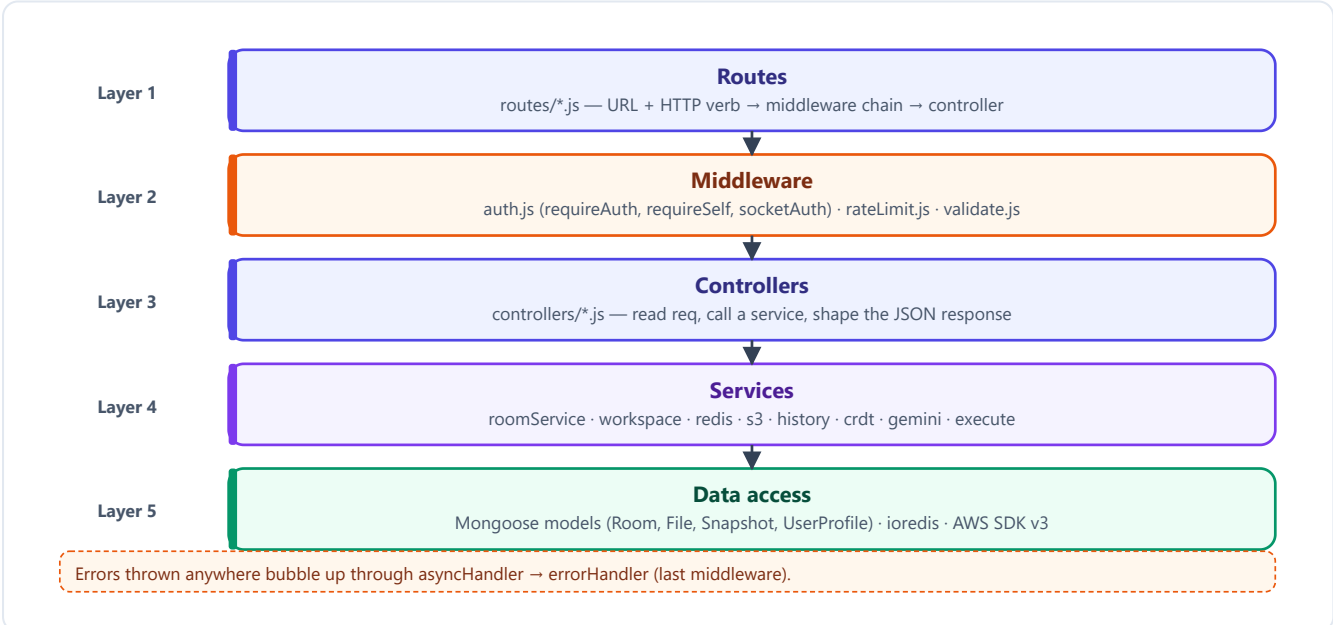


Figure 6. The backend is split into five layers. Each layer only calls the one below it.

- **Routes** (`routes/*.js`) map an HTTP verb and URL to a chain of middleware plus one controller.
- **Middleware** handles cross-cutting concerns: auth, rate limits, validation.
- **Controllers** are thin. They read `req`, call one service and shape the response.

- **Services** hold business logic and are the only place that orchestrates Redis, S3 and MongoDB.
- **Data access** is Mongoose models plus the ioredis and AWS SDK clients.

The socket controller follows the same rule: its header comment says it contains "no MongoDB queries, no S3 calls … all of that is delegated to services". This separation is what made it possible to add the CRDT and version history later without rewriting the handlers.

4.5 Repository map

Path	Purpose
`Backend/src/server.js`	Composition root: Express, CORS, JSON parser, routes, error handler, Socket.IO, background flusher, graceful shutdown.
`Backend/src/config/`	`db.js` (Mongo connect), `redis.js` (ioredis + in-memory fallback), `aws.js` (S3 client), `gemini.js`.
`Backend/src/middleware/`	`auth.js`, `rateLimit.js`, `validate.js`, `errorHandler.js`.
`Backend/src/routes/`, `controllers/`	REST endpoints and thin handlers; `socketController.js` for all socket events.
`Backend/src/services/`	`crdt`, `workspace`, `redis`, `s3`, `history`, `roomService`, `geminiService`, `execute`, `fileService`, `user`.
`Backend/src/models/`	Mongoose schemas: `Room`, `File`, `Snapshot`, `UserProfile`.
`Backend/src/utils/`	`AppError`, `asyncHandler`, `redisKeys` (key patterns and tuning constants), `fileHelpers`.
`Backend/test/`	16 tests: CRDT convergence, history hashing, rate limiter, token bucket, auth.
`Frontend/src/pages/`	`Landing`, `Login`, `Dashboard`, `Assistant`, `Docs`, `Room` (the workspace, about 1,300 lines).
`Frontend/src/components/Room/`	AIPanel, HistoryPanel, CompareView, BottomPanel, Terminal, PreviewPanel, TitleBar, StatusBar, modals.
`Frontend/src/services/`	`api.js`, `socket.js`, `aiService.js`, `webcontainer.js`, `db.js` (IndexedDB), `templates.js`.
`Frontend/src/lib/`	`monacoSetup.js` (workers, formatters), `rooms.js` (+ tests).
`Frontend/vite.config.js`, `vercel.json`, `netlify.toml`	COOP/COEP headers for `/dashboard`, `/room`, `/webcontainer` and worker scripts.
`.github/workflows/ci.yml`	CI: backend syntax check + tests; frontend lint + tests + build.

4.6 Key architectural decisions (ADR summary)

Decision	Why	Cost / trade-off
CRDT (Yjs) for text, not whole-file last-writer-wins	Correct merging of concurrent edits; offline-tolerant; tiny binary updates	Server must hold a doc in memory per active room; harder to scale horizontally

Decision	Why	Cost / trade-off
Redis as write buffer in front of S3	Keystrokes must not each become an S3 PUT (latency and cost)	A window of data exists only in Redis; durability depends on Redis persistence
Token bucket on S3 flushes	Bounds S3 writes per room even under heavy typing	Throttled flush means slightly older data in S3
Redis `SET NX EX` lock per room for persist	Manual save and background flusher must not interleave writes	Simple lock without fencing tokens (see Chapter 11)
Contents in S3, metadata in MongoDB	Right tool per data shape; Mongo's 16 MB document limit; S3 is cheap for bytes	Two writes per file; no atomic transaction across them
Firebase Auth + local JWT verification	No password storage; verification is a local signature check	Vendor lock-in; revocation needs extra work
Code runs in WebContainers (browser)	No server compute or sandboxing cost; per-user isolation for free	Node.js only; needs COOP/COEP; Chromium-centred support
AI through the backend	API key stays secret; rate limits per user	Adds a hop of latency

Interview kit · Architecture overview

SIMPLE EXPLANATION

The browser draws everything and lets you type; the server checks who you are, merges everyone's typing, and saves it. Fast temporary data is kept in Redis, files in S3, and the list of rooms and versions in MongoDB.

TECHNICAL EXPLANATION

Three tiers. A React SPA communicates via REST (Express 5) and Socket.IO with one Node.js process that holds an authoritative Yjs document per active room. Redis is the write buffer and coordination layer (hash per room, dirty set, token buckets, `SET NX EX` locks, rate-limit counters). A background interval flushes dirty rooms to S3 (contents) and MongoDB (metadata and snapshots). Identity is Firebase, verified locally with JWKS.

SAY IT LIKE THIS

> *"It's a three-tier system with a twist: both the browser and the server hold a CRDT replica of each open room. The browser applies your edits instantly and sends tiny Yjs updates over Socket.IO; the server merges them, relays them, and writes the result back to Redis. Persistence to S3 and MongoDB happens asynchronously in the background, throttled by a token bucket, so typing speed never depends on cloud latency."*

CODEFUSIONAI EXAMPLE

Pressing `Ctrl+S` emits `save-workspace`; the handler calls `persistWorkspaceToS3(roomId, true)`, which bypasses the token bucket but still takes the room lock, then broadcasts `workspace-saved` so every collaborator's status bar shows "Saved".

LIKELY FOLLOW-UPS

- **Why not microservices?** One team, one deployable, tight coupling between sockets and the CRDT. Microservices would add network hops and operational cost without a scaling need yet. The layering keeps the option open: AI or persistence could become separate workers.
- **Where is the single point of failure?** The single Node process (it holds the live Y.Docs) and Redis. Chapter 21 shows how to remove both.

- **Why is REST separate from sockets?** Different traffic shapes: one-off and cacheable versus continuous, bidirectional and tiny.

MISTAKES TO AVOID

- Drawing Redis as “the database”. It is a cache and coordination layer; S3 and Mongo are the system of record.
- Forgetting that sign-in goes browser → Firebase directly.

CHAPTER 5

Technology Choices and Alternatives

For every technology: what it is and how it works inside, why we picked it, the problem it solves, the alternatives, why our choice wins *in this context*, and its downsides. Interviewers love “why X over Y”, and the honest answer is always “for these constraints”.

5.1 React 19 (UI library)

What and how. React describes UI as a function of state. When state changes, React re-renders components into a virtual tree, diffs it against the previous one (reconciliation) and applies the minimal DOM changes. Hooks (`useState` , `useEffect` , `useMemo` , `useRef`) hold state and side effects inside function components.

Why here. The workspace is a dense, stateful UI: file tree, tabs, editor, panels, toasts, modals, live user lists. React's component model and huge ecosystem (`@monaco-editor/react` , react-markdown, lucide icons) made it the fastest route. Large refs such as the editor instance, cursors and widgets are kept in `useRef` so high-frequency events do not trigger re-renders.

Alternative	Why not here
Vue / Svelte	Equally capable; smaller ecosystem for Monaco and Yjs bindings; React matches the team's experience and the job market.
Angular	Heavier framework with more ceremony for a project this size.
Next.js	Server rendering adds nothing to an authenticated, highly interactive editor; it would complicate COOP/COEP and WebContainers.

Trade-offs. Re-render pitfalls (a `setState` on every cursor move would be costly), and a large client bundle. Monaco alone is several MB, so it is loaded only on the room route.

5.2 Vite 8 (build tool and dev server)

What and how. In development, Vite serves source files as native ES modules and transforms them on demand, so start-up is instant regardless of app size; hot module replacement swaps only the edited module. For production it bundles with Rollup (Rolldown in newer versions), splitting code into chunks.

Why here. Fast feedback, first-class worker support (Monaco's language workers are imported as `?worker`), and easy custom middleware. The `configureServer` hook adds the COOP/COEP headers the WebContainer routes need.

Alternatives. webpack / Create React App (CRA is deprecated and slower), Parcel, Turbopack. **Trade-off:** dev and prod use different pipelines, so rare “works in dev, breaks in build” bugs exist. CI runs `vite build` on every push to catch them.

5.3 Monaco Editor (code editor)

What and how. Monaco is the editor that powers VS Code, packaged for the browser. It keeps text in a *model* (one per file URI) and renders only the visible lines. Language features (completion, hover, rename, formatting, diagnostics) run in **web workers**: TypeScript/JavaScript, JSON, CSS and HTML each have a worker.

Why here. Developers already know its keyboard shortcuts and behaviour; it gives IntelliSense, multi-cursor, a minimap and a built-in `DiffEditor` (used for "compare with version") for free.

Alternative	Comparison
CodeMirror 6	Lighter, modular, excellent on mobile, with a first-class Yjs binding (y-codemirror.next). Monaco wins on out-of-the-box IntelliSense and familiarity; CodeMirror wins on bundle size and mobile.
Ace	Older, smaller ecosystem, weaker language services.

REAL BUG WE HIT

Monaco's right-click actions (Go to Symbol, Change All Occurrences, Format Document) silently did nothing. The cause: no `MonacoEnvironment.getWorker` was configured, so the language workers never started. Once they were configured, the worker scripts were then blocked by the isolated room page because they lacked COEP headers. The fix is `lib/monacoSetup.js` (Vite `?worker` imports) plus headers for worker files. Chapter 22 tells the story.

5.4 Yjs + y-monaco (CRDT)

What and how. Yjs is a CRDT library. A `Y.Doc` contains shared types (`Y.Map`, `Y.Text`, `Y.Array`). Every change produces a compact binary *update*; applying updates in any order, any number of times, yields the same document (updates are commutative and idempotent). `y-monaco`'s `MonacoBinding` mirrors a `Y.Text` and a Monaco model in both directions.

Why here. It solves the core problem: two people editing the same file at once. Our model is `doc.getMap("files")` → `Map<path, Y.Text>`, so every file is an independent CRDT text.

Alternative	Comparison
Operational Transformation (ShareDB, Google Docs style)	Needs a central server to transform concurrent operations against each other; correct OT is notoriously hard. CRDTs merge without transformation and tolerate offline edits.
Automerge	Also a CRDT, JSON-document oriented; Yjs is faster for text and has mature editor bindings.
Whole-file last-writer-wins (the original design)	Simple but loses edits whenever two people type at once. This was the bug that motivated Yjs.
Per-file locking ("only one editor at a time")	Correct but kills the collaboration experience; needs lock expiry and UI for waiting.

Trade-offs. Memory per document (deleted content leaves tombstones until garbage-collected), and the server holds docs in RAM, which is the main obstacle to multiple backend instances.

5.5 Socket.IO (real-time transport)

What and how. Socket.IO opens with an HTTP request (Engine.IO handshake), then upgrades to a WebSocket when possible, falling back to long-polling behind strict proxies. On top it adds events, rooms (`socket.join(roomId)`, `io.to(roomId).emit(...)`), acknowledgements, heartbeats, automatic reconnection with back-off, and binary payloads (our Yjs updates are `Uint8Array`).

Why here. We need server push, per-room broadcast and reconnection. Implementing those on raw WebSockets means rebuilding Socket.IO. Middleware (`io.use(socketAuth)`) gives one place for authentication.

Alternative	Comparison
HTTP polling / long polling	High latency or many wasted requests; no efficient push (see the polling figure in Chapter 3).
Raw WebSocket (`ws`)	Lighter and standard, but no rooms, acks or reconnection. y-websocket uses this route.
Server-Sent Events	Server → client only; edits still need separate POSTs.
Managed (Pusher, Ably, Liveblocks)	Less ops work, but cost, vendor lock-in, and less to learn and show.
WebRTC (y-webrtc)	Peer-to-peer and low server cost, but NAT traversal, no server authority and hard persistence.

Trade-offs. Its own protocol (clients must use the Socket.IO client); sticky sessions plus the Redis adapter are required for multiple nodes; `maxHttpBufferSize` is set to 1 GB for folder uploads, which is a memory-abuse risk.

5.6 Node.js 22 + Express 5 (server)

What and how. Node.js runs JavaScript on V8 with an event loop and non-blocking I/O, ideal for many concurrent, mostly idle connections. Express is a minimal HTTP framework: an ordered chain of middleware functions `(req, res, next)`. Express 5 adds native promise support, so a rejected promise from an `async` handler is forwarded to the error handler automatically.

Why here. One language end to end (shared mental model, and Yjs runs on both sides), excellent WebSocket support, and a huge library ecosystem.

Alternatives. NestJS (structure via decorators/DI, more boilerplate), Fastify (faster, schema-based validation), Go or Java (better CPU throughput, but Yjs and the client stack are JavaScript). **Trade-offs:** CPU-bound work blocks the loop; a thrown error in a callback can crash the process if not handled.

> **INTERVIEW NUGGET**
>
> The code wraps handlers in `asyncHandler`, which was needed in Express 4. In Express 5 rejected promises already reach the error handler, so the wrapper is now redundant but harmless. Mentioning this shows you understand the framework's version differences.

5.7 MongoDB + Mongoose (metadata store)

What and how. MongoDB stores BSON documents in collections, supports secondary and compound indexes, atomic single-document updates (`$addToSet`, `$set`), and replication. Mongoose adds schemas, validation, defaults, timestamps and a model API on top.

Why here. Our metadata is document-shaped and evolves (rooms gained `accessType`, `description`, `participants`; snapshots carry an array of file summaries). There are no complex joins: queries are "rooms where I'm owner or participant, newest first" and "snapshots for room R, by version", which compound indexes serve directly. Atlas gives a free managed cluster.

Alternative	Comparison
PostgreSQL	Strong relational integrity, transactions and JSONB; an equally valid choice. Mongo's flexible schema made iteration faster, and the relations here are shallow.
Firestore	Fits with Firebase, real-time listeners; but query limitations, per-read pricing and less control.
DynamoDB	Massive scale, but access patterns must be designed up front; less ad-hoc querying.

Trade-offs. There are no foreign keys, so integrity (for example deleting a room's files and snapshots) is enforced in application code (`roomService.deleteRoom`). Multi-document transactions exist but are not used here.

5.8 Redis + ioredis (cache and coordination)

What and how. Redis is an in-memory data-structure server. It executes commands one at a time on its main thread, so each command is **atomic**. That is why `INCR`, `SET NX` and `HINCRBY` are safe across processes. Optional persistence: RDB snapshots and/or an AOF (append-only file). ioredis is a Node client with auto-reconnect and pipelining.

Why here. Four jobs in one tool: the hot workspace cache (hash per room), coordination (a lock per room), throttling (token buckets, rate-limit counters) and change tracking (the dirty-rooms set). All need sub-millisecond operations that are atomic across processes.

Alternative	Comparison
In-process memory (Map)	Fastest, but lost on restart and not shared across instances. That is exactly what the `MockRedis` fallback is, which is fine for development only.
Memcached	Plain key-value only: no hashes, sets or `NX` semantics for locks.
MongoDB as the buffer	Disk-backed writes for every keystroke batch; more load on the system of record.

Trade-offs. RAM is expensive; data written since the last persistence point can be lost in a crash; Redis is another moving part (mitigated by the in-memory fallback, which itself weakens the guarantees).

5.9 Amazon S3 (object storage)

What and how. S3 stores objects (bytes + metadata) under keys in buckets. It is designed for 99.999999999% (eleven nines) durability by storing data redundantly across multiple Availability Zones, and since December 2020 it provides strong read-after-write consistency. Access is via HTTPS APIs (`PutObject`, `GetObject`, `ListObjectsV2`, `DeleteObject`) or time-limited **presigned URLs**.

Why here. Source files and snapshot JSON are blobs, read whole and written whole. S3 is cheap per GB, unlimited in size, extremely durable, and keeps large content out of MongoDB documents, which have a 16 MB limit.

Alternatives. MongoDB GridFS (one fewer service, but heavier DB load), Cloudflare R2 (S3-compatible, no egress fees), Google Cloud Storage, local disk (not durable, not shared). **Trade-offs:** each request costs tens of milliseconds and a per-request fee, hence the Redis buffer and token bucket.

5.10 Firebase Authentication + jose

What and how. Firebase hosts sign-up, sign-in (Google OAuth, email/password) and password reset. After sign-in the client holds an **ID token** (RS256 JWT, 1-hour lifetime) and a refresh token; the SDK refreshes the ID token automatically. Our server verifies the token with `jose`: it downloads Google's public keys (a JWKS) once, caches them, and checks the signature plus `iss = https://securetoken.google.com/<projectId>`, `aud = projectId` and expiry.

Why here. We never store passwords, get Google sign-in for free, and verification is a local cryptographic check with no network call per request. Using `jose` instead of `firebase-admin` means no service-account secret is needed on the server, only the public project ID.

Alternatives. Self-built auth (bcrypt + sessions + JWT: more control and more risk), Auth0 / Clerk / Supabase Auth, or `firebase-admin`'s `verifyIdToken` (also checks revocation if asked, but needs credentials). **Trade-offs:** vendor lock-in; a stolen token stays valid until it expires (up to 1 hour) unless you add revocation checks.

5.11 Google Gemini (AI)

What and how. A hosted large language model. We call `gemini-2.5-flash` through the `@google/generative-ai` SDK with a text prompt and receive markdown text.

Why here. Fast and inexpensive (Flash tier), generous free quota for development, a long context window that comfortably fits a 12,000-character file excerpt, and good code ability.

Alternatives. OpenAI GPT models, Anthropic Claude, self-hosted open models (Llama, Qwen via Ollama/vLLM: private, but GPU cost). **Trade-offs:** external dependency (quota and outages, mapped to friendly 503 errors), code leaves your infrastructure, and non-deterministic answers. Note that Google has since introduced the newer `@google/genai` SDK; migrating is a sensible maintenance item.

5.12 WebContainers + xterm.js (in-browser execution)

What and how. StackBlitz WebContainers run a Node.js-compatible runtime inside the browser using WebAssembly and a virtual file system, with a service worker that serves dev-server ports to an iframe. They need `SharedArrayBuffer`, so the page must be cross-origin isolated. xterm.js renders a terminal and pipes

keystrokes to the container's `jsh` shell.

Why here. Running untrusted user code on our server is the riskiest thing a code platform can do. WebContainers move execution into each user's own browser sandbox, at zero server cost.

Alternatives. Server-side containers per user (Docker/Firecracker: any language, but expensive and hard to secure), Judge0 or Piston APIs (good for single files and competitive programming), or the current unsandboxed `/api/execute` (not acceptable for production). **Trade-offs:** Node.js only, Chromium-centred support, the COOP/COEP requirement (which complicates OAuth popups and third-party embeds), and a commercial licence is required for some production uses (check StackBlitz's terms).

5.13 Tailwind CSS, Vitest, node:test, GitHub Actions

Tool	Why	Alternatives
Tailwind CSS 4 + tokens	Utility classes for speed; `tokens.css` keeps one consistent light palette	CSS Modules, styled-components, plain CSS
Vitest	Same config and transforms as Vite; Jest-compatible API	Jest
`node --test`	Built into Node 22, zero dependencies for backend tests	Jest, Mocha
GitHub Actions	Lives next to the code; free for public repositories	CircleCI, GitLab CI, Jenkins

5.14 Comparison cheat table

Question	Our choice	One-line justification
Real-time transport?	Socket.IO	Push + rooms + reconnection + binary out of the box
Conflict handling?	Yjs CRDT	Concurrent edits merge deterministically, no locks
Where do file bytes live?	S3	Durable, cheap, unlimited; avoids the 16 MB doc limit
Where does metadata live?	MongoDB	Flexible documents + compound indexes for our queries
Hot state and coordination?	Redis	Atomic in-memory ops shared across processes
Identity?	Firebase + jose	No passwords stored; local JWT verification
Code execution?	WebContainers	Sandboxed in the user's browser, zero server cost
AI?	Gemini 2.5 Flash via backend	Fast, cheap; key stays server-side

Interview kit · "Why did you choose X?"

SIMPLE EXPLANATION

Each tool was picked because it is the simplest thing that solves one specific problem well: Socket.IO for live updates, Yjs for merging edits, Redis for speed, S3 for storing files, MongoDB for lists of things.

TECHNICAL EXPLANATION

The choices follow the data's shape and access pattern. Small, frequent, bidirectional messages go over WebSockets; concurrent edits to a sequence need a CRDT; blobs that are read and written whole go to object storage; queryable metadata goes to an indexed document store; cross-process atomic counters and locks go to Redis.

SAY IT LIKE THIS

> *"I chose each piece by the shape of the data and how it's accessed. File contents are blobs we read and write whole, so S3, which is durable and cheap, beats putting them in MongoDB documents with a 16 MB limit. Metadata needs queries like 'rooms I own, newest first', so MongoDB with compound indexes. Keystrokes are too frequent for S3, so Redis buffers them, and because Redis commands are atomic I also use it for locks and rate limits."*

CODEFUSIONAI EXAMPLE

`RoomSchema.index({ participants: 1, lastActive: -1 })` makes the dashboard query "rooms I joined, most recent first" an index scan instead of a collection scan.

LIKELY FOLLOW-UPS

- **Would PostgreSQL work?** Yes, with tables `rooms`, `files`, `snapshots` and a `room_participants` join table. I would gain foreign keys and transactions. I chose Mongo for schema flexibility while iterating.
- **Why not store files in Redis permanently?** RAM cost, and Redis is not the durable system of record. It is a buffer with a 2-hour TTL.
- **Is Socket.IO a WebSocket?** It uses WebSockets as its main transport, but it is its own protocol with a fallback.

MISTAKES TO AVOID

- "Because it's popular" or "because it's fast" with no context.
- Claiming an alternative is "bad". Say when *it* would be the better choice.

PART III

Implementation Deep Dive

Module by module through the real code: server bootstrap, APIs, database, authentication, real-time collaboration, caching and locking, storage, history, AI, the frontend and code execution.

CHAPTER 6

Backend Walkthrough

This chapter reads `Backend/src/server.js` top to bottom, then the cross-cutting utilities every other module depends on. If you can explain this file line by line, you can explain how the backend starts, serves and shuts down.

6.1 server.js: the composition root

Backend/src/server.js (abridged)

```
dotenv.config();                                  // 1. load .env into process.env
const app = express();
app.use(cors({                                    // 2. CORS: which browser origins may call us
  origin: (origin, callback) => callback(null, true),
  methods: ["GET", "POST", "PUT", "DELETE", "PATCH", "OPTIONS"],
  credentials: true,
}));
app.use(express.json({ limit: "50mb" }));         // 3. parse JSON bodies (large folder uploads)
connectDB();                                      // 4. Mongo connection (async, not awaited)

app.use("/api/ai", aiRoutes);                     // 5. mount routers
app.use("/api/filesystem", fileSystemRoute);
app.use("/api/execute", executeCodeRoute);
app.use("/api/rooms", roomRoutes);
app.use("/api/users", userRoutes);
app.use("/api/files", fileRoutes);
app.get("/", (req, res) => res.json({ status: "ok", ... }));  // 6. health check
app.use(errorHandler);                            // 7. MUST be last

const httpServer = createServer(app);             // 8. one HTTP server for both
const io = new Server(httpServer, { cors: {...}, maxHttpBufferSize: 1e9 });
io.use(socketAuth);                               // 9. every socket must be authenticated
registerSocketHandlers(io);                       // 10. all socket events
startWorkspaceSyncInterval(30000);                // 11. background flusher every 30 s
httpServer.listen(PORT);                          // 12. listen on 5000
process.on("SIGTERM", () => shutdown("SIGTERM")); // 13. graceful shutdown
```

1. **dotenv** reads `Backend/.env` (Mongo URI, Redis URL, AWS keys, Gemini key, Firebase project ID). Secrets never live in code. `.env` is git-ignored and `.env.example` documents the variable names.
2. **CORS** is a browser rule: a page from origin A may call origin B only if B's response allows it. The callback `callback(null, true)` reflects *any* origin, with `credentials: true`. This is convenient for preview deployments, but in production you should allow-list your frontend origins (Chapter 18). Because auth uses a Bearer header rather than cookies, the practical risk is lower than it looks.
3. **Body parser** with a 50 MB limit allows large uploads, but also lets a client send big bodies. A tighter per-route limit is a hardening item.
4. **connectDB** is called without `await`; Mongoose buffers queries until it connects. If the URI is missing it logs a warning; if the connection fails it exits the process, which is the right “fail fast” behaviour.

5. **Routers** are mounted under prefixes, so each router file is small and focused.
6. **Health check** at `/` returns JSON for load-balancer probes and uptime monitors.
7. **Error handler last.** Express identifies error middleware by its four parameters `(err, req, res, next)`; it must be registered after the routes so errors from them reach it.
8. **Express and Socket.IO share one HTTP server** and port. The WebSocket upgrade request arrives on the same port as REST.
9. `io.use(socketAuth)` runs before any connection is accepted; a socket without a valid token never reaches the handlers.
10. **Socket handlers** live in `controllers/socketController.js` (Chapter 10).
11. **Background flusher.** Every 30 s it scans `dirty_rooms` and flushes rooms that reached the edit threshold or have been idle for 10 s (Chapter 11).
12. **Listen** on `PORT` (default 5000).
13. **Graceful shutdown** on `SIGTERM` / `SIGINT`: stop the interval, stop accepting connections, exit; force-exit after 10 s.

GAP WORTH MENTIONING

The shutdown handler does **not** flush dirty rooms to S3 before exiting. Their latest edits remain in Redis (and are picked up after restart if Redis survived). With the in-memory Redis fallback, though, they would be lost. Adding "flush all dirty rooms, then exit" is an easy, high-value improvement.

6.2 Configuration modules

File	What it does	Interview angle
`config/db.js`	`mongoose.connect(MONGO_URI)`; exits on failure	Fail fast when the system of record is unavailable
`config/redis.js`	`RedisWrapper` around ioredis. On a connection error it switches permanently to `MockRedis`, an in-memory implementation of the same commands (get/set NX EX, incr, hashes, sets, TTL)	Graceful degradation for development; dangerous in production because state is no longer shared or durable
`config/aws.js`	One shared `S3Client` with region and credentials from env	Reuse clients (connection pooling); prefer IAM roles over static keys in production
`config/gemini.js`	`new GoogleGenerativeAI(apiKey)`; warns if the key is empty	Key only on the server; never shipped to the browser

Backend/src/config/redis.js (abridged)

```
this.client = new Redis(url, {
  maxRetriesPerRequest: 1,
  retryStrategy: (times) => {
    if (times > 1) { this.switchToMock(); return null; }  // give up after one retry
    return 50;                                             // retry once after 50 ms
  },
});
this.client.on("error", () => this.switchToMock());
```

Line by line: `maxRetriesPerRequest: 1` makes a command fail quickly instead of hanging; `retryStrategy` returns a delay in milliseconds, or `null` to stop reconnecting; on the second failure the wrapper swaps in `MockRedis`. Every caller uses the wrapper, so the swap is invisible to the rest of the code. That is the **adapter/proxy pattern**.

6.3 Error handling: AppError, asyncHandler, errorHandler

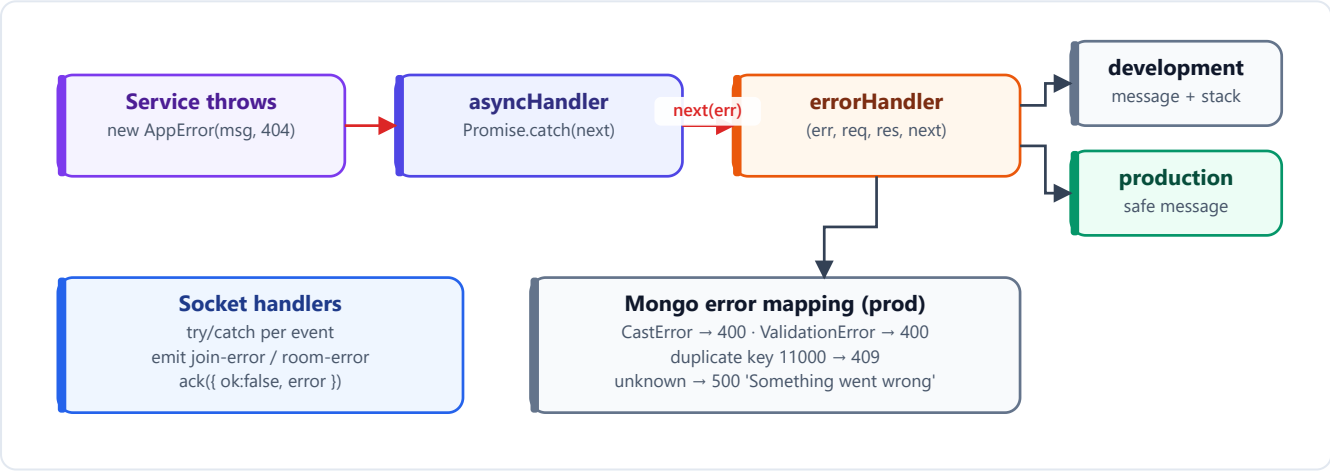


Figure 7. How an error travels from a service to a JSON response, and how socket handlers report errors.

Backend/src/utils/AppError.js

```
class AppError extends Error {
  constructor(message, statusCode) {
    super(message);
    this.statusCode = statusCode;
    this.status = String(statusCode).startsWith("4") ? "fail" : "error";
    this.isOperational = true;          // expected error: safe to show its message
    Error.captureStackTrace(this, this.constructor);
  }
}
```

The key idea is the split between **operational errors** (expected: "room not found", "not the owner", "too many requests"), whose message is safe to show, and **programmer errors** (bugs: `undefined is not a function`), whose details must be hidden. In production, `errorHandler` returns the message only when `isOperational` is true, and otherwise returns a generic 500 and logs the error. It also maps Mongoose errors: `CastError` → 400, `ValidationError` → 400, duplicate key (code 11000) → 409.

Backend/src/utils/asyncHandler.js

```
const asyncHandler = (fn) => (req, res, next) =>
  Promise.resolve(fn(req, res, next)).catch(next);
```

This wraps an async controller so a rejected promise calls `next(err)`. In Express 4 this was mandatory, because unhandled rejections were not caught. In Express 5 it is redundant but harmless.

Sockets are different. There is no `next(err)` pipeline for socket events, so each handler has its own `try/catch` and reports with an event (`join-error`, `room-error`) or an acknowledgement callback (`reply({ ok: false, error })` in `restore-version`).

6.4 Validation middleware

Backend/src/middleware/validate.js (usage)

```
router.post("/chat", validate({ body: { message: "required|string" } }), chatWithAI);
```

`validate()` is a tiny, dependency-free factory: each field gets pipe-separated rules (`required`, `string`, `number`, `boolean`, `array`), and failures become one `AppError(400)` listing every problem. It checks presence and type, not length, format or nesting. A schema library such as Zod or Joi would add length limits, enums and nested objects. That is a good "what would you improve" answer.

6.5 Utilities: Redis key registry

`utils/redisKeys.js` centralises every key pattern and tuning constant, so no file contains a magic string like `"room:" + id + ":files"`. It also defines the TTLs and the token-bucket configuration:

Backend/src/utils/redisKeys.js

```
export const TOKEN_BUCKET_CONFIG = {
  flushEditThreshold: 10,      // edits before an automatic flush
  idleFlushTimeoutMs: 10000,   // or this long idle with pending edits
  bucketCapacity: 5,           // burst of S3 flushes
  refillRateMs: 5000,          // +1 flush token every 5 s
};
export const REDIS_TTL = { workspace: 7200, persistLock: 60, tokenBucket: 86400 };
export const encodeFilePath = (p) => Buffer.from(p).toString("base64");
```

Why base64 for hash fields? Paths can contain characters such as `:`, spaces or Unicode. Encoding gives a safe, reversible field name.

Interview kit · Backend structure and error handling

SIMPLE EXPLANATION

`server.js` plugs everything together: security checks first, then the URL handlers, then a final catch-all that turns any error into a clean message.

TECHNICAL EXPLANATION

Layered Express app with ordered middleware (CORS → JSON → per-route auth, rate limit and validation → controller), centralised error handling distinguishing operational from programmer errors, and an HTTP server shared with Socket.IO, which has its own auth middleware. Background work (the flusher) is started and stopped explicitly in the composition root.

SAY IT LIKE THIS

> *"The backend is layered: routes, middleware, thin controllers, services, and models. Errors are thrown as an AppError with a status code from anywhere in a service; they bubble up to one error-handling middleware that returns a safe JSON message. In production it hides stack traces and unknown errors, and maps Mongo errors like duplicate keys to 409."*

CODEFUSIONAI EXAMPLE

Deleting someone else's room: `roomService.deleteRoom` throws `AppError("Unauthorized: Only the room owner can delete this session.", 403)`; `asyncHandler` forwards it; `errorHandler` responds `403 { success:false, message }`; the dashboard shows it in a toast.

LIKELY FOLLOW-UPS

- **Why must the error handler be last?** Express runs middleware in order; only errors from earlier middleware reach it.
- **What happens on an uncaught exception in a socket callback?** Without try/catch, an exception inside an async socket handler becomes an unhandled rejection. Node 15+ crashes the process by default, which drops every room. That is why every handler wraps its body. A process manager (PM2, a container orchestrator) should restart it.
- **Why does Mongo connect failure exit?** Running without the system of record would serve wrong answers; failing fast lets the orchestrator restart and alert.

MISTAKES TO AVOID

- Saying CORS protects the API from attackers. It is a browser policy, and curl ignores it; authentication is the real protection.
- Returning raw error objects or stack traces to clients in production.

CHAPTER 7

API Design and Request Lifecycle

7.1 Endpoint reference

Method	Path	Guards	Purpose
GET	`/api/rooms/public`	none	Active public rooms for the hub (limited fields: no owner UID or participants)
GET	`/api/rooms/user/:userId`	auth, requireSelf	Rooms I own or joined, newest first
POST	`/api/rooms/:roomId/close`	auth, owner check	Close a room (no one can join)
POST	`/api/rooms/:roomId/resume`	auth, owner check	Reopen a closed room
DELETE	`/api/rooms/:roomId`	auth, 20/h, owner check	Delete room, files, S3 objects, snapshots, cache
GET	`/api/rooms/:roomId/history`	auth	List versions (metadata only)
GET	`/api/rooms/:roomId/history/:version`	auth, 60/min	Full files of one version
GET	`/api/users/profile/:userId`	none	Display name and photo (defaults if missing)
POST	`/api/users/profile`	auth, 20/min	Upsert my profile (uid taken from the token)
POST	`/api/ai/chat`	auth, 30/min, validate	General chat; the room panel and Assistant use this
POST	`/api/ai/review|explain|optimize`	auth, 30/min, validate	Structured prompts (not used by the current UI)
POST	`/api/files/upload-url` · `register` · `folder`	auth, 120/min	Presigned upload, metadata registration, folder record
GET/PUT/DELETE	`/api/files/room/:roomId`, `/download/:id`, `/:id`	auth, 120/min	List, presigned download, rename, delete
POST	`/api/filesystem/folder`	auth, path guard	Legacy: creates a temp folder on the server disk
POST	`/api/execute`	auth, 10/min	Runs JS/Python/C++/Java on the host (unsandboxed)
GET	`/`	none	Health check

AUTHORISATION GAPS TO KNOW ABOUT

The history endpoints and every `/api/files` endpoint require a signed-in user but do **not** check that the user belongs to the room. A user who guesses a room ID or file ID could read history or delete a file record. This is an *IDOR* (Insecure Direct Object Reference). The fix is a reusable `requireRoomMember` middleware that loads the room and checks `ownerId === uid || participants.includes(uid)`.

7.2 Socket.IO event reference

Event	Direction	Payload and behaviour
`create-room`	client → server	`{ roomId, roomName, username, template, files, accessType, description }`. The owner is taken from the token. Creates the Room, uploads template files, seeds the cache.
`join-room`	client → server	`{ roomId, username, photoURL }`. Checks the room exists and is not closed, adds the participant, then sends `yjs-sync`, `receive-code`, `room-users`, `room-joined`.
`yjs-update`	both	`{ roomId, update: Uint8Array }`. Members only; malformed updates are rejected; relayed to the others; schedules write-back.
`code-change`	client → server	Full file list for *structural* changes (create/delete/rename/upload); reconciled with the CRDT; cached; counted as an edit token.
`receive-code`	server → client	File tree JSON (with CRDT contents merged in)
`cursor-change` / `cursor-update`	both	Cursor position and active file, relayed to the others in the room
`update-presence` / `room-users`	both	Active file per user; broadcast list of users
`save-workspace` / `workspace-saved`	both	Manual save (bypasses the bucket, takes the lock) → `{ success, timestamp, reason }` to everyone
`restore-version` (ack)	client → server	`{ version }`; replaces the workspace and CRDT; persists as a new version; acknowledges `{ ok }`; emits `version-restored`
`close-room` / `room-closed`	both	Owner closes; everyone is told to leave
`disconnecting`	internal	Updates the user list; if the room is empty, schedules dropping the Y.Doc after 5 minutes

7.3 The life of a REST request

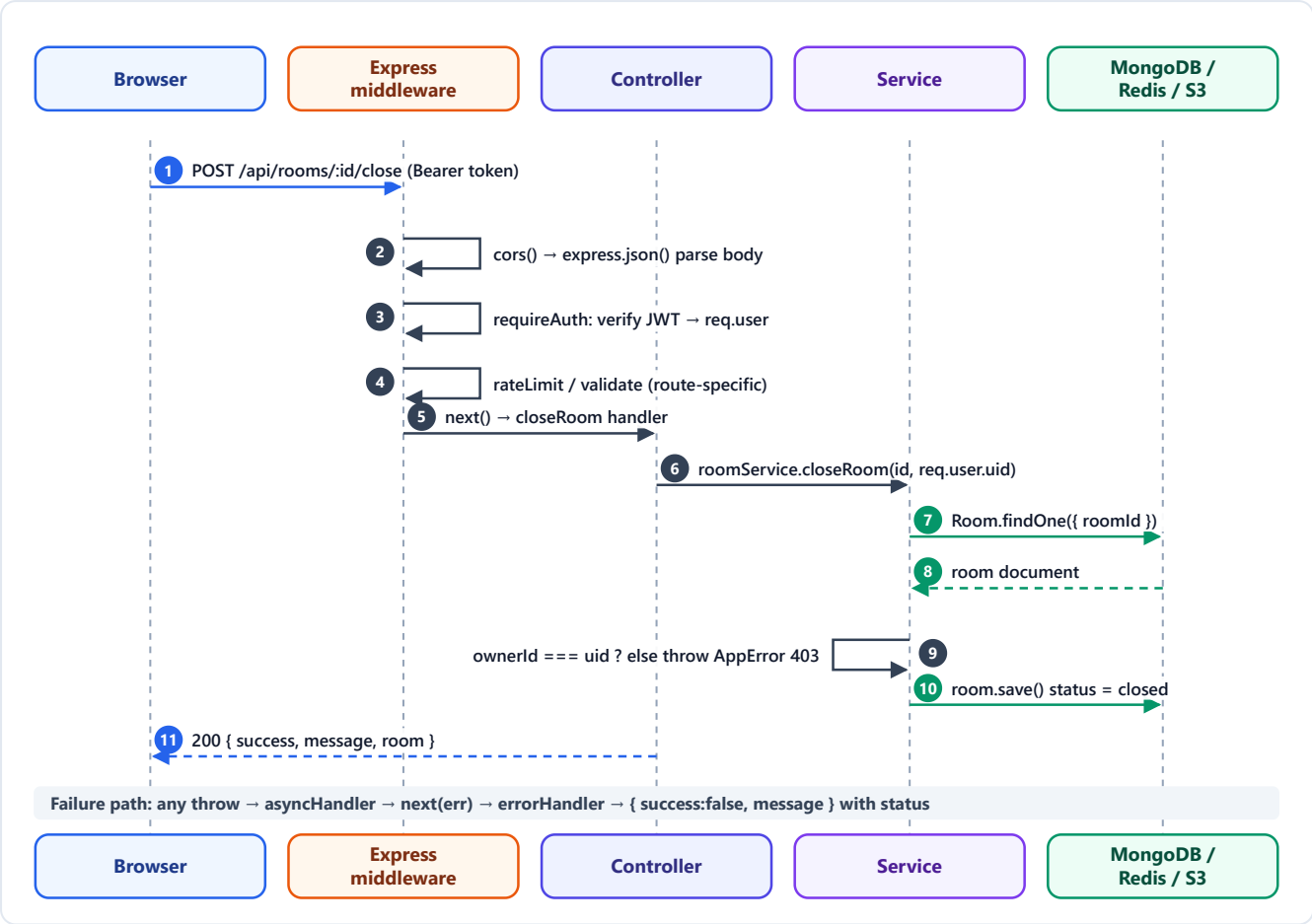


Figure 8. Lifecycle of POST /api/rooms/:id/close, from browser to MongoDB and back, including the failure path.

1. The dashboard calls `apiFetch("/api/rooms/cfai-ab12-cd34/close", { method: "POST" })`, which attaches `Authorization: Bearer <ID token>`.
2. Express runs global middleware: CORS headers, then JSON body parsing.
3. The route's `requireAuth` extracts the Bearer token, verifies it with `jose`, and sets `req.user = { uid, email, name }`. An invalid token produces `next(AppError 401)`.
4. Route-specific guards (rate limit, validation) run if configured.
5. The controller `closeRoom` reads `req.params.roomId` and `req.user.uid`; it never trusts a user ID from the body.
6. The service loads the room with `Room.findOne({ roomId })`.
7. Authorisation: if `room.ownerId !== uid` it throws `AppError 403`.
8. It sets `status = "closed"` and calls `room.save()`.
9. The response `200 { success: true, message, room }` goes back to the browser.
10. On any failure the error travels `asyncHandler → errorHandler` and comes back as `{ success:false, message }` with the right status.

7.4 API design principles used (and where they bend)

- **Resource-oriented URLs**: `/rooms/:roomId/history/:version`. Actions that are not CRUD (`close`, `resume`) are modelled as sub-resource POSTs, a common pragmatic choice.
- **Identity from the token, never the body**: `ownerId` in `create-room` and `uid` in profile updates come from `socket.user` / `req.user`. The code comment says this explicitly.
- **Consistent error shape**: `{ success:false, status, message }` with HTTP status codes 400/401/403/404/409/429/500/503.
- **Standard rate-limit headers**: `RateLimit-Limit`, `RateLimit-Remaining`, `RateLimit-Reset`, plus `Retry-After` on 429.
- **Where it bends:** some successful responses are bare arrays (kept for frontend compatibility, as the controller comments explain), there is no API versioning (`/api/v1`), and there is no pagination on room lists or history (history is capped at 50).

Interview kit · API design

SIMPLE EXPLANATION

Every URL is a "thing" (room, file, version). Before the handler runs, the request passes a few checkpoints: are you signed in, are you sending too fast, is the data well-formed.

TECHNICAL EXPLANATION

REST resources with method semantics, a middleware chain per route (`requireAuth` → `rateLimit` → `validate`), identity derived from the verified JWT, owner checks in the service layer, and a centralised error envelope with meaningful status codes.

SAY IT LIKE THIS

> *"Each route declares its guards in order: authentication, rate limit, validation. Controllers never trust IDs from the request body. The user ID always comes from the verified token, and ownership is checked in the service before any write. Errors come back in one consistent shape with proper status codes, including 429 with Retry-After for rate limits."*

CODEFUSIONAI EXAMPLE

`router.get("/user/:userId", requireAuth, requireSelf("userId"), ...)`: `requireSelf` returns 403 if the `:userId` in the URL is not the caller's own UID, so you cannot list someone else's rooms.

LIKELY FOLLOW-UPS

- **401 versus 403?** 401: no or invalid credentials. 403: valid identity, not allowed.
- **How would you version the API?** URL prefix `/api/v1`, or a header; keep v1 alive while clients migrate.
- **Idempotency?** GET, PUT and DELETE should be idempotent; close and resume are naturally idempotent (setting a status twice is harmless).
- **How would you add pagination?** Cursor-based on `(lastActive, _id)` using the existing compound index.

MISTAKES TO AVOID

- Claiming every route checks room membership. History and files do not.
- Confusing validation (shape of input) with authorisation (permission).

CHAPTER 8

Database Design

8.1 Schema and relationships

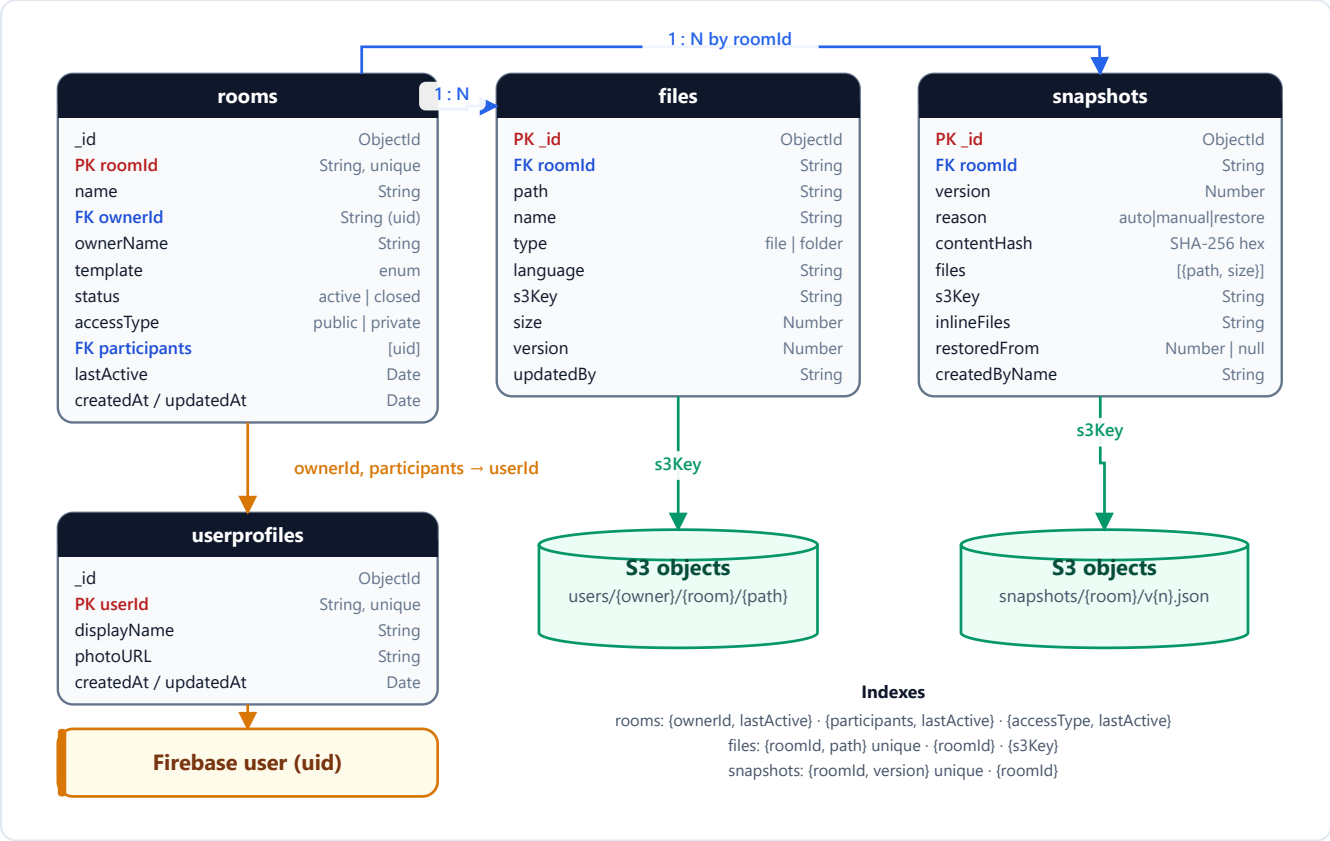


Figure 9. Collections, key fields, relationships and indexes. Contents live in S3; documents point to them via s3Key.

Reading the diagram:

- **rooms**: one document per room, identified by the human-friendly `roomId` (unique index), not by `_id`. `ownerId` and `participants` hold Firebase UIDs, a logical foreign key to an identity that lives outside our database.
- **files**: one document per file or folder. The compound unique index `{ roomId, path }` guarantees a path appears once per room and makes "all files of a room" an index range scan. `s3Key` points to the content; `s3Key` is *required only when* `type === "file"` (a conditional validator).
- **snapshots**: one document per saved version, unique on `{ roomId, version }`. Holds summary metadata (`files: [{ path, size }]`, totals, hash), so the History panel lists versions without downloading content.
- **userprofiles**: display name and photo keyed by Firebase UID.

8.2 The Mongoose models in code

Backend/src/models/Room.js (abridged)

```
const RoomSchema = new mongoose.Schema({
  roomId:     { type: String, required: true, unique: true, trim: true },
  name:       { type: String, default: "Untitled Project", trim: true },
  ownerId:    { type: String, required: true, trim: true },
  template:   { type: String, default: "react", enum:
["react","node","python","cpp","java","html","blank"] },
  status:     { type: String, enum: ["active", "closed"], default: "active" },
  accessType: { type: String, enum: ["public", "private"], default: "private" },
  participants: [{ type: String }],
  lastActive: { type: Date, default: Date.now },
}, { timestamps: true });                       // adds createdAt / updatedAt

RoomSchema.index({ ownerId: 1, lastActive: -1 });       // "my rooms, newest first"
RoomSchema.index({ participants: 1, lastActive: -1 });  // multikey: "rooms I joined"
RoomSchema.index({ accessType: 1, lastActive: -1 });    // public hub
```

Line-by-line points. `enum` rejects invalid values at write time. `trim` normalises strings. `timestamps` maintains `createdAt/updatedAt`. The index on `participants` is a **multikey index**: MongoDB indexes every element of the array, so `{ participants: uid }` is an index lookup.

Backend/src/models/Snapshot.js (abridged)

```
const SnapshotSchema = new mongoose.Schema({
  roomId: { type: String, required: true, index: true },
  version: { type: Number, required: true },
  reason: { type: String, enum: ["auto", "manual", "restore"], default: "auto" },
  contentHash: { type: String, required: true },   // SHA-256: skip identical versions
  files: [{ path: String, size: Number, _id: false }],
  s3Key: { type: String, default: "" },
  inlineFiles: { type: String, default: "" },      // fallback when S3 is unavailable
  restoredFrom: { type: Number, default: null },
}, { timestamps: true });
SnapshotSchema.index({ roomId: 1, version: -1 }, { unique: true });
```

8.3 Query patterns and why each index exists

Query (from the code)	Index used
`Room.find({ $or: [{ ownerId }, { participants: uid }] }).sort({ lastActive: -1 })`	Both branches of `$or` can use their own index (`ownerId_1_lastActive_-1`, `participants_1_lastActive_-1`)
`Room.find({ accessType: "public", status: "active" }).sort({ lastActive: -1 })`	`accessType_1_lastActive_-1` (status filtered after)
`File.find({ roomId })`, `File.findOne({ roomId, path })`	`roomId_1_path_1` (prefix serves roomId-only queries too)
`Snapshot.findOne({ roomId }).sort({ version: -1 })`	`roomId_1_version_-1`: the newest version is the first index entry

Query (from the code)	Index used
`Snapshot.find({ roomId }).sort({ version:-1 }).skip(50)` (prune)	Same index

INDEX NUGGET

Because `{ roomId: 1, path: 1 }` is a compound index, the separate `{ roomId: 1 }` index on files is redundant. Any query on `roomId` alone can use the compound index's prefix. Dropping redundant indexes saves RAM and speeds up writes.

8.4 Design decisions and normalisation

- **Referencing, not embedding, files in rooms.** Originally the whole workspace lived in one `Room.files` JSON string. The legacy fallback in `loadWorkspace` still migrates it. Embedding hits the 16 MB document limit, rewrites the whole room on every save, and causes write contention. Separate `files` documents let each file update independently.
- **Participants embedded as an array.** Fine for small groups (bounded by people per room); `$addToSet` makes joins idempotent. For thousands of members per room, a separate membership collection would be better.
- **Denormalised** `ownerName` avoids a lookup when listing public rooms; the trade-off is staleness if the owner renames themself.
- **No cross-collection transactions.** Deleting a room runs `Promise.allSettled` over S3, snapshots, files and cache, then deletes the room. A partial failure can leave orphans. A periodic cleanup job, or a Mongo transaction for the DB parts, would tighten this.

Interview kit · Database design

SIMPLE EXPLANATION

MongoDB keeps the “index cards”: which rooms exist, which files each room has, and which versions were saved. The actual file text lives in S3, and each card says where to find it.

TECHNICAL EXPLANATION

Four collections; logical foreign keys by `roomId` and Firebase UID; compound indexes matched to each query's equality-then-sort shape; uniqueness constraints that protect invariants (`{roomId, path}` , `{roomId, version}`); content externalised to S3 to stay far from the 16 MB BSON limit and keep documents small.

SAY IT LIKE THIS

> *“I designed indexes from the queries backwards. The dashboard asks for rooms I own or joined, sorted by last activity, so there are compound indexes ownerId plus lastActive and participants plus lastActive, and the second one is a multikey index over the array. Files are unique per room and path, which both enforces the invariant and serves the ‘load a room’ query. File content isn't in Mongo at all, only an s3Key, so documents stay tiny.”*

CODEFUSIONAI EXAMPLE

Version numbering: `createSnapshot` reads the latest version through `{roomId:1, version:-1}` and inserts `version + 1`. The unique index means that even if two processes raced, one insert would fail with a duplicate key rather than create two "v13" documents.

LIKELY FOLLOW-UPS

- **Why not store code in MongoDB?** 16 MB document limit, heavy documents slow every query and replication, and S3 is cheaper and more durable for blobs. GridFS exists, but adds DB load.
- **How do you avoid N+1 queries when loading a room?** One `File.find({ roomId })` returns all metadata; contents come from S3. Those S3 reads are currently sequential, so I would parallelise them with a concurrency limit.
- **Sharding key?** `roomId` (hashed): every query is scoped to a room, so queries stay on one shard.

MISTAKES TO AVOID

- Saying "MongoDB has no schema". Mongoose enforces one at the application level.
- Forgetting that an index costs memory and slows writes.

CHAPTER 9

Authentication and Authorisation

9.1 The complete flow

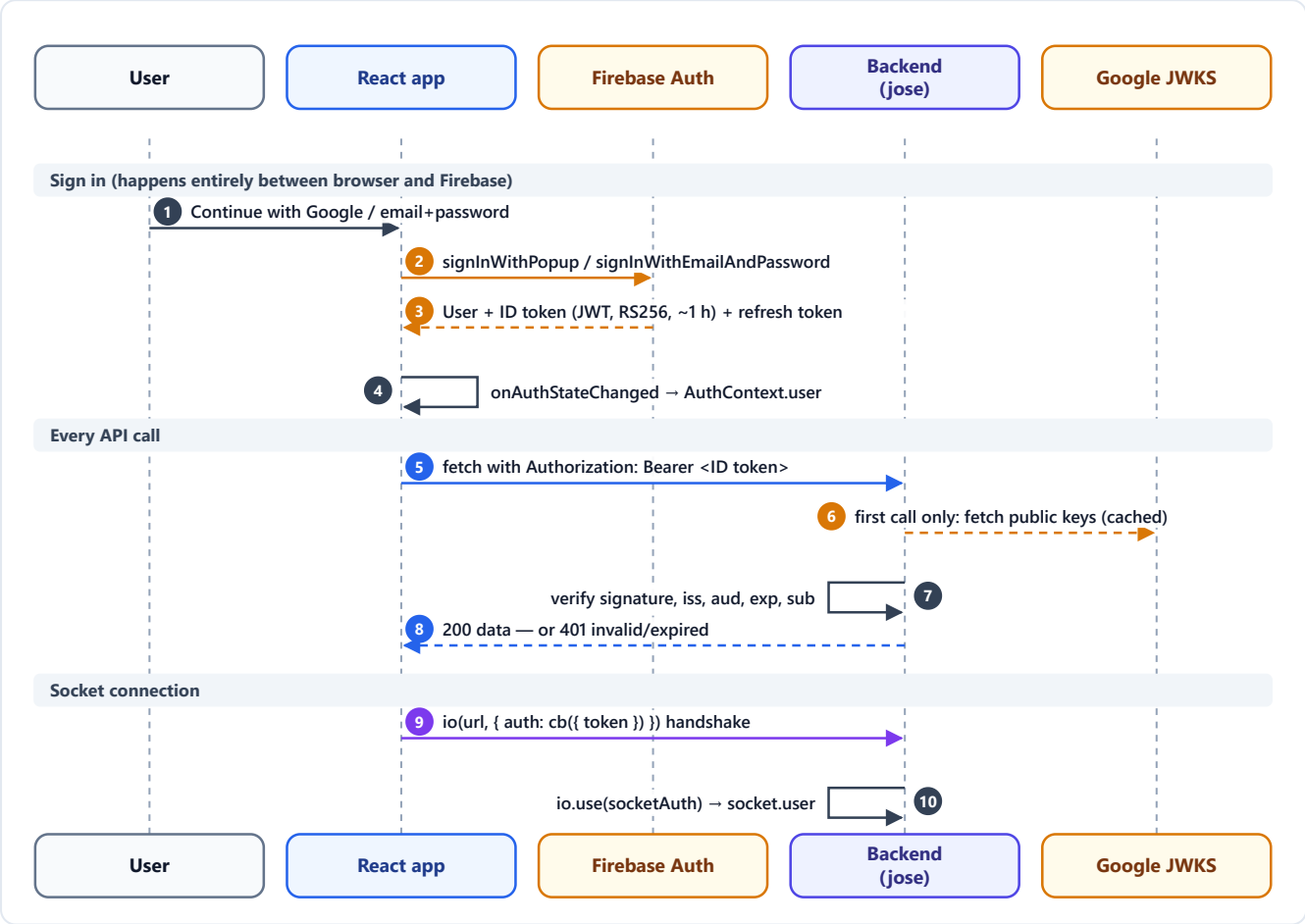


Figure 10. Sign-in with Firebase, then token verification on every REST call and once per socket connection.

1. The user picks Google or email/password on `/login`.
2. The Firebase SDK talks to Firebase directly (`signInWithPopup` or `signInWithEmailAndPassword`). For a new account, `registerUser` also calls `updateProfile` to store the display name, and the app POSTs it to `/api/users/profile` so collaborators see the name.
3. Firebase returns the user, a short-lived **ID token** (JWT, about 1 hour) and a long-lived refresh token.
4. `AuthContext` listens with `onAuthStateChanged` and exposes `user` and `loading` to the app; `ProtectedRoute` redirects to `/login` when there is no user.
5. Each API call goes through `apiFetch`, which calls `getIdToken()`; the SDK returns a cached token or silently refreshes it.
6. On the first verification, `jose` downloads Google's JWKS and caches it, refreshing when keys rotate.
7. Verification is a local RS256 signature check plus claim checks.
8. The handler runs with `req.user`; failures return 401 “Invalid or expired session”.

9. Sockets send the token in `handshake.auth`.
10. `io.use(socketAuth)` verifies it once per connection and stores `socket.user`.

9.2 The verification code, line by line

Backend/src/middleware/auth.js

```
const JWKS_URL =
"https://www.googleapis.com/service_accounts/v1/jwk/securetoken@system.gserviceaccount.com";
const jwks = createRemoteJWKSet(new URL(JWKS_URL));     // fetched lazily, cached, rotated

export const verifyIdToken = async (token) => {
  const projectId = process.env.FIREBASE_PROJECT_ID;
  if (!projectId) throw new AppError("Server auth is not configured ...", 500);
  if (!token) throw new AppError("Authentication required.", 401);
  try {
    const { payload } = await jwtVerify(token, jwks, {
      issuer: `https://securetoken.google.com/${projectId}`,   // minted by OUR Firebase project
      audience: projectId,                                      // intended for OUR app
    });                                                         // also checks exp / nbf
    if (!payload.sub) throw new Error("Token has no subject");
    return { uid: payload.sub, email: payload.email, name: payload.name };
  } catch {
    throw new AppError("Invalid or expired session. Please sign in again.", 401);
  }
};
```

- **Why check `iss` and `aud`?** Every Firebase project's tokens are signed with the same Google keys. Without the audience check, a token from *someone else's* Firebase project would pass the signature check. This is the most common JWT-verification mistake.
- **Why a generic 401 message?** Do not tell attackers whether the signature, issuer or expiry failed.
- **Misconfiguration is a 500.** A missing project ID is the server's fault, not the user's, and a test asserts this.

Backend/src/middleware/auth.js

```
export const requireSelf = (param = "userId") => (req, res, next) => {
  if (req.params[param] !== req.user?.uid) return next(new AppError("You can only access your own
data.", 403));
  next();
};
export const socketAuth = async (socket, next) => {
  try { socket.user = await verifyIdToken(socket.handshake.auth?.token); next(); }
  catch (err) { const e = new Error(err.message); e.data = { status: err.statusCode || 401 };
next(e); }
};
```

9.3 Authorisation model

Action	Rule enforced	Where
List my rooms	URL user = token user	`requireSelf`

Action	Rule enforced	Where
Close / reopen / delete room	Caller is the owner	`roomService` (`ownerId !== userId` → 403); socket `close-room` too
Send edits, cursors, presence	Socket has joined that room	`if (!socket.rooms.has(roomId)) return;`
Restore a version	Socket is in the room	`restore-version` handler
Join a room	Room exists and is not closed	`join-room` (private rooms *not* enforced)
Update profile	Only your own (uid from token)	`upsertProfile`

KNOWN AUTHORISATION GAPS (STATE THEM BEFORE THE INTERVIEWER FINDS THEM)

- **Private rooms are not enforced on join.** The room ID works like a capability: anyone signed in who has it can join. The fix: on join, allow only the owner, existing participants or invited users (an `invites` list or signed invite links) when `accessType === "private"`.
- `create-room` **with an existing ID** calls `socket.join(roomId)` before checking anything, so a client that knows an ID can enter the broadcast group without the join checks. Fix: if the room already exists, reject or route through the join logic.
- **History and file routes** lack a membership check (IDOR), as covered in Chapter 7.
- **Room IDs** come from `Math.random` ($36^8 \approx 2.8 \times 10^{12}$ possibilities). That is unguessable in practice, but not cryptographically random; `crypto.randomUUID()` is better.
- **Token revocation.** A token stays valid until it expires (up to 1 hour) even after sign-out or a password change. `firebase-admin` 's `verifyIdToken(token, true)` can check revocation for sensitive actions.

9.4 Session management

There is no server-side session store. The session *is* the Firebase token pair held by the SDK in the browser (IndexedDB). This is **stateless authentication**: any backend instance can verify any request without shared session state, which helps horizontal scaling. Sign-out (`signOut(auth)`) clears the local tokens; the socket disconnects when the page leaves the protected area.

Interview kit · Authentication and authorisation

SIMPLE EXPLANATION

Firebase checks your password and hands your browser a signed pass. Our server checks the pass's signature on every request; then it checks whether this pass is allowed to do the specific thing, for example only the owner can delete a room.

TECHNICAL EXPLANATION

Firebase ID tokens (RS256 JWTs) are verified locally with `jose` against a cached remote JWKS, validating signature, issuer, audience and expiry. Identity is attached to `req.user` / `socket.user` ; authorisation is enforced in middleware (`requireSelf`), in services (owner checks) and in socket handlers (room membership via `socket.rooms`).

SAY IT LIKE THIS

> *"We don't store passwords at all. Firebase handles sign-in and gives the browser a one-hour JWT. The backend verifies it locally with Google's public keys: signature, issuer, audience and expiry, so there is no network call per request. The socket is authenticated once at the handshake. Then authorisation is separate: the user ID always comes from the token, owner-only actions check ownerId in the service, and socket events are ignored unless the socket has joined that room."*

CODEFUSIONAI EXAMPLE

An attacker sends `POST /api/users/profile` with `{ userId: "victim", displayName: "hacked" }`. Nothing happens to the victim: the controller ignores the body's userId and uses `req.user.uid`.

LIKELY FOLLOW-UPS

- **Why JWT and not sessions?** Stateless verification scales horizontally and works the same for REST and sockets. Downside: revocation is hard, so keep tokens short-lived.
- **What if the JWKS endpoint is down?** `jose` caches keys, so verification continues; only a key rotation during an outage would fail.
- **Where is the token stored, and what about XSS?** The Firebase SDK keeps it in IndexedDB. XSS could steal it, so the defences are no `dangerouslySetInnerHTML` with user content, React's escaping, and a Content Security Policy.
- **How does a long-lived socket handle token expiry?** It is verified only at connect time. A connection can outlive the token; for strict setups, re-verify on reconnect (already happens) or disconnect sockets periodically.

MISTAKES TO AVOID

- Saying "JWTs are encrypted". They are signed and base64-encoded, so anyone can read the payload.
- Forgetting the audience check, or trusting a userId sent in the request body.

CHAPTER 10

Real-Time Collaboration: Socket.IO and Yjs

This is the heart of the project and the part interviewers dig into most: how a keystroke reaches every collaborator, and why two people typing at once never lose each other's work.

10.1 Joining a room

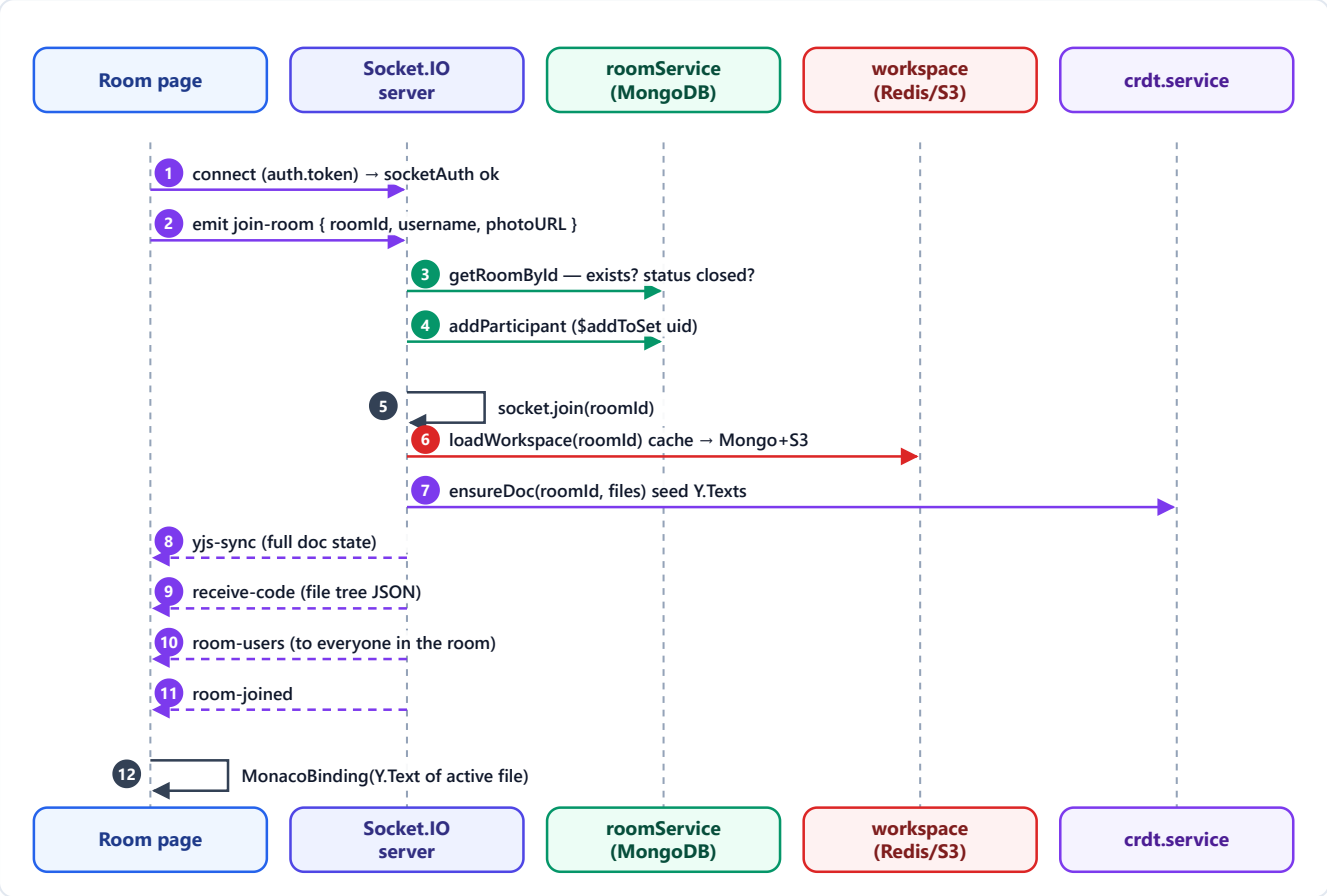


Figure 11. What happens when a user opens /room/:roomId. Order matters: the CRDT state is sent before the file tree.

1. The socket connects; `socketAuth` has already verified the token.
2. The Room page fetches the user's profile name and photo, then emits `join-room`.
3. The server loads the room; a missing room yields `join-error "Room not found"`, and a closed room yields an “Access Denied” message.
4. `addParticipant` runs `$addToSet`, so joining twice is harmless (idempotent).
5. `socket.join(roomId)` adds the socket to the broadcast group.
6. `loadWorkspace` returns the files from Redis, or rebuilds them from MongoDB + S3 (Chapter 11).
7. `crdt.ensureDoc` creates the room's Y.Doc if this is the first member (seeding one Y.Text per file), or cancels a pending idle drop.
8. `yjs-sync` sends the whole document state as one binary update.

9. `receive-code` sends the file tree, with the CRDT's text merged in.
10. `room-users` goes to everyone, so all avatar stacks update.
11. `room-joined` confirms.
12. The client binds Monaco to the Y.Text of the active file with `new MonacoBinding(text, model, new Set([editor]))`.

Why yjs-sync before receive-code? The editor must bind to up-to-date text. If the tree arrived first, the editor could show stale content for a moment and then jump.

10.2 The server-side CRDT service

Backend/src/services/crdt.service.js (abridged)

```
const rooms = new Map();                        // roomId → { doc, dropTimer }

export const ensureDoc = (roomId, files = []) => {
  let entry = rooms.get(roomId);
  if (!entry) {
    const doc = new Y.Doc();
    entry = { doc, dropTimer: null };
    rooms.set(roomId, entry);
    seedFiles(doc, files);                      // one Y.Text per file path
  }
  if (entry.dropTimer) { clearTimeout(entry.dropTimer); entry.dropTimer = null; }
  return entry.doc;
};

export const applyClientUpdate = (roomId, update, origin) => {
  const doc = getDoc(roomId);
  if (!doc) return false;
  try {
    Y.applyUpdate(doc, update instanceof Uint8Array ? update : new Uint8Array(update), origin);
    return true;
  } catch (err) {                               // malformed bytes: reject, don't crash
    return false;
  }
};

export const scheduleDrop = (roomId, delayMs = 5 * 60 * 1000) => {
  const entry = rooms.get(roomId);
  entry.dropTimer = setTimeout(() => { entry.doc.destroy(); rooms.delete(roomId); }, delayMs);
  entry.dropTimer.unref?.();                    // don't keep the process alive for this timer
};
```

- **One authoritative doc per room**, held in a `Map` in process memory.
- **Document model:** `doc.getMap("files")` maps a path to a `Y.Text`. Each file merges independently.
- **Seeding inside** `doc.transact(..., "server")` groups all inserts into one update and tags it with an origin.
- **Malformed updates are rejected** with try/catch, so a buggy or malicious client cannot crash the server. A unit test covers this.

- **Memory management:** five minutes after the last member leaves, the doc is destroyed; the next join rebuilds it from storage. `unref()` stops idle timers from blocking shutdown.

10.3 A keystroke's journey

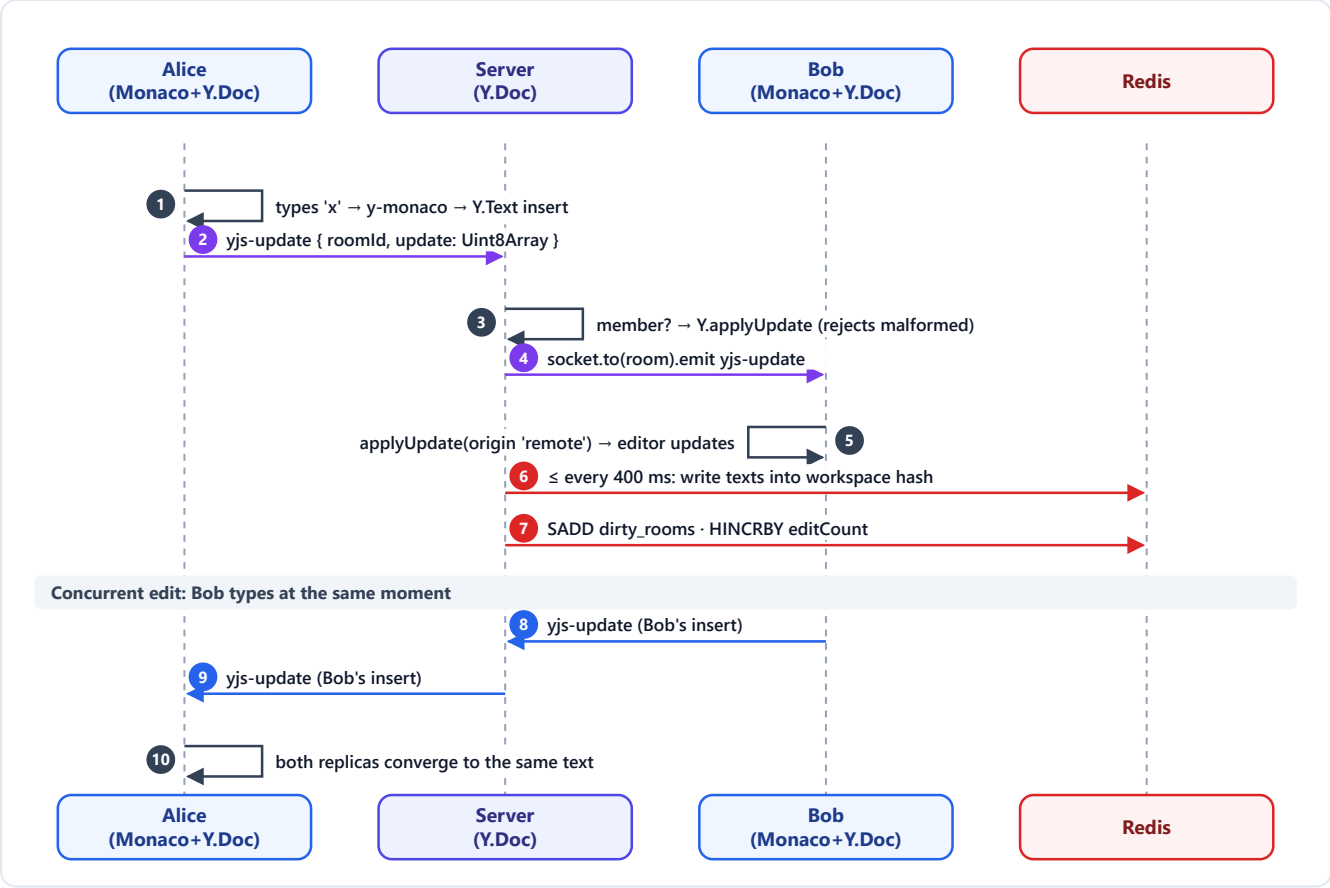


Figure 12. Alice types; the update is applied on the server, relayed to Bob, and written back to Redis. Concurrent edits from Bob merge the same way.

Backend/src/controllers/socketController.js

```
socket.on("yjs-update", ({ roomId, update }) => {
  if (!socket.rooms.has(roomId) || !update) return;          // members only
  if (!crdt.applyClientUpdate(roomId, update, socket.id)) return;
  socket.to(roomId).emit("yjs-update", update);              // everyone except the sender
  scheduleWriteBack(roomId);                                 // fold into Redis ≤ every 400 ms
});
```

Frontend/src/pages/Room.jsx (abridged)

```
const ydoc = useMemo(() => new Y.Doc(), [roomId]);
// outgoing: every local change becomes a socket message
const onLocalUpdate = (update, origin) => {
  if (origin === "remote") return;           // don't echo what we just received
  setSaveStatus("unsaved");
  socket.emit("yjs-update", { roomId, update });
};
ydoc.on("update", onLocalUpdate);
// incoming
const applyRemote = (update) => { if (update) Y.applyUpdate(ydoc, new Uint8Array(update),
"remote"); };
socket.on("yjs-sync", applyRemote);
socket.on("yjs-update", applyRemote);
```

The origin trick prevents infinite loops. When the client applies a remote update, Yjs fires its `update` event too; tagging the transaction with origin `"remote"` lets `onLocalUpdate` skip it, so the update is not sent back to the server.

Why `socket.to(roomId)` and not `io.to(roomId)`? `socket.to` excludes the sender, who already has the change. `io.to` is used when *everyone* must receive something, such as the server-made update after a structural change.

10.4 Write-back: from CRDT to Redis

Backend/src/controllers/socketController.js

```
const writeBackTimers = new Map();
const scheduleWriteBack = (roomId) => {
  if (writeBackTimers.has(roomId)) return;              // already scheduled: coalesce
  writeBackTimers.set(roomId, setTimeout(async () => {
    writeBackTimers.delete(roomId);
    const contents = crdt.getContents(roomId);          // Map<path, string>
    const cached = (await getCachedWorkspace(roomId)) || (await loadWorkspace(roomId)) || [];
    const merged = cached.map((f) => (!f.isFolder && contents.has(f.path) ? { ...f, content:
contents.get(f.path) } : f));
    await cacheWorkspace(roomId, merged);               // HSET per file
    await markRoomDirty(roomId);                        // SADD dirty_rooms
    const { shouldFlush } = await incrementEditTokens(roomId);
    if (shouldFlush && (await persistWorkspaceToS3(roomId))) io.to(roomId).emit("workspace-saved",
{...});
  }, 400));
};
```

This is a **throttle** (not a trailing debounce): the first keystroke schedules a write 400 ms later, and keystrokes during that window simply ride along. Fifty keystrokes in 400 ms become one Redis write and one edit token. That is why edit tokens count *write-backs* rather than raw keystrokes.

10.5 Structural changes: create, delete, rename, upload

File-tree changes are rarer and need a full picture, so the client sends the whole file list on `code-change`. The server reconciles it with the CRDT:

Backend/src/services/crdt.service.js

```
export const reconcileFiles = (roomId, files) => {
  const doc = ensureDoc(roomId, []);
  const map = doc.getMap("files");
  const live = new Set(files.filter((f) => f?.path && !f.isFolder).map((f) => f.path));
  const update = captureUpdate(doc, () => {
    doc.transact(() => { for (const key of [...map.keys()]) if (!live.has(key)) map.delete(key);
}, SERVER_ORIGIN);
    seedFiles(doc, files);                          // new paths get a Y.Text
  });
  // CRDT text is authoritative for content: a stale list can't overwrite live typing
  const merged = files.map((f) => (f?.path && !f.isFolder && map.has(f.path) ? { ...f, content:
map.get(f.path).toString() } : f));
  return { files: merged, update };
};
```

The key safety rule is in the last comment: **for existing files, content comes from the CRDT, not from the list the client sent**. Without this, Alice creating a new file would send a file list containing her slightly stale copy of `App.jsx` and overwrite Bob's last second of typing. The tests cover “reconcile keeps CRDT content, seeds new files and drops deleted ones”.

10.6 Presence and live cursors

- **Presence:** on every active-file change the client emits `update-presence`; the server stores `socket.activeFile` and broadcasts `room-users` (built from `io.in(roomId).fetchSockets()`).
- **Cursors:** `editor.onDidChangeCursorPosition` emits `cursor-change`; others receive `cursor-update` and render a coloured decoration plus a name-tag *content widget* that fades after 3 s of inactivity. Cursors in other files are hidden.
- **Cleanup:** when `room-users` no longer contains a user, their decorations and widgets are removed. `disconnecting` (which fires while `socket.rooms` is still populated) broadcasts the reduced list.

IMPROVEMENT TO MENTION

Cursors are plain socket events, so they don't move when text is inserted above them until the next cursor event. Yjs has an **awareness protocol** (`y-protocols/awareness`, already in package.json) that y-monaco can render as remote selections anchored to CRDT positions. Using it would also remove the custom cursor code.

10.7 Correctness: why CRDT merging is safe

Yjs gives three guarantees. Each one maps to a test in `Backend/test/crdt.test.js`:

- **Convergence:** replicas that received the same set of updates have the same state, whatever the order (test: “concurrent edits to the same file converge on every replica”).

- **Intention preservation:** an insert stays next to the characters it was typed next to, because positions are relative to character IDs, not absolute indexes.
- **Idempotency:** applying an update twice is harmless, so a resend after reconnection does not duplicate text.

Backend/test/crdt.test.js (abridged)

```
test("concurrent edits to the same file converge on every replica", () => {
  // two clients start from the server state, edit the same Y.Text "offline",
  // exchange updates in different orders → all three docs end with identical text
});
```

10.8 Offline and reconnection behaviour

If Bob's Wi-Fi drops, his local Y.Doc keeps accepting edits, and the Socket.IO client *buffers* the `yjs-update` events emitted while offline. Socket.IO reconnects automatically (up to 5 attempts, 1 s apart) on a **new** server-side socket. The `connect` handler emits `join-room` again, and the server replies with `yjs-sync` (the full state). Applying it merges everyone else's changes into Bob's doc, because CRDT merges are order-independent.

HONEST LIMITATION

The buffered offline updates can reach the server *before* the new socket has re-joined the room, and the handler ignores updates from sockets not in the room (`socket.rooms.has(roomId)`). So Bob keeps his offline edits on screen, but other users may never receive them. The robust fix is the standard y-protocols two-step sync on every join: the client sends `Y.encodeStateVector(doc)`, the server answers with `Y.encodeStateAsUpdate(serverDoc, clientVector)`, and the client sends back whatever the server is missing. Then nothing depends on message ordering.

Interview kit · Real-time collaboration

SIMPLE EXPLANATION

Everyone has their own copy of the document. When you type, a tiny "here's what I changed" message goes to the server, which forwards it to everyone else. Each change carries enough information to be placed correctly even if others typed at the same moment, so all copies end up identical.

TECHNICAL EXPLANATION

Each client holds a Yjs Y.Doc bound to Monaco via y-monaco. Local transactions emit binary updates over Socket.IO; the server applies them to its authoritative doc (rejecting malformed ones), relays them with `socket.to(room)`, and throttles a write-back of the doc's text into Redis every 400 ms. Structural changes are reconciled with the CRDT as the source of truth for contents. Presence and cursors are separate socket events.

SAY IT LIKE THIS

> *"The original version sent the whole file on each change and the last writer won, so if two people typed at once one of them lost work. I replaced that with Yjs. Every file is a Y.Text, every keystroke becomes a small binary update with unique character IDs, and merges are commutative and idempotent, so every replica*

> *converges without locks. The server keeps an authoritative copy per room, relays updates to the other members, and writes the text back to Redis at most every 400 milliseconds, which feeds the persistence pipeline."*

CODEFUSIONAI EXAMPLE

Alice inserts "import x" on line 1 while Bob edits line 20. Both updates reference character IDs, not line numbers, so after each applies the other's update both see both changes, and no line shifts corrupt either edit.

LIKELY FOLLOW-UPS

- **CRDT or OT?** OT transforms operations against concurrent ones via a central server and is hard to get right; CRDTs merge by construction and tolerate offline edits. The cost is more metadata per character (tombstones).
- **Why keep a doc on the server at all?** To give late joiners the full state instantly, to validate membership, and to persist. A pure relay would need some client to be online to serve history.
- **What if two users rename the same file at the same time?** Structural changes are whole-list messages processed in arrival order: the last one wins for the tree. Contents are safe because the CRDT is authoritative.
- **Does the server need to understand the edits?** No, it just applies binary updates; the Yjs algorithm does the work.

MISTAKES TO AVOID

- Saying "we lock the file while someone is typing". There is no per-file editing lock; the CRDT makes one unnecessary.
- Saying Socket.IO itself resolves conflicts. It only transports messages.

CHAPTER 11

Caching, Locking and Persistence

Typing generates thousands of tiny changes per minute. Writing each one to S3 would be slow and expensive; never writing would lose work. This chapter explains the pipeline in between: a Redis write buffer, a dirty set, edit tokens, a token bucket and a distributed lock.

11.1 Redis data model

Key pattern	Type	Value	Expiry
`room:{roomId}:files`	**Hash**	field = base64(path) → JSON {path, content, isFolder, language}	2 h, sliding
`dirty_rooms`	**Set**	roomIds with unsaved changes	no TTL
`room:{roomId}:token_bucket`	**Hash**	editCount · lastEditTime · s3Tokens · lastRefill	TTL 24 h
`room:{roomId}:persist:lock`	**String**	"1" set with NX EX — mutual exclusion for persist	TTL 60 s
`ratelimit:{name}:{uid|ip}:{window}`	**String**	INCR counter for the fixed window	TTL = window

Figure 13. Every Redis key CodeFusionAI uses, with its type, contents and expiry.

- `room:{roomId}:files` **(Hash).** One field per file (base64 path) whose value is that file's JSON. A hash rather than one big string means updating file A doesn't rewrite file B. The original design stored the whole workspace as one string, so two users editing different files raced and one overwrote the other.
- `dirty_rooms` **(Set).** Room IDs with unsaved changes. Sets are idempotent: marking a room dirty 1,000 times stores it once.
- `room:{roomId}:token_bucket` **(Hash).** `editCount`, `lastEditTime`, `s3Tokens` and `lastRefill`.
- `room:{roomId}:persist:lock` **(String).** Exists only while a persist job runs; expires after 60 s.
- `ratelimit:…` **(String).** Fixed-window counters for API rate limits.

11.2 Reading: the cache-aside waterfall

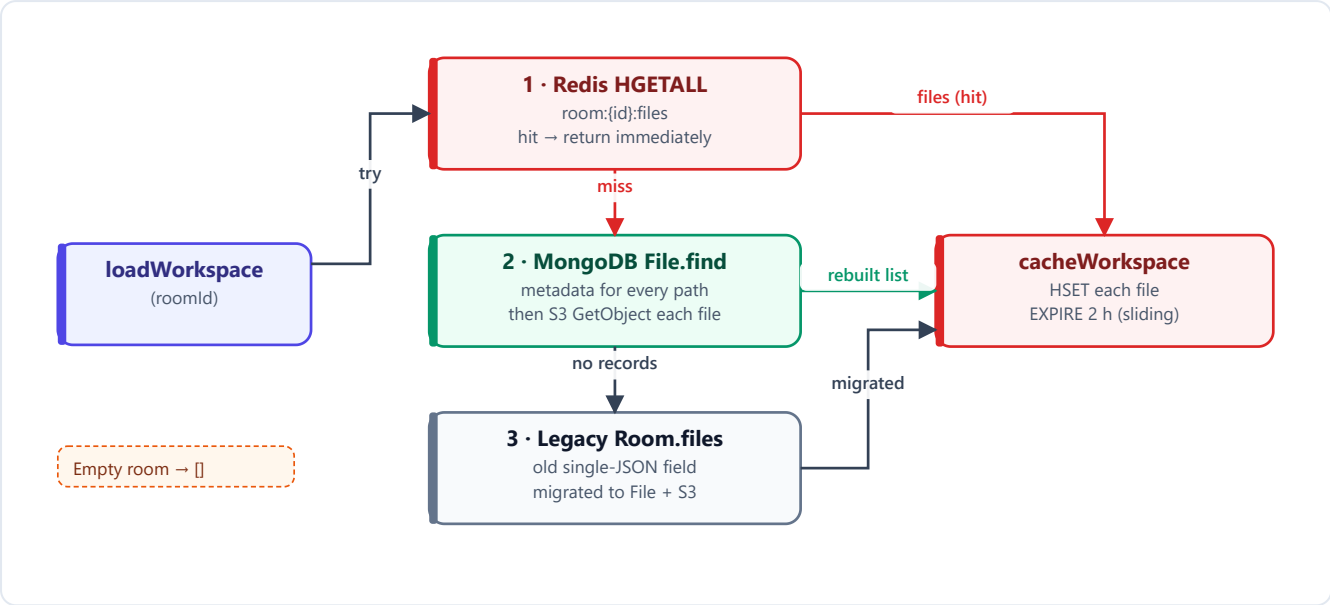


Figure 14. loadWorkspace tries Redis first, then rebuilds from MongoDB + S3, then migrates the legacy format, and fills the cache.

1. **Cache hit:** `HGETALL room:{id}:files` returns every file; the result is sorted by path for a stable order.
2. **Miss → rebuild:** `File.find({ roomId })` gives metadata; each file's content is fetched from S3 with `GetObject`. A failed S3 read yields an empty file rather than failing the whole room (graceful degradation).
3. **Legacy migration:** rooms created before the File collection existed have a `Room.files` JSON string. It is parsed and migrated into File documents + S3 on first load.
4. **Fill:** `cacheWorkspace` writes the result back with `HSET` per file and a 2-hour sliding TTL.

This is the **cache-aside** (lazy loading) pattern: the application, not the cache, decides when to load. The TTL keeps idle rooms from occupying RAM forever; every write refreshes it (a sliding expiry).

11.3 Writing: the full persistence pipeline

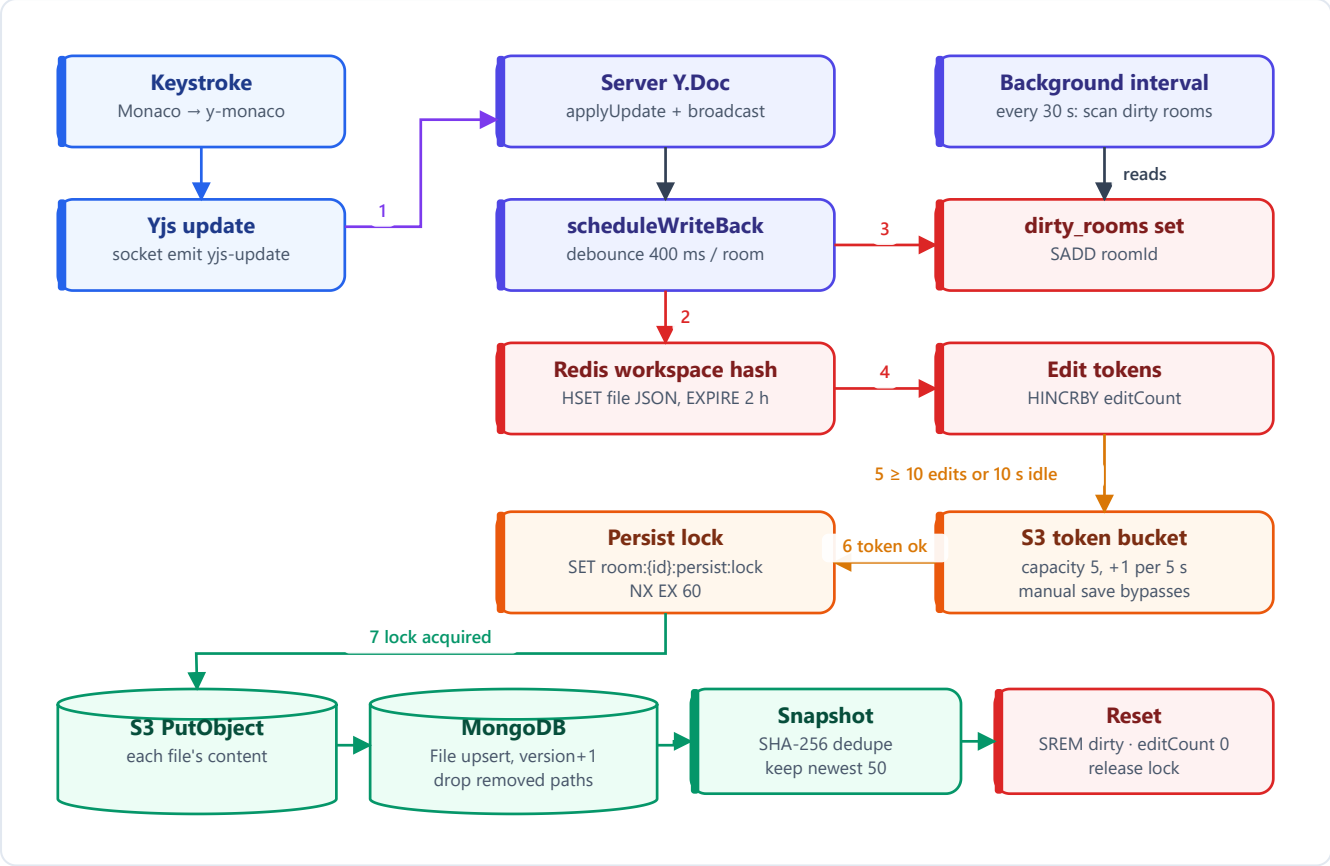


Figure 15. From keystroke to S3: the seven steps of the write path.

1. **Yjs update** reaches the server doc and is relayed (Chapter 10).
2. **Write-back:** at most every 400 ms the doc's text is merged into the Redis hash.
3. **Dirty:** `SADD dirty_rooms roomId`.
4. **Edit token:** `HINCRBY editCount 1` and set `lastEditTime`.
5. **Flush decision:** immediately when `editCount ≥ 10`, or by the 30-second background interval when the room has pending edits and has been idle ≥ 10 s.
6. **Token bucket:** the flush consumes one S3 token (capacity 5, refill 1 per 5 s). Without a token it is skipped and the room stays dirty. Manual save skips this step.
7. **Lock and persist:** `SET … NX EX 60`. If acquired: upload each file to S3, upsert File metadata (`version + 1`), delete metadata and objects for files that no longer exist, clear dirty, reset edit tokens, record a snapshot, and release the lock in `finally`.

Backend/src/services/workspace.service.js (abridged)

```
export const persistWorkspaceToS3 = async (roomId, isManualSave = false, meta = {}) => {
  if (!isManualSave && !(await checkAndConsumeS3Token(roomId))) return false;   // throttled
  if (!(await acquirePersistLock(roomId))) return false;                         // someone else
is saving
  try {
    const files = await getCachedWorkspace(roomId);
    if (!files) return false;
    for (const file of files) {
      if (file.isFolder) { await File.findOneAndUpdate({ roomId, path: file.path }, {...}, {
upsert: true }); continue; }
      const dbFile = await File.findOne({ roomId, path: file.path });
      const s3Key = dbFile ? dbFile.s3Key : createS3Key(ownerId, roomId, file.path);
      try { await uploadFileContent(s3Key, file.content || ""); } catch { continue; }  // skip,
keep going
      await File.findOneAndUpdate({ roomId, path: file.path },
        { s3Key, version: dbFile ? dbFile.version + 1 : 1, size, language, ... }, { upsert: true
});
    }
    // remove metadata + S3 objects for deleted/renamed paths
    const removed = await File.find({ roomId, path: { $nin: livePaths } });
    await Promise.allSettled(removed.map((f) => deleteS3Object(f.s3Key)));
    await File.deleteMany({ _id: { $in: removed.map((f) => f._id) } });
    await clearRoomDirty(roomId);
    await resetEditTokens(roomId);
    await createSnapshot(roomId, files, { reason: isManualSave ? "manual" : "auto", ...meta });
    return true;
  } finally {
    await releasePersistLock(roomId);                                            // always
  }
};
```

11.4 The token bucket, explained

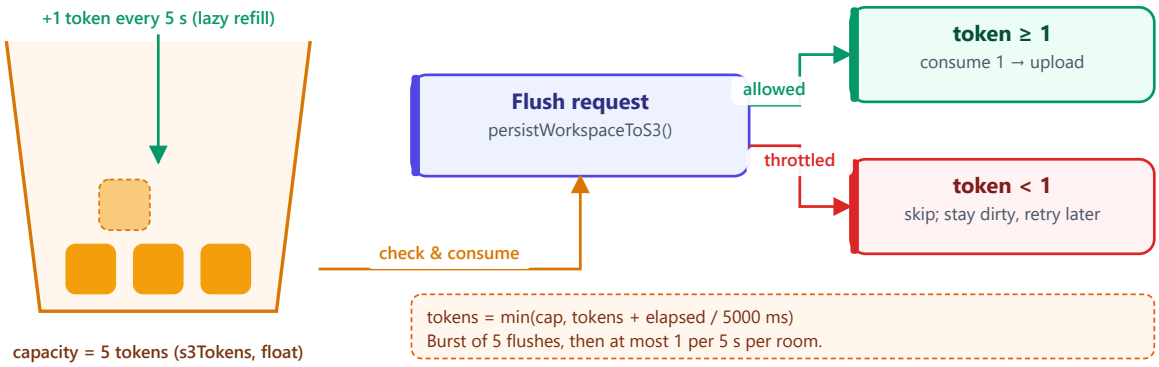


Figure 16. Token bucket for S3 flushes: up to 5 flushes in a burst, then one every 5 seconds per room.

DEFINITION: TOKEN BUCKET

A rate-limiting algorithm. A bucket holds up to *C* tokens and refills at *r* tokens per second. Each action spends one token; with no token, the action is refused (or delayed). It allows short **bursts** (up to *C*) while enforcing a long-run **average rate** (*r*).

Backend/src/services/redis.service.js

```
export const checkAndConsumeS3Token = async (roomId) => {
  const key = REDIS_KEYS.tokenBucket(roomId);
  const now = Date.now();
  const data = await redisClient.hgetall(key);
  let s3Tokens = data.s3Tokens !== undefined ? parseFloat(data.s3Tokens) : 5;       // start full
  let lastRefill = data.lastRefill !== undefined ? parseInt(data.lastRefill, 10) : now;
  const elapsed = now - lastRefill;
  if (elapsed > 0) {                                    // lazy refill: compute instead of a
timer
    s3Tokens = Math.min(5, s3Tokens + elapsed / 5000);
    lastRefill = now;
  }
  if (s3Tokens >= 1) {
    s3Tokens -= 1;
    await redisClient.hset(key, "s3Tokens", String(s3Tokens), "lastRefill", String(lastRefill));
    return true;                                        // allowed
  }
  await redisClient.hset(key, "s3Tokens", String(s3Tokens), "lastRefill", String(lastRefill));
  return false;                                         // throttled
};
```

Lazy refill is the clever part: no timer refills buckets. Every check computes how many tokens *would* have accumulated since `lastRefill`. This costs O(1) per check, needs no background job, and works for millions of rooms.

Worked example. A room with a full bucket triggers 7 threshold flushes within 2 seconds. Flushes 1–5 succeed (bucket 5 → 0); 6 and 7 are throttled, and the room stays dirty. Ten seconds later the bucket holds 2 tokens, and the next trigger succeeds. S3 writes for that room are bounded at roughly 5 + t/5 over t seconds, no matter how fast people type.

BUG FOUND AND FIXED (GREAT INTERVIEW STORY)

The in-memory Redis mock's `hset` stored only the first field/value pair, and `hgetall` on a missing key returned `null`. So `lastRefill` was never saved, and `data.s3Tokens` threw on `null`, which the catch block turned into "fail open: allow". The bucket *always allowed*. The fix made the mock match real Redis (multi-pair `HSET`, `HGETALL` → `{}`), and a regression test "token bucket refills over time (lastRefill must be persisted)" now guards it. Lesson: a fake must honour the real contract, or tests pass for the wrong reason.

CONCURRENCY CAVEAT

The check is a read-modify-write across two commands (`HGETALL` then `HSET`), so two processes checking the same room at the same instant could both spend the last token. With one process this is harmless; across instances, the fix is to run the whole algorithm atomically in a **Lua script** (`EVAL`), which Redis executes without interleaving.

11.5 The distributed persist lock

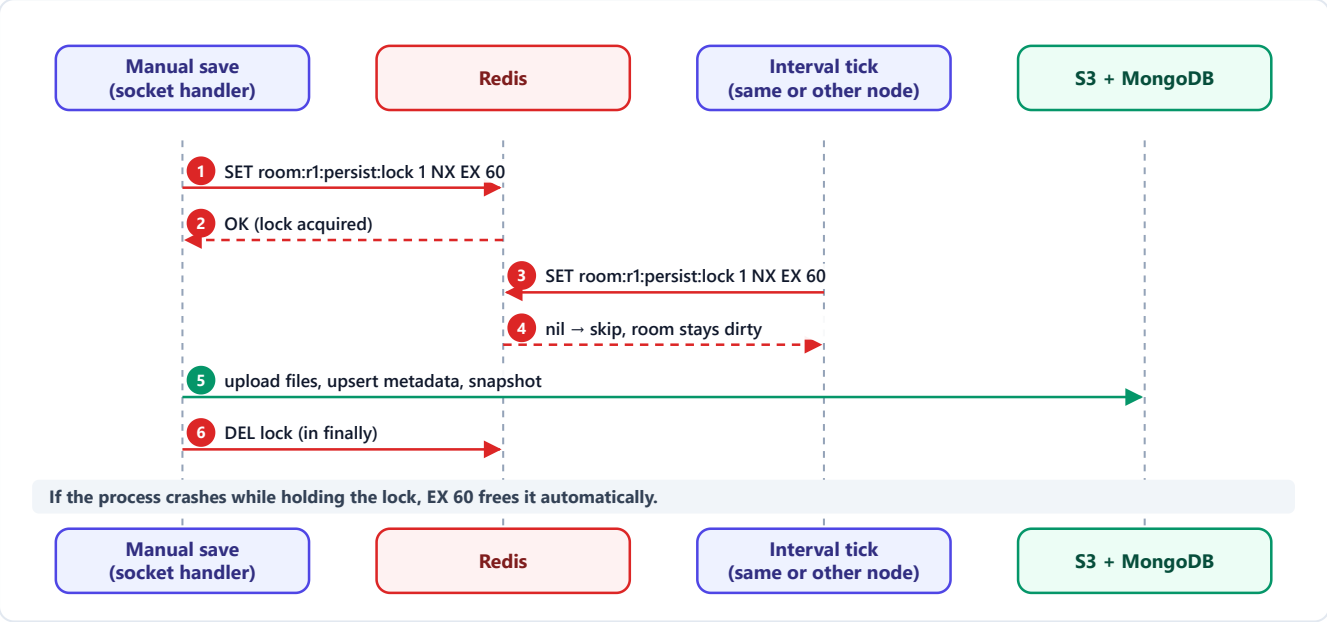


Figure 17. Two persist attempts for the same room. Only the first SET NX succeeds; the other skips and the room stays dirty for the next tick.

Backend/src/services/redis.service.js

```
export const acquirePersistLock = async (roomId) => {
  const result = await redisClient.set(REDIS_KEYS.persistLock(roomId), "1", "NX", "EX", 60);
  return result === "OK";          // null when the key already exists
};
export const releasePersistLock = async (roomId) => {
  await redisClient.del(REDIS_KEYS.persistLock(roomId));
};
```

Why a lock at all? A manual save and the background flusher (or two backend instances) could persist the same room at the same time. Then the S3 writes interleave, file `version` counters double-increment, and an older snapshot could overwrite a newer one. `SET key value NX EX 60` is one atomic command: set only if the key does **N**ot e**X**ist, with a 60-second **EX**piry. Exactly one caller gets `"OK"`.

Why the 60 s expiry? If the process crashes while holding the lock, the `finally` never runs. Without a TTL the room could never be saved again (a deadlock). With it, the lock frees itself.

WHAT A SENIOR INTERVIEWER WILL PROBE

- **Unsafe release.** `DEL` removes the lock even if it is no longer ours. Scenario: persist A runs longer than 60 s, the lock expires, B acquires it, A finishes and deletes *B's* lock, and C can now start alongside B. Fix: store a random token as the value and release with a Lua script that deletes only if the value matches.
- **No fencing token.** Even with a safe release, a paused process (for example a long GC pause) can wake up after its lock expired and still write. Fix: a monotonically increasing **fencing token** checked by the storage layer, or write snapshots with a conditional on the version number (the unique `{roomId, version}` index already gives partial protection).
- **Single Redis node.** If Redis fails over and the replica hadn't received the lock, two holders are possible. Redlock (a lock across N independent Redis nodes) is one answer, though it is debated. Using fencing is the principled one.
- **Comment mismatch.** The code comment says the lock "fails open", but on a Redis error it returns `false`, so it actually *fails closed* (skip the save, try later). Failing closed is the safer choice for a lock.

11.6 Failure-mode table

Failure	What happens	Data at risk
S3 PutObject fails for one file	That file is skipped (`continue`); others persist; the room is still marked clean	That file's latest content stays only in Redis until the next edit
S3 down for a snapshot	Snapshot JSON stored inline in MongoDB (`inlineFiles`)	None
Redis unreachable at start-up	Wrapper switches to the in-memory MockRedis	Everything not yet in S3 on restart; no sharing across instances
Rate-limit check errors	Fails open (request allowed)	None (availability over strictness)
Token-bucket check errors	Fails open (flush allowed)	None (durability over cost)
Process crash mid-persist	Lock expires in ≤ 60 s; the room stays dirty; the next tick retries	None beyond the last Redis write-back
Process crash between keystroke and write-back	In-memory Y.Doc is lost	Up to ~400 ms of typing

SUBTLE BUG WORTH SPOTTING

When an S3 upload fails, the loop `continue`s, but the room is still cleared from `dirty_rooms` at the end. If no further edits come, the failed file is not retried, and S3 keeps the older version (Redis has the new one only until its 2-hour TTL). A fix: track failures, and keep the room dirty if any upload failed.

Interview kit · Caching, locking and persistence

SIMPLE EXPLANATION

Typing goes into a fast scratchpad (Redis). Every so often, but not too often, the scratchpad is copied into permanent storage (S3 and MongoDB). A “now saving” sign (the lock) stops two copies from happening at once.

TECHNICAL EXPLANATION

Write-behind caching: the CRDT is written back to a Redis hash per room (throttled to 400 ms), the room is added to a dirty set and an edit counter is incremented, and flushes are triggered by an edit threshold or an idle timeout. A per-room token bucket (capacity 5, refill 1 per 5 s, lazy refill) caps S3 write frequency. A `SET NX EX` lock gives per-room mutual exclusion with deadlock protection. Reads use cache-aside with a sliding TTL.

SAY IT LIKE THIS

> *“Persistence is write-behind. Edits land in a Redis hash per room, and the room goes into a dirty set. A room is flushed when it gets ten edit batches or has been idle for ten seconds, but each flush must take a token from a per-room token bucket, five tokens refilling one every five seconds, so a hyperactive room can't spam S3. Before flushing I take a Redis lock with SET NX EX 60, so a manual save and the background job never write the same room concurrently, and the TTL prevents deadlock if the process dies. I know the release isn't owner-checked yet. The fix is a random token and a compare-and-delete Lua script.”*

CODEFUSIONAI EXAMPLE

Ten collaborators hammer a room. Write-backs coalesce to one per 400 ms, threshold flushes fire every ~4 s, and the bucket allows 5 quickly and then 1 per 5 s. So S3 sees roughly 12 writes per minute per file instead of thousands.

LIKELY FOLLOW-UPS

- **Write-through versus write-behind?** Write-through writes the cache and the store synchronously, so it is safe but slow per edit. Write-behind is fast but has a durability window. Typing needs write-behind.
- **Why a hash and not one JSON string?** Per-file independence; updating one file doesn't rewrite or race with the others.
- **How do you avoid the thundering herd on a cold room?** Today, two first joiners could both rebuild from S3. A short “loading” lock (or single-flight promise per room) would dedupe it.
- **Why not make every keystroke durable?** Cost and latency; the CRDT plus a 400 ms write-back makes the loss window tiny already.

MISTAKES TO AVOID

- Calling the persist lock a “file lock that stops users editing”. It only serialises *saving*.
- Claiming “zero data loss”. State the window and its causes.

CHAPTER 12

Amazon S3 Storage

12.1 Bucket layout and data flow

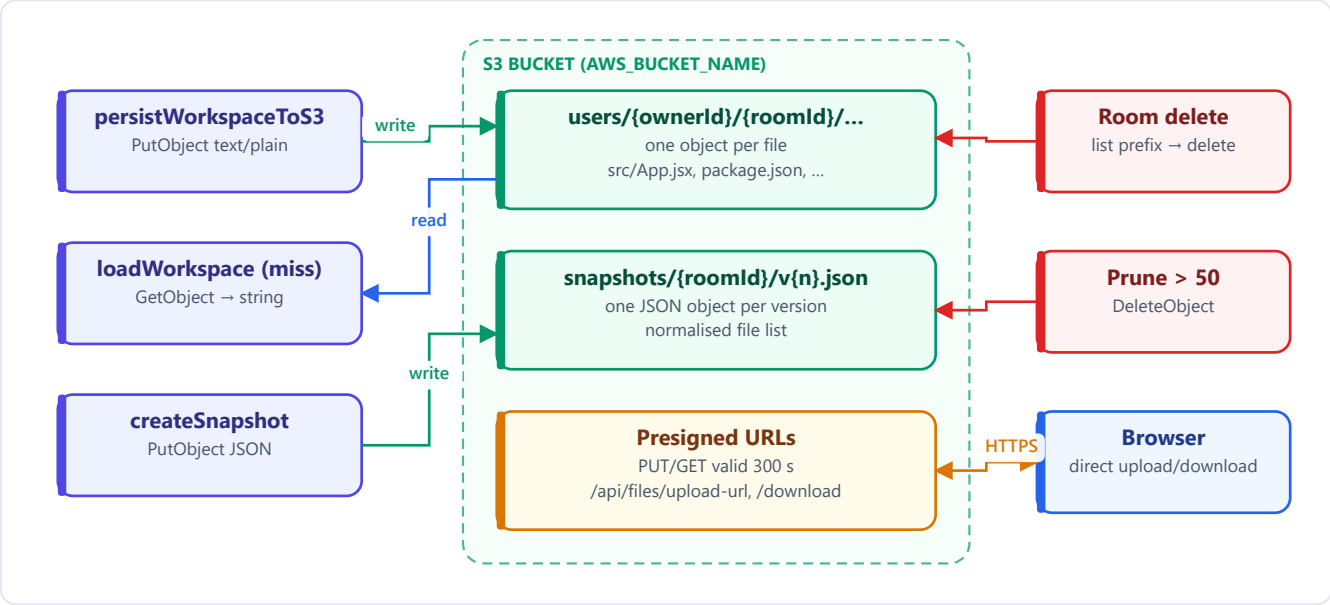


Figure 18. What is stored in S3, who writes and reads it, and how presigned URLs let the browser talk to S3 directly.

- **Workspace files:** `users/{ownerId}/{roomId}/{path}`, for example `users/u123/cfai-ab12-cd34/my-app/src/App.jsx`. Grouping by owner and then room makes "delete a room" a prefix delete (`ListObjectsV2` + `DeleteObject`) and allows per-user lifecycle or IAM rules later.
- **Snapshots:** `snapshots/{roomId}/v{n}.json`, one object per version holding the normalised file list. One object per version is far cheaper than copying every file per version.
- **Presigned URLs:** `getSignedUrl(s3Client, new PutObjectCommand(...), { expiresIn: 300 })` creates a URL that lets the holder upload (or download) one specific key for 5 minutes, without AWS credentials. The browser then talks to S3 directly, so large files don't flow through Node.

Backend/src/services/s3.service.js

```
export const createS3Key = (ownerId, roomId, filePath) =>
  `users/${ownerId || "default_user"}/${roomId}/${filePath.replace(/\\/g, "/")}`;   // Windows
paths → "/"

export const uploadFileContent = async (s3Key, content, contentType = "text/plain") => {
  await s3Client.send(new PutObjectCommand({ Bucket: BUCKET_NAME, Key: s3Key, Body: content,
ContentType: contentType }));
  return true;
};
export const getFileContent = async (s3Key) => {
  const response = await s3Client.send(new GetObjectCommand({ Bucket: BUCKET_NAME, Key: s3Key }));
  return await response.Body.transformToString("utf-8");          // stream → string
};
```

12.2 Why S3 and not MongoDB for contents

Criterion	S3	MongoDB documents
Size limit	Up to 5 TB per object	16 MB per document
Durability	Designed for 11 nines, multi-AZ	Depends on the replica set and backups
Cost per GB	Low (object storage pricing)	High (database storage, RAM for working set)
Query by content	No	Yes (not needed here)
Effect on DB performance	None	Large documents bloat the cache and replication
Direct browser access	Presigned URLs	Must proxy through the API

12.3 Security and operations

- The bucket should block public access; only the server (IAM user/role) and presigned URLs can touch objects. **Not verifiable** from the repository.
- Prefer an **IAM role** (on EC2/ECS/Lambda) over static access keys in `.env`; scope the policy to this bucket and the needed actions.
- Enable **versioning** and lifecycle rules (expire old versions, move to Infrequent Access) for an extra safety net.
- **Performance:** S3 supports at least 3,500 PUT and 5,500 GET requests per second *per prefix*; our per-room prefixes spread load naturally.

OPTIMISATION TO PROPOSE

`persistWorkspaceToS3` uploads *every* file on every flush, sequentially. Better: keep a content hash per file (in the File document or Redis) and upload only files whose hash changed, in parallel with a concurrency limit (for example 8). A 200-file room where one file changed would go from 200 PUTs to 1.

Interview kit · S3

SIMPLE EXPLANATION

S3 is a giant, very safe hard drive in the cloud where every file has an address (its key). We keep the actual code there, and MongoDB just remembers each file's address.

TECHNICAL EXPLANATION

Object storage with a hierarchical key convention (owner/room/path), server-side writes via SDK v3 commands, presigned URLs for direct client transfer, snapshot objects per version, and prefix-based deletion. It is strongly consistent for read-after-write since 2020.

SAY IT LIKE THIS

> *"File contents go to S3 under users/owner/room/path, and Mongo stores only metadata and the S3 key. S3 is built for blobs: effectively unlimited size, eleven nines of durability, cheap per gigabyte, and no 16 MB document limit. Because each S3 request has latency and cost, I never write per keystroke. Redis buffers, and a token bucket bounds the flush rate."*

CODEFUSIONAI EXAMPLE

Deleting a room calls `deleteS3Prefix("users/{owner}/{room}/")`, which lists and deletes every object of the room, plus its snapshot objects via `deleteRoomSnapshots`.

LIKELY FOLLOW-UPS

- **What does a presigned URL contain?** The bucket, key, method, expiry and a signature computed with the server's credentials (SigV4); S3 recomputes and compares the signature.
- **ListObjectsV2 returns 1,000 keys max. Is that a problem?** Yes, `deleteS3Prefix` doesn't paginate, so rooms with more than 1,000 objects would leave leftovers. Fix: loop on `ContinuationToken` and use `DeleteObjects` (batch of up to 1,000).
- **Consistency?** Strong read-after-write for PUTs and DELETEs, so a load right after a flush sees the new data.

MISTAKES TO AVOID

- Saying S3 is a database or a file system; it's a key → object store with no rename (rename = copy + delete).
- Exposing AWS keys to the frontend. Presigned URLs exist precisely so you don't.

CHAPTER 13

Version History

13.1 How a version is recorded

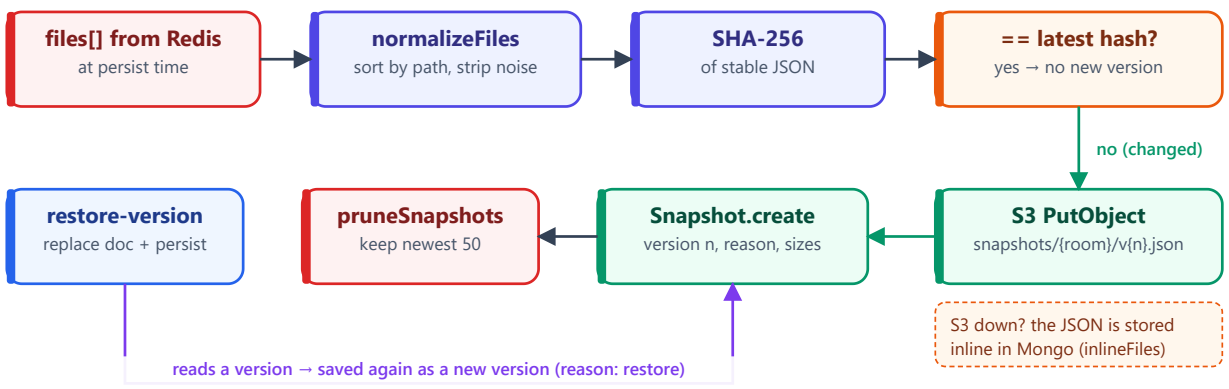


Figure 19. Snapshot pipeline: normalise, hash, skip if unchanged, upload JSON, insert metadata, prune. Restore reuses the same path.

Backend/src/services/history.service.js (abridged)

```
export const normalizeFiles = (files = []) =>
  files.filter((f) => f && f.path)
       .map((f) => f.isFolder ? { path: f.path, isFolder: true }
                              : { path: f.path, isFolder: false, content: f.content || "",
language: f.language || undefined })
       .sort((a, b) => a.path.localeCompare(b.path));          // stable order → stable hash

export const hashFiles = (normalized) =>
  crypto.createHash("sha256").update(JSON.stringify(normalized)).digest("hex");

export const createSnapshot = async (roomId, files, { reason = "auto", ... } = {}) => {
  const normalized = normalizeFiles(files);
  if (normalized.length === 0) return null;
  const contentHash = hashFiles(normalized);
  const latest = await Snapshot.findOne({ roomId }).sort({ version: -1 }).select("version
contentHash").lean();
  if (latest && latest.contentHash === contentHash && reason !== "restore") return null;   //
nothing changed
  const version = (latest?.version || 0) + 1;
  // upload JSON to S3 (or keep it inline if S3 fails), insert metadata, prune to 50
};
```

1. **Normalise:** keep only meaningful fields and sort by path. Without sorting, the same workspace in a different order would hash differently and create a fake new version.
2. **Hash:** SHA-256 of the JSON is a fingerprint; identical content gives an identical hash.

3. **De-duplicate:** if the newest version has the same hash, skip. The background flusher can then run often without spamming identical versions. A restore is always recorded, even if identical, because it is a meaningful event.
4. **Number:** `latest + 1`. This runs inside the persist lock, so two flushes can't claim the same number, and the unique index is a second line of defence.
5. **Store:** one JSON object per version in S3 (`snapshots/{room}/v{n}.json`); metadata (sizes, paths, author, reason) in Mongo. If S3 fails, the JSON is stored inline.
6. **Prune:** keep the newest 50; delete older metadata and S3 objects.

13.2 Restoring a version

Backend/src/controllers/socketController.js

```
socket.on("restore-version", async ({ version }, ack) => {
  const roomId = socket.roomId;
  if (!roomId || !socket.rooms.has(roomId)) return reply({ ok: false, error: "Join the room
first." });
  const { files } = await getSnapshotFiles(roomId, version);
  await replaceWorkspace(roomId, files);                          // Redis = exactly these files
  const update = crdt.replaceContents(roomId, files);           // rewrite every Y.Text
  if (update) io.to(roomId).emit("yjs-update", update);         // everyone's editor updates
  await markRoomDirty(roomId);
  await persistWorkspaceToS3(roomId, true, { reason: "restore", restoredFrom: Number(version), ...
});
  io.to(roomId).emit("receive-code", JSON.stringify(files));
  io.to(roomId).emit("version-restored", { version: Number(version), by: socket.username });
  reply({ ok: true });                                            // acknowledgement to the caller
});
```

A restore is itself a new version. Restoring v7 when the latest is v12 creates v13 with `restoredFrom: 7`. Nothing is destroyed, so you can "undo a restore" by restoring v12. This is the same idea as `git revert` versus `git reset`.

Why `replaceContents` and not new docs? Collaborators' editors are bound to the existing Y.Texts. Rewriting the text inside one transaction produces one update that every client applies, so their editors change in place without re-binding.

Acknowledgements: the client calls `socket.timeout(20000).emit("restore-version", { version }, cb)`, so the UI can show a spinner and an error if the server doesn't answer within 20 s.

13.3 The History UI

- `HistoryPanel.jsx` calls `GET /api/rooms/:roomId/history` (metadata only) and refreshes on `workspace-saved` and `version-restored`.
- **Compare** fetches one version's files and opens Monaco's `DiffEditor` (`CompareView.jsx`) against the current file.
- **Restore** asks for confirmation, then emits `restore-version` with an acknowledgement timeout.

Interview kit · Version history

SIMPLE EXPLANATION

Every time the room is saved, we keep a photo of all the files. If nothing changed since the last photo, we don't take a new one. You can look at an old photo or bring it back, and bringing it back is saved as a new photo, so nothing is ever lost.

TECHNICAL EXPLANATION

Content-addressed de-duplication with SHA-256 over a canonical (sorted, normalised) JSON serialisation; immutable snapshots stored as one S3 object each with Mongo metadata; monotonically increasing version numbers assigned under the persist lock with a unique index as a backstop; bounded retention (50); restore implemented as a forward change via a CRDT transaction plus a new snapshot.

SAY IT LIKE THIS

> *"Every successful flush records a snapshot, but first I hash a canonical form of the workspace: paths sorted and fields normalised. If the hash matches the last version I skip it, so the background flusher never creates duplicates. Each version is a single JSON object in S3 with metadata in Mongo, capped at 50. Restoring is non-destructive: it rewrites the CRDT texts in one transaction so every collaborator's editor updates live, and saves the result as a new version with restoredFrom set."*

CODEFUSIONAI EXAMPLE

Two flushes 30 s apart with no edits in between produce the same hash, so `createSnapshot` returns `null` and the History panel doesn't fill with identical entries.

LIKELY FOLLOW-UPS

- **Why SHA-256 and not comparing contents?** Comparing means loading the previous version from S3; a 64-character hash in Mongo is an O(1) check.
- **Storage growth?** Full copies × 50 per room. Improvement: store per-file content hashes and only changed blobs (content-addressed storage, like Git's objects).
- **Why is sorting essential?** JSON serialisation is order-sensitive; without canonicalisation, equal workspaces could hash differently (a test covers this).

MISTAKES TO AVOID

- Calling it "Git integration". It's snapshot history, with no branches or merges.
- Saying restore "rolls back" or deletes newer versions; it doesn't.

CHAPTER 14

AI Integration

Two AI surfaces exist: the right-hand AI panel inside a room, which is aware of the open file and selection, and a full-page Assistant with saved chats. Both call the same backend endpoint, which calls Gemini.

14.1 Request and response flow

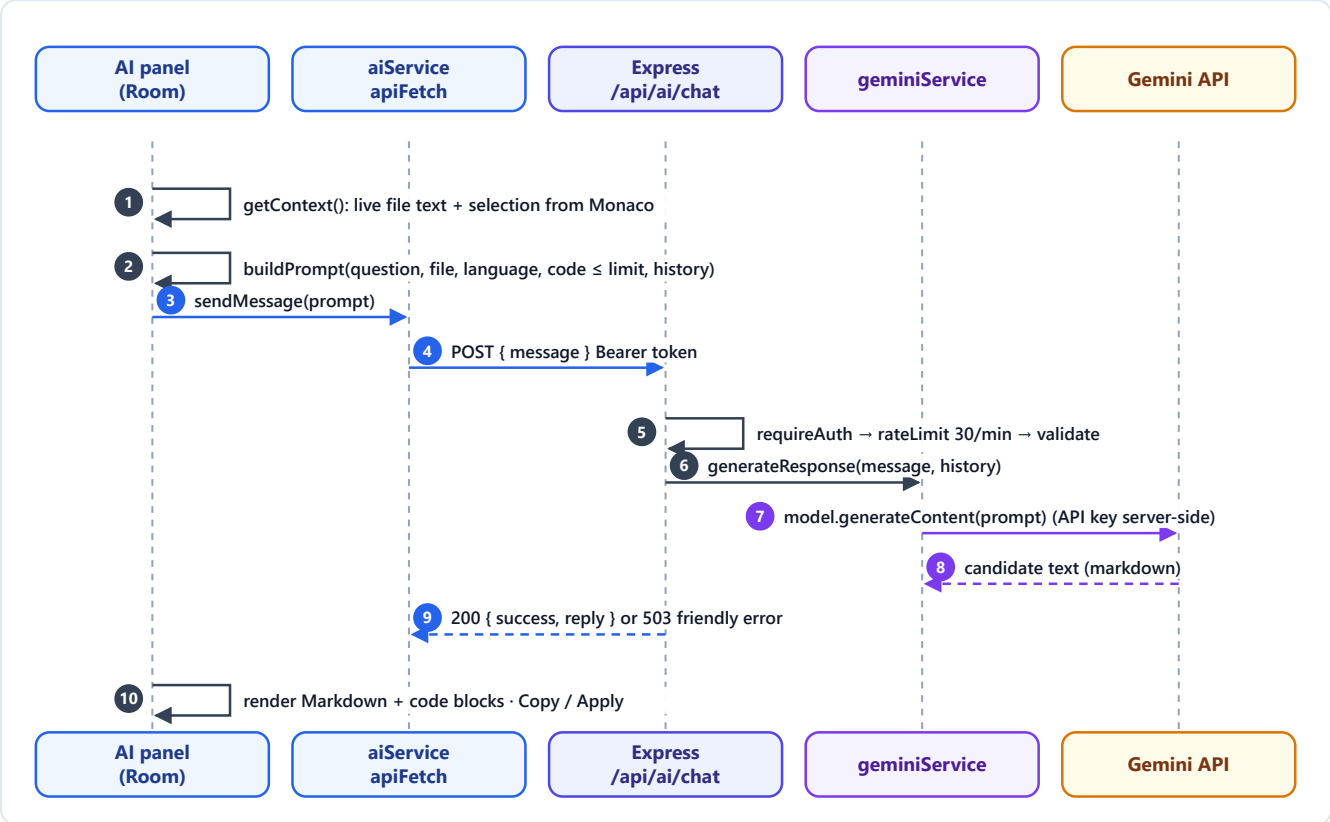


Figure 20. From "Explain this code" in the AI panel to rendered markdown: prompt assembly in the browser, guarded backend call, Gemini, friendly errors.

1. **Read live context.** At send time, `getContext()` reads the active model's full text and the current selection directly from Monaco.
2. **Build the prompt.** `buildPrompt` combines an instruction, the file name and language, the selection (or the whole file), capped at 12,000 characters, the recent turns, and the question.
3. **Send.** `sendMessage(prompt)` uses `apiFetch`, which adds the Bearer token.
4. **HTTP call:** `POST /api/ai/chat { message }`.
5. **Guards:** `requireAuth`, then a rate limit of 30 per minute per user, then `validate({ message: "required|string" })`.
6. **Service:** `generateResponse(message, history)`.
7. **Gemini:** `genAI.getGenerativeModel({ model: "gemini-2.5-flash" }).generateContent(prompt)`. The API key exists only on the server.

8. **Model reply:** markdown text.
9. **Response:** `200 { success, reply }`, or a 503 with a human-readable reason.
10. **Render:** `Markdown.jsx` (react-markdown + GFM + Prism) shows code blocks with *Copy* and *Apply*.

14.2 Prompt construction

Frontend/src/components/Room/AIPanel.jsx (abridged)

```
const MAX_CONTEXT_CHARS = 12000;
const buildPrompt = ({ question, file, language, code, selection, history }) => {
  const snippet = (selection?.trim() ? selection : code || "").slice(0, MAX_CONTEXT_CHARS);
  const scope = selection?.trim() ? "the user's selected code" : "the active file";
  return [
    "You are an expert pair programmer inside the CodeFusionAI IDE. Reply in GitHub-flavoured
markdown; put code in fenced blocks with a language tag.",
    file ? `Active file: ${file} (${language || "plain text"}). Below is ${scope}:` : "",
    fenced(language, snippet),              // the snippet inside a markdown code fence
    ...history.slice(-6).map(...),          // recent turns for follow-up questions
    `User: ${question}`,
  ].join("\n\n");
};
```

- **Selection beats file:** if the user highlighted lines, only those are sent. That gives a more focused answer, fewer tokens and lower cost.
- **Character cap:** 12,000 characters (roughly 3,000 tokens) keeps latency and cost predictable, even for huge files.
- **Short history:** a few recent turns give follow-up questions ("now add tests for that") their context without growing the prompt forever.
- **Output format instruction:** asking for fenced code blocks with a language tag lets the UI find code to Apply.

REAL BUG: "AI ONLY WORKS FOR APP.JSX"

The panel originally received `code` from React state, which was set when a file was *opened* and by `onChange`. With path-based Monaco models, switching tabs doesn't fire `onChange`, and remote CRDT edits update the model rather than that state. So the AI kept seeing an old copy, usually the first file opened (`App.jsx`). The fix: `getEditorContext()` reads `editor.getModel().getValue()` and the live selection *at send time*. Lesson: read from the source of truth at the moment of use, and don't keep a copy that can go stale.

14.3 Applying AI code safely

Frontend/src/pages/Room.jsx

```
const handleApplyCodeSuggestion = (newContent) => {
  const editor = editorRef.current;
  const model = editor.getModel();
  const selection = editor.getSelection();
  const range = selection && !selection.isEmpty() ? selection : model.getFullModelRange();
  editor.executeEdits("ai-assist", [{ range, text: newContent, forceMoveMarkers: true }]);
  toast.success(...);                    // "Undo with Ctrl Z"
};
```

`executeEdits` goes through Monaco's edit stack, so `Ctrl+Z` undoes it. Because the editor is bound to Yjs, the change becomes a normal CRDT update that reaches every collaborator. It replaces the selection if there is one, otherwise the whole file.

14.4 Backend service and error mapping

Backend/src/services/geminiService.js (abridged)

```
const callGemini = async (prompt) => {
  try {
    const result = await genAI.getGenerativeModel({ model: "gemini-2.5-flash"
}).generateContent(prompt);
    return result.response.text();
  } catch (error) {
    if (/SERVICE_BLOCKED|PERMISSION_DENIED|has not been used|is disabled/i.test(error.message))
      throw new AppError("The AI service isn't configured correctly: ... enable the Generative
Language API for the key.", 503);
    if (status === 401 || status === 403 || /API key not valid/i.test(error.message))
      throw new AppError("... the server's Gemini API key was rejected by Google.", 503);
    if (status === 429) throw new AppError("The AI service is over its quota right now. Try again
in a minute.", 503);
    throw new AppError("Failed to generate AI response. Please try again.", 503);
  }
};
```

Upstream failures become **503 Service Unavailable** with specific but safe messages. They never return 401, because that would make the frontend think the *user's* session expired. A real incident shaped this: a key whose project restricted the Generative Language API failed with `API_KEY_SERVICE_BLOCKED`, and the generic "failed" message hid the cause.

The `/review`, `/explain` and `/optimize` endpoints wrap the code in role-specific prompt templates (for example "You are a senior software engineer performing a code review … Code Quality, Potential Bugs, Performance, Security, Suggestions"). The current UI uses `/chat` with suggested prompts instead. **Partial**

14.5 The full-page Assistant

`pages/Assistant.jsx` uses `useChatStore`: a list of chats with titles, the active chat ID, and send/retry, all persisted in `localStorage` (keys `codefusionai_chats` and `codefusionai_active_chat_id`). The last 6 turns are included in each prompt. Chats are per browser and not synced to the server. That is a privacy plus, and a

portability minus.

14.6 Cost, safety and privacy considerations

- **Cost control:** per-user rate limit (30/min), context cap (12k chars), Flash-tier model.
- **Key safety:** the key lives only in `Backend/.env`. Calling Gemini from the browser would expose it in network traffic.
- **Prompt injection:** file contents are untrusted input to the model. A file could contain "ignore previous instructions". The impact is limited because the model has no tools and its output is only displayed or applied after a user click. Never auto-execute AI output.
- **Rendering safety:** replies are rendered with react-markdown, which does not render raw HTML by default, so model output can't inject scripts.
- **Privacy:** code is sent to Google. Enterprise users would need a data-processing agreement or a self-hosted model.
- **Improvements:** stream tokens (`generateContentStream` + server-sent events) for faster perceived latency; migrate to the newer `@google/genai` SDK; add retrieval across the project's files for multi-file questions.

Interview kit · AI integration

SIMPLE EXPLANATION

When you ask the AI something, the app grabs the code you're looking at, wraps it with your question, and sends it through our server to Google's Gemini. The answer comes back formatted, and you can paste suggested code into the file with one click.

TECHNICAL EXPLANATION

Client-side prompt assembly from the live Monaco model (selection-first, 12k-character cap, recent turns), sent through an authenticated, rate-limited endpoint that holds the API key and maps upstream errors to 503s. Responses render as sanitised markdown; Apply uses `executeEdits` so the change is undoable and flows through the CRDT.

SAY IT LIKE THIS

> *"The assistant is context-aware: at the moment you press send it reads the active file, or just your selection, straight from the editor, caps it at about twelve thousand characters, and adds the last few turns so follow-ups work. The request goes through my backend, so the Gemini key never reaches the browser, and it's rate-limited per user. If you click Apply, the code is inserted with Monaco's executeEdits, so it's one undo step and it syncs to collaborators through Yjs like any other edit."*

CODEFUSIONAI EXAMPLE

Select a 20-line function, click "Find bugs": only those 20 lines plus the file name and language are sent, and the reply's corrected function can be applied over exactly that selection.

LIKELY FOLLOW-UPS

- **How would you make it understand the whole project?** Retrieval: chunk files, embed them, store vectors, retrieve the top-k relevant chunks per question, and stay within the context budget.
- **How would you stream responses?** Use `generateContentStream`, forward chunks over SSE or the existing socket, and render incrementally.

- **What if Gemini is down?** The user sees a 503 message; the editor is unaffected. You could add a fallback provider behind an interface.

MISTAKES TO AVOID

- Saying the AI “edits files automatically”; it only suggests, and the user applies.
- Calling the frontend's direct Gemini usage “fine”. It must go through the backend.

CHAPTER 15

Frontend Walkthrough

15.1 Routing and component tree

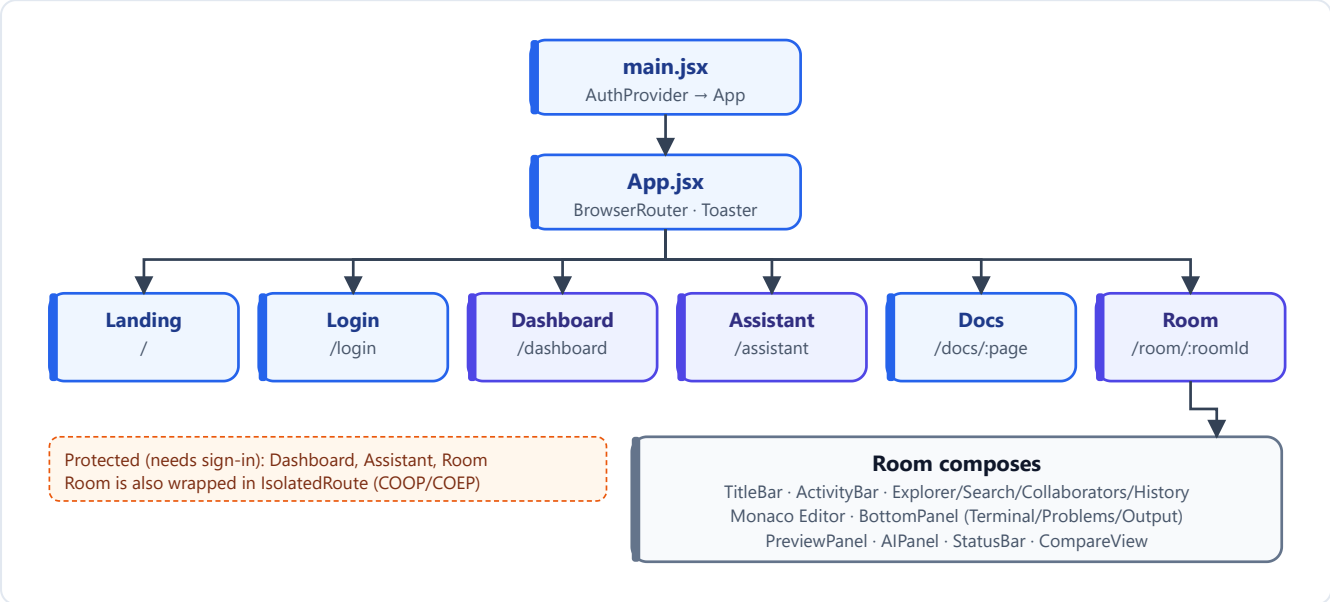


Figure 21. App structure: AuthProvider wraps the router; three protected routes; the Room page composes the workspace panels.

Frontend/src/App.jsx (abridged)

```
<Route path="/dashboard" element={<ProtectedRoute><Dashboard /></ProtectedRoute>} />
<Route path="/assistant" element={<ProtectedRoute><Assistant /></ProtectedRoute>} />
<Route path="/room/:roomId" element={
  <IsolatedRoute>              {/* reload once if the page isn't cross-origin isolated */}
    <ProtectedRoute>           {/* redirect to /login if signed out */}
      <Room />
    </ProtectedRoute>
  </IsolatedRoute>} />
```

- `AuthContext` subscribes to `onAuthStateChanged` and provides `{ user, loading }`. While `loading` is true, protected routes wait instead of redirecting, so you don't get a flash of the login page.
- `ProtectedRoute` renders `<Navigate to="/login" replace />` when signed out.
- `IsolatedRoute` solves a real bug. The COOP/COEP headers are only sent for `/room`, `/dashboard` and `/webcontainer`, because they break Google's sign-in popup elsewhere. Navigating client-side from `/docs` to a room keeps the old, non-isolated document, so WebContainers fail. `IsolatedRoute` checks `window.crossOriginIsolated` and reloads once (guarded by a `sessionStorage` flag to avoid infinite reload loops).

15.2 The Room page (the workspace)

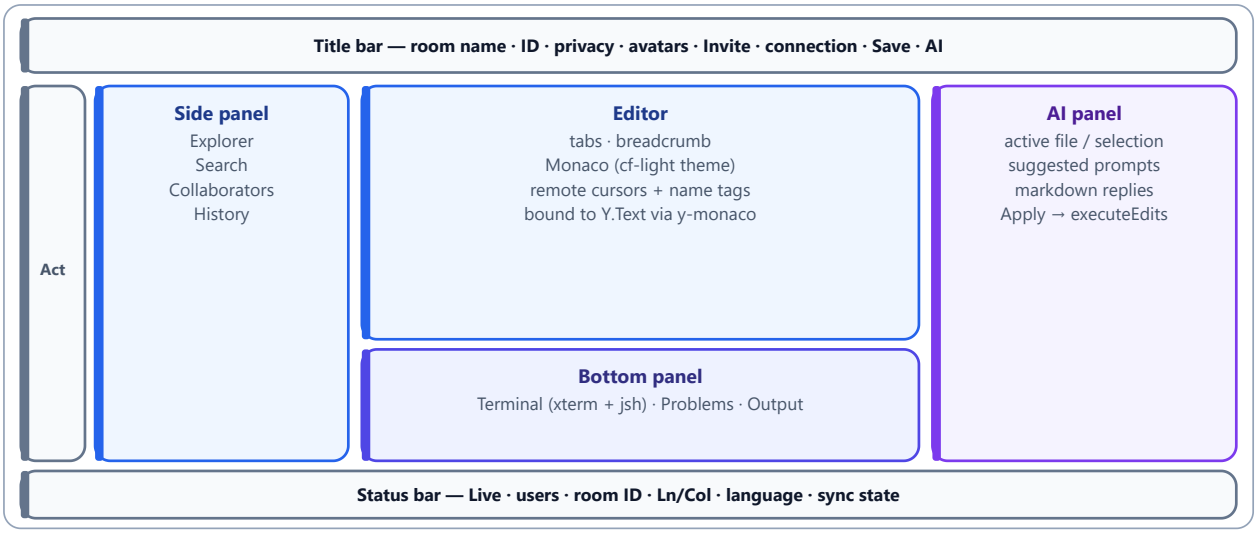


Figure 22. Workspace layout: title bar, activity bar, side panel, editor with bottom panel, AI panel, status bar.

`pages/Room.jsx` is the largest file (about 1,300 lines). Its responsibilities, in the order its effects run:

1. **Local-first paint:** loads the room's files from IndexedDB (written at creation), so the editor shows something immediately; a ref flag stops this slower load from overwriting newer server data.
2. **Room metadata:** fetches the user's rooms to show the room name and privacy in the title bar.
3. **Connection status:** `connect`, `disconnect` and `reconnect_attempt` drive the status badge.
4. **Socket wiring:** join on connect; handlers for `receive-code`, `yjs-sync/update`, `room-users`, `cursor-update`, `workspace-saved`, `version-restored`, `join-error`, `room-closed`; all are removed in the effect cleanup (otherwise each re-render would stack duplicate listeners).
5. **CRDT bridge:** sends local Yjs updates; mirrors Y.Text changes into the `files` state with `observeDeep` (used by the explorer, search and WebContainer sync).
6. **Editor binding:** when the active file changes, it destroys the old `MonacoBinding` and binds the new file's Y.Text; for a brand-new file it waits for the Y.Text to arrive.
7. **WebContainer sync:** debounced (1 s) mirroring of `files` into the container's file system.
8. **Shortcuts:** `Ctrl/⌘+S` emits `save-workspace`.

Frontend/src/pages/Room.jsx — binding Monaco to the CRDT

```
useEffect(() => {
  const editor = editorRef.current;
  if (!editorReady || !editor || !activeFile) return;
  const map = ydoc.getMap("files");
  let binding = null;
  const bind = () => {
    const text = map.get(activeFile);
    const model = editor.getModel();
    if (!text || !model || binding) return;
    binding = new MonacoBinding(text, model, new Set([editor]));
    bindingRef.current = binding;
  };
  bind();
  const onKeys = () => bind();               // new file: its Y.Text arrives a moment later
  if (!binding) map.observe(onKeys);
  return () => { map.unobserve(onKeys); binding?.destroy(); };   // unbind on file switch
}, [activeFile, editorReady, ydoc]);
```

15.3 Monaco setup

- **Path-based models:** `<Editor path={activeFile} />` gives each file its own model, so switching tabs keeps undo history and scroll position per file.
- **Workers:** `lib/monacoSetup.js` sets `self.MonacoEnvironment.getWorker` to return Vite-bundled workers (editor, JSON, CSS, HTML, TypeScript). JSX parsing is enabled, and semantic validation is turned off because imports resolve only inside the WebContainer.
- **Formatters:** built-in formatters for JS/TS/JSON/CSS/HTML; a simple brace-indent formatter is registered only for languages Monaco can't format (C++, Java, C#, Go, Rust, PHP).
- **Theme:** a custom light theme (`cf-light`) defined in `beforeMount` .
- **Problems panel:** `monaco.editor.onDidChangeMarkers` feeds diagnostics into the bottom panel; clicking one jumps to the line.

15.4 State management choices

State	Where	Why
Signed-in user	React Context	Needed everywhere, changes rarely
File contents	Y.Doc (source of truth) + `files` mirror	The CRDT merges; the mirror feeds the tree, search and the WebContainer
Editor, cursors, widgets	`useRef`	High-frequency updates must not re-render React
Toasts, confirm dialogs, palette	Tiny module stores (`toastStore` , `confirmStore`)	Callable from anywhere without prop drilling
AI chats	`localStorage` / `sessionStorage`	Survive reloads with no backend storage
Template seed	IndexedDB (`idb`)	Instant first paint for a new room

There is no Redux or Zustand: the data that really needs synchronising (code) already has a better store, the CRDT. Adding a global store would duplicate state.

15.5 Dashboard, login and design system

- **Dashboard:** stats strip, quick actions, recent-rooms table, public hub; create/join/settings modals; a `Ctrl+K` command palette. Room creation generates `cfai-xxxx-xxxx`, writes template files to IndexedDB, emits `create-room`, and navigates to the room.
- **Login:** sign in, create account (with full name, validation and password strength), Google sign-in, and password reset via Firebase.
- **Design system:** light-only tokens in `tokens.css` (paper/ink/rule/accent), reusable `.cf-*` components, `Modal` using the native `<dialog>`, toasts and confirm dialogs instead of `alert()` / `confirm()`, and Lucide icons.

Interview kit · Frontend architecture

SIMPLE EXPLANATION

React draws the screens. The code editor is VS Code's editor in the browser, wired to the shared document so everyone sees the same text. Pages that need sign-in send you to the login page if you're signed out.

TECHNICAL EXPLANATION

A React 19 SPA with route guards (auth and cross-origin isolation); Monaco with path-scoped models and Vite-bundled language workers; y-monaco binding per active file; the CRDT as the source of truth for content, with a derived React mirror; refs for high-frequency editor state; and effect cleanups that remove every socket listener.

SAY IT LIKE THIS

> *"The frontend keeps one rule: the Yjs document is the source of truth for code. Monaco is bound to the active file's Y.Text, and a React mirror of the files is derived from the doc for the explorer, search and WebContainer. Anything high-frequency, like cursors and content widgets, lives in refs so it doesn't re-render React. The room route is wrapped in a guard that reloads once if the page isn't cross-origin isolated, because WebContainers need SharedArrayBuffer."*

CODEFUSIONAI EXAMPLE

The socket effect's cleanup calls `socket.off(...)` for every event. Without it, navigating between rooms would attach duplicate `receive-code` handlers, and each update would be applied twice.

LIKELY FOLLOW-UPS

- **Room.jsx is 1,300 lines. Would you refactor it?** Yes: extract `useRoomSocket`, `useCrdtBinding`, `useRemoteCursors` and `useWebContainerSync` hooks, and add tests per hook.
- **How do you avoid re-render storms?** Refs for cursors, `useMemo` for derived data, and only presence lists in state.
- **How is Monaco kept out of the landing bundle?** It is imported only by the room route; route-level code splitting (`React.lazy`) would make the split explicit.

MISTAKES TO AVOID

- Saying the code is stored in React state; React mirrors the CRDT.
- Forgetting effect cleanups when explaining sockets in React.

CHAPTER 16

Running Code: WebContainers and /api/execute

16.1 In-browser execution with WebContainers

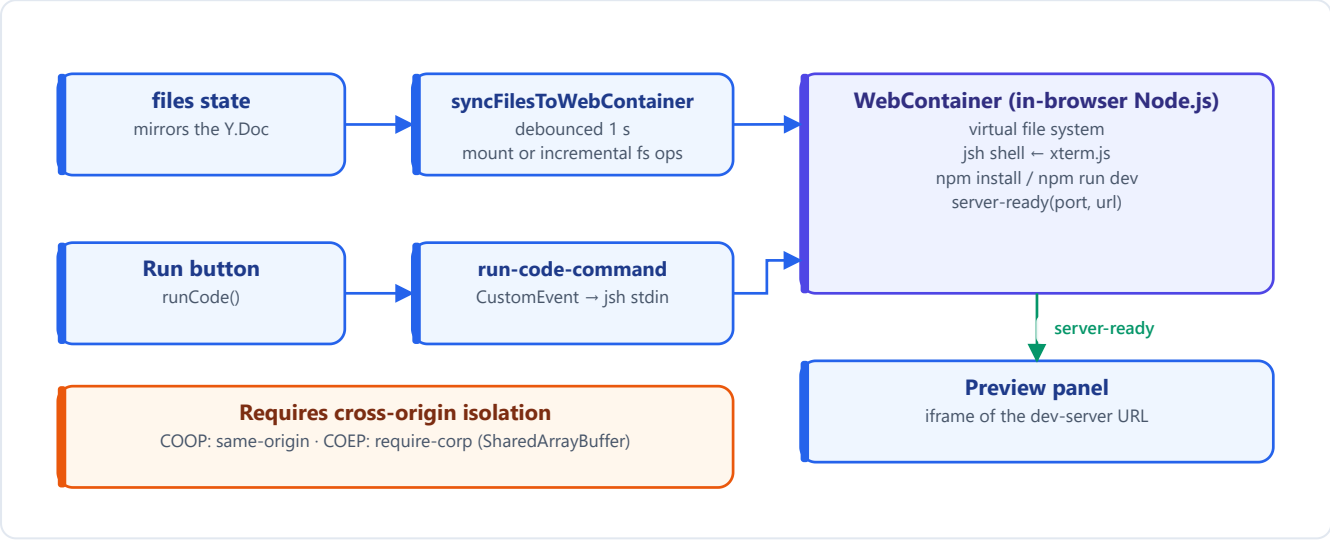


Figure 23. Files are mirrored into the WebContainer; Run writes a command into the shell; the dev server's URL opens in the preview.

Frontend/src/pages/Room.jsx — runCode (abridged)

```
await syncFilesToWebContainer(files);
const pkgFile = files.find((f) => f.path.endsWith("package.json"));
if (pkgFile) {
  const dir = pkgFile.path.split("/").slice(0, -1).join("/");
  const needInstall = await shouldRunNpmInstall(dir, pkgFile.content);   // no node_modules, or
package.json changed
  cmd = needInstall ? `cd "${dir}" && npm install && npm run dev\r` : `cd "${dir}" && npm run
dev\r`;
} else if (language === "javascript") {
  cmd = `node "${activeFile}"\r`;
} else {
  cmd = `echo "Only JavaScript/Node.js is supported in WebContainers natively."\r`;
}
window.dispatchEvent(new CustomEvent("run-code-command", { detail: { cmd } }));   // the terminal
writes it to jsh stdin
```

- **Boot once:** `WebContainer.boot()` is memoised in a module-level promise; only one container per page is allowed.
- **Smart sync:** the first sync, or more than 10 changes, mounts a whole file-system tree; otherwise it applies incremental `fs.writeFile` / `mkdir` / `rm` calls, compared against a map of the last synced contents.
- **Skip redundant installs:** it remembers the `package.json` content per directory and re-runs `npm install` only when it changed or `node_modules` is missing.
- **Preview:** `instance.on("server-ready", (port, url) => ...)` opens the preview panel with the dev server's URL.

- **Terminal:** xterm.js with the fit addon; each terminal tab spawns its own `jsh` process, and resizing is forwarded with `process.resize`.

WHY COOP/COEP ARE REQUIRED

WebContainers use `SharedArrayBuffer` and Atomics to emulate threads and synchronous I/O. Browsers re-enabled `SharedArrayBuffer` after the Spectre attacks only for cross-origin-isolated pages, so the page must send `COOP: same-origin` and `COEP: require-corp`. These headers are set in `vite.config.js` (dev), `vercel.json` and `netlify.toml` (prod). Worker scripts need them too; otherwise the isolated page refuses to start Monaco's workers.

16.2 Server-side execution: /api/execute

Backend/src/services/execute.service.js (abridged)

```
const runDir = path.join(TEMP_DIR, `run_${Date.now()}_${Math.random().toString(36).slice(2, 9)}`);
fs.writeFileSync(path.join(runDir, config.fileName), code);
exec(command, { timeout: 10000 }, (error, stdout, stderr) => {   // e.g. node "…/temp.js"
  cleanupRunDir(runDir);
  resolve(error ? { success: false, output: stderr || error.message } : { success: true, output:
stdout });
});
```

CRITICAL: THIS IS REMOTE CODE EXECUTION BY DESIGN

Any signed-in user can run arbitrary code with the backend's privileges. It can read `.env` (AWS, Mongo and Gemini secrets), call internal services, or fill the disk. The protections are a unique directory per run, a 10 s timeout, auth, and 10 requests per minute. None of them is a sandbox. The UI does not use this endpoint. In an interview, say exactly this, then give the fix: delete the route, or run code in a hardened sandbox (a per-run container or microVM with no network, a read-only root, CPU/memory/pids limits, a non-root user, seccomp), or call a dedicated service such as Judge0 or Piston.

Interview kit · Code execution

SIMPLE EXPLANATION

Instead of running people's code on our server (dangerous and expensive), we run it inside each user's own browser, in a mini Node.js that lives in the tab.

TECHNICAL EXPLANATION

WebContainers provide a WASM-based Node.js runtime with a virtual FS and service-worker networking. The app mirrors the CRDT's files into it incrementally, runs `npm install` only when `package.json` changes, pipes an xterm.js terminal to `jsh`, and opens a preview on `server-ready`. It requires cross-origin isolation.

SAY IT LIKE THIS

> *"Running untrusted code is the riskiest thing a code platform does, so I pushed execution to the client with WebContainers. Each user's code runs in their own browser sandbox, which costs us nothing and can't touch our servers. The trade-off is Node.js only and the COOP/COEP requirement. There's also a legacy /api/execute*

> *endpoint that runs code on the server without a sandbox; it isn't used by the UI, and I'd remove it or put it behind a proper container sandbox before production."*

CODEFUSIONAI EXAMPLE

A React template room: Run → `npm install && npm run dev` in the terminal → Vite starts inside the container → `server-ready` fires → the preview iframe shows the app, all without our backend.

LIKELY FOLLOW-UPS

- **How would you support Python or C++?** A sandboxed execution service (Firecracker/gVisor containers, or Judge0) with per-run limits and no network; or Pyodide (Python in WASM) in the browser.
- **Why do COOP/COEP break Google sign-in popups?** COOP same-origin severs the `window.opener` link the popup uses to report back, which is why the headers are applied only on routes that need them.

MISTAKES TO AVOID

- Calling `exec` with a timeout a "sandbox".
- Claiming WebContainers run any language.

CHAPTER 17

End-to-End Data Flow

One user's complete session, from opening the site to leaving, tying every chapter together.

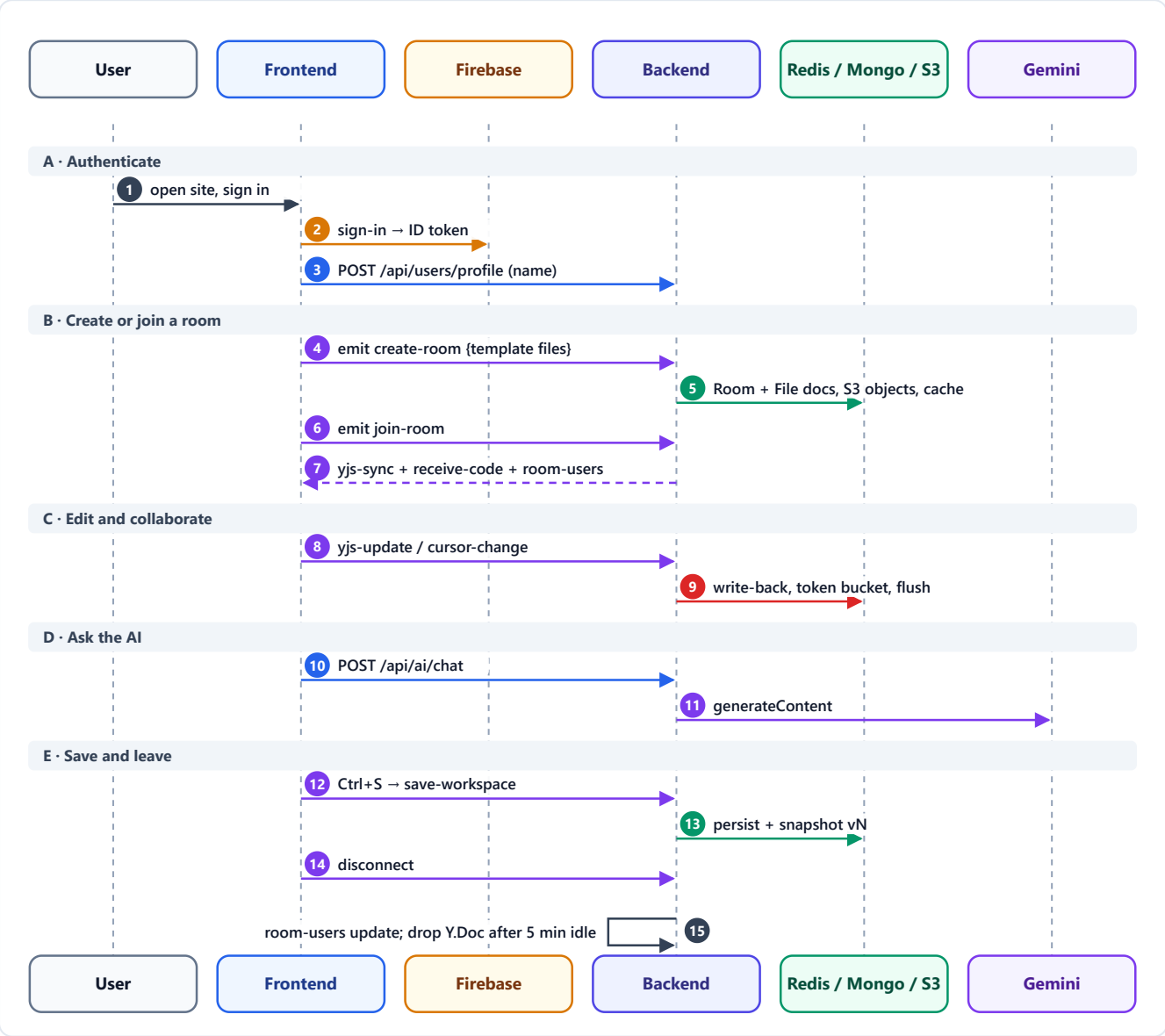


Figure 24. A complete session: authenticate, create/join, collaborate, ask AI, save, leave.

17.1 A · Open the site and authenticate

1. The browser downloads the Vite-built SPA from the static host. The landing page loads without Monaco.
2. The user signs in on `/login`. The Firebase SDK talks to Firebase and stores the session.
3. `AuthContext` receives the user; for a new account the display name is POSTed to `/api/users/profile` (token-verified, rate-limited, upserted by UID).
4. Navigating to `/dashboard`: `ProtectedRoute` passes, and the page fetches `/api/rooms/user/:uid` and `/api/rooms/public` in parallel.

17.2 B · Create or join a room

1. **Create:** the dashboard generates `cfai-xxxx-xxxx`, builds template files under a root folder, saves them to IndexedDB, and emits `create-room`.
2. The server creates the Room (owner from the token), uploads each template file to S3, upserts File docs, and caches the workspace in Redis.
3. The browser navigates to `/room/:id`; `IsolatedRoute` ensures isolation (reloading once if needed).
4. The Room paints from IndexedDB, connects the socket, and emits `join-room`; the server answers with `yjs-sync`, `receive-code` and `room-users` (Chapter 10).
5. **Join by ID:** the same, minus creation. The room must exist and not be closed.

17.3 C · Edit and collaborate

1. A keystroke changes the Monaco model; y-monaco converts it into a Y.Text operation, and the Y.Doc emits an update.
2. The client emits `yjs-update`; the server applies it and relays it to the others, whose editors update.
3. Cursor moves emit `cursor-change`; others render coloured cursors and name tags.
4. Within 400 ms the server writes the text back to Redis, marks the room dirty, and counts an edit token.
5. Creating, renaming or deleting a file emits the full list on `code-change`; the server reconciles it with the CRDT and broadcasts.
6. After 10 edit batches, or 10 s idle (checked every 30 s), a token-bucket-approved, locked flush writes S3 + Mongo and a snapshot; everyone sees “Saved”.

17.4 D · Run and ask AI

1. Run syncs files into the WebContainer and types the right command into the terminal; the preview opens on `server-ready`.
2. The AI panel reads the live file or selection, calls `/api/ai/chat`, and renders the reply; Apply inserts code as an undoable, synced edit.

17.5 E · Save, restore and leave

1. `Ctrl+S` emits `save-workspace`: bucket bypassed, lock taken, persisted, snapshot recorded, `workspace-saved` to all.
2. The History panel lists versions; Compare opens a diff; Restore creates a new version and updates every editor live.
3. Closing the tab disconnects the socket. `disconnecting` broadcasts the shorter user list; if the room is now empty, the Y.Doc is dropped after 5 minutes. Pending changes are already in Redis and will be flushed by the background interval once the idle rule triggers.
4. Owner actions (close, reopen, delete) happen from the dashboard via REST with owner checks.

PRACTISE THIS OUT LOUD

“Walk me through what happens when I type a character” is one of the most common deep-dive prompts. Use Section C as your script: model change → Y.Text op → binary update → socket → server apply + relay → other editors → 400 ms Redis write-back → dirty set + edit token → threshold or idle → token bucket → lock → S3 + Mongo + snapshot → “Saved”.

PART IV

Production Engineering

Security, testing and CI, deployment, scalability, reliability, monitoring, and what goes wrong in the real world.

CHAPTER 18

Security

18.1 Defence in depth

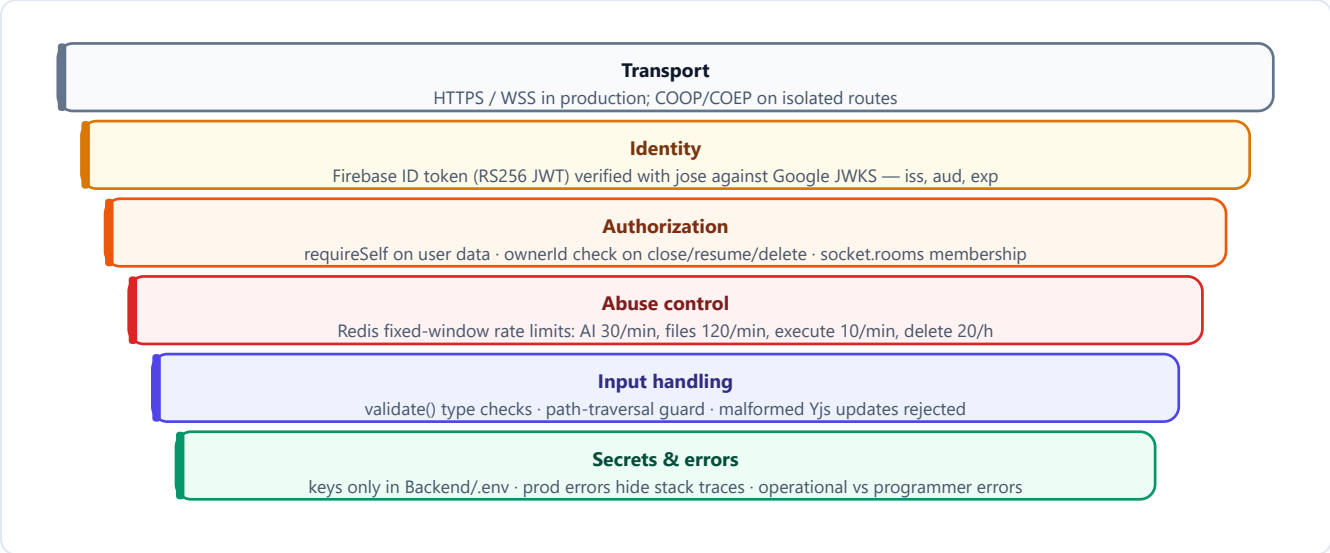


Figure 25. Security layers from the network edge inward. No single layer is trusted to stop everything.

Each layer assumes the one above it might fail. Even if CORS allowed a malicious origin, it would still need a valid token. Even with a valid token, owner-only actions check ownership. Even if someone floods a route, the rate limiter caps the damage.

18.2 Authentication versus authorisation recap

Authentication is implemented uniformly: every REST route except public listings and profile reads, and every socket connection, requires a verified Firebase JWT. Authorisation is implemented per action. Chapter 9 lists what is enforced and the known gaps (private-room join, history and file routes, `create-room` on an existing ID).

18.3 Secrets and environment variables

- Server secrets (`MONGO_URI`, `REDIS_URL`, `AWS_*`, `GEMINI_API_KEY`) live in `Backend/.env`, which is git-ignored; `Backend/.env.example` lists the names without values.
- Frontend `VITE_*` variables are **compiled into the public bundle**. Only non-secret values belong there. The Firebase web config (apiKey, projectId) is designed to be public; Firebase security comes from authorised domains and server-side token checks, not from hiding that key.
- `FIREBASE_PROJECT_ID` on the server is public too. The backend needs no Firebase secret because it verifies tokens with public keys.
- **Incident to learn from:** an older commit contained a MongoDB connection string. Rotating the database password is mandatory once a secret has been pushed, because git history keeps it even after the file changes. Tools like *gitleaks* in CI, or GitHub secret scanning, catch this earlier.

- **Production:** use the platform's secret manager (AWS Secrets Manager or SSM, or Vercel/Render environment settings) and IAM roles instead of long-lived AWS keys.

18.4 Input validation and injection

Threat	Mitigation in the code	Gap / improvement
Malformed JSON bodies	`express.json()` rejects invalid JSON with 400	Limit is 50 MB, too generous for most routes
Wrong types / missing fields	`validate()` middleware on AI, execute and file routes	No length or format limits; use Zod or Joi
NoSQL injection (`{"$gt": ""}`)	Queries use values like `{ roomId }` taken from route params (strings)	Body fields aren't type-checked everywhere; enable Mongoose `sanitizeFilter` or cast to String
Path traversal (`../../etc`)	`/api/filesystem/folder` resolves the path and checks it stays inside the room's workspace; `roomId` must match `^[a-zA-Z0-9_-]+$`	Good pattern; reuse it anywhere paths touch disk
Malicious CRDT updates	`applyClientUpdate` catches decode errors and rejects them	No size limit per update; cap it
Socket flooding / huge payloads	Membership checks	`maxHttpBufferSize: 1e9` (1 GB) lets one message exhaust memory; lower it and add per-socket event rate limits
XSS	React escapes text; markdown rendered without raw HTML; usernames set via `textContent` -like rendering	Cursor name tags use `domNode.innerHTML = username` , so a name like `<img onerror>` would execute. Switch to `textContent` . Add a CSP.
Command injection	n/a	`/api/execute` runs arbitrary code by design. Remove or sandbox it.

TWO CONCRETE FINDINGS TO FIX FIRST

1. `domNode.innerHTML = username` in the remote-cursor widget: a stored-XSS vector, because display names are user-controlled and shown to everyone in the room. The fix is one line: `domNode.textContent = username` . **2.** `/api/execute` : remote code execution for any signed-in user.

18.5 CORS

CORS is a *browser* mechanism. The server says which origins may read its responses; the browser enforces it. The backend currently reflects every origin with `credentials: true` . Since authentication uses a Bearer token in a header (not cookies), a malicious site cannot make requests *as* the user without first stealing the

token, so the practical risk is limited. Best practice is still an allow-list built from `FRONTEND_URL` (already in `.env.example`) plus known preview domains. Note that CORS does nothing against curl or server-to-server attackers; authentication is the real gate.

18.6 Rate limiting

Backend/src/middleware/rateLimit.js

```
export const rateLimit = ({ name, limit, windowSec }) => async (req, res, next) => {
  const identity = req.user?.uid || req.ip;                    // per user when signed in
  const window = Math.floor(Date.now() / 1000 / windowSec);     // e.g. minute number
  const key = `ratelimit:${name}:${identity}:${window}`;
  const count = await redisClient.incr(key);                   // atomic across instances
  if (count === 1) await redisClient.expire(key, windowSec);   // auto-cleanup
  res.setHeader("RateLimit-Remaining", String(Math.max(0, limit - count)));
  if (count > limit) { res.setHeader("Retry-After", String(resetIn)); return next(new
AppError(..., 429)); }
  next();
};
```

Route group	Limit	Reason
`/api/ai/*`	30 per minute per user	Each call costs Gemini quota
`/api/execute`	10 per minute	Expensive and dangerous
`/api/files/*`	120 per minute	Normal file operations
`DELETE /api/rooms/:id`	20 per hour	Destructive
History version fetch	60 per minute	Each hits S3
Profile update	20 per minute	Abuse prevention

Algorithm: fixed window. It is simple and cheap (one `INCR`), but allows up to 2× the limit across a window boundary (30 requests at 0:59 and 30 more at 1:00). A *sliding-window log* (sorted set of timestamps) or *sliding-window counter* fixes that, and the token bucket we already use for S3 is another option. **Fail-open:** if Redis errors, requests are allowed, choosing availability over strictness. **Atomicity nit:** if the process dies between `INCR` and `EXPIRE`, the key never expires; `SET key 0 EX w NX` then `INCR`, or a Lua script, fixes it.

18.7 File access permissions and data security

- S3 objects are reachable only via server credentials or short-lived presigned URLs (5 minutes).
- Presigned upload URLs are generated for any room ID the caller names, without a membership check. This needs `requireRoomMember`.
- MongoDB: use a least-privilege database user, IP allow-listing or private networking on Atlas, TLS (the default on Atlas), and backups.
- Public endpoints expose only safe fields: the public room listing deliberately `select`s fields without owner UID or participants.

18.8 Security headers and transport

Production must use HTTPS/WSS (the hosting platform's TLS). The backend could add `helmet` for standard headers (HSTS, X-Content-Type-Options, frame options), and the frontend a Content-Security-Policy. COOP/COEP already give the room pages a strong isolation boundary.

Interview kit · Security

SIMPLE EXPLANATION

We check who you are on every request, check what you're allowed to do, slow down anyone sending too many requests, keep secret keys only on the server, and never show internal errors to users.

TECHNICAL EXPLANATION

Stateless JWT authentication with full claim validation, action-level authorisation (identity from the token, owner checks, socket-room membership), Redis-backed fixed-window rate limits with standard headers, input validation, path-traversal guards, a production error handler that hides internals, and secrets only in server env.

SAY IT LIKE THIS

> *"Security is layered. Every API call and socket handshake carries a Firebase JWT that I verify locally, including issuer and audience. User IDs always come from the token, never the request body; owner-only actions are checked in the service layer; and socket events are ignored unless the socket joined that room. Expensive routes are rate-limited per user in Redis. I also know the gaps: private rooms aren't enforced on join, a couple of routes need a room-membership check, the cursor name tag uses innerHTML, and the legacy execute endpoint isn't sandboxed. I'd fix those first."*

CODEFUSIONAI EXAMPLE

An attacker loops `POST /api/ai/chat`: request 31 within the minute gets `429 Too many requests. Try again in 23s.` with `Retry-After: 23`, protecting the Gemini quota.

LIKELY FOLLOW-UPS

- **How would you handle a leaked JWT?** Short expiry limits the window; for sensitive actions, check revocation (`verifyIdToken(token, true)`), and revoke refresh tokens on password change.
- **Why rate-limit by user and not IP?** Many users share an IP (offices, NAT), and one attacker can rotate IPs; a signed-in identity is more precise. IP is the fallback for anonymous routes.
- **What's the OWASP category of the files-route gap?** Broken Object Level Authorization (OWASP API Security #1) / IDOR.

MISTAKES TO AVOID

- "HTTPS makes it secure": transport security doesn't fix authorisation.
- "The Firebase API key is a secret": it isn't, and saying so shows a misunderstanding.

CHAPTER 19

Testing and Continuous Integration

19.1 What is tested

File	Tests
`Backend/test/crdt.test.js` (5)	Seeding ignores folders; concurrent edits converge; reconcile keeps CRDT content, seeds new files, drops deleted ones; restore rewrites texts; malformed updates are rejected without throwing
`Backend/test/history.test.js` (3)	Normalisation sorts and strips; the hash is stable regardless of input order; the hash changes when any content changes
`Backend/test/ratelimit-and-auth.test.js` (8)	Limiter allows N then 429; limits are per user; bucket burst then throttle; bucket refills over time; edit-token threshold; missing/malformed token → 401; missing project ID → 500; `requireSelf` blocks other users
`Frontend/src/lib/rooms.test.js`	Template colours, relative-time formatting (`timeAgo` : this caught a bug that divided minutes by 24)
`Frontend/src/components/AIAssist/useChatStore.test.js`	Prompt building with recent history

Backend tests run with Node's built-in runner (`node --test`) and use the in-memory Redis fallback: CI points `REDIS_URL` at a closed port so the wrapper switches to MockRedis. That makes tests hermetic, with no external services needed.

Backend/test/ratelimit-and-auth.test.js (abridged)

```
test("token bucket refills over time (lastRefill must be persisted)", async () => {
  // drain the bucket, fake 10 s passing, expect two more flushes to be allowed
});
test("verifyIdToken rejects missing and malformed tokens with 401", async () => {
  await assert.rejects(verifyIdToken(""), { statusCode: 401 });
  await assert.rejects(verifyIdToken("not.a.jwt"), { statusCode: 401 });
});
```

19.2 CI pipeline

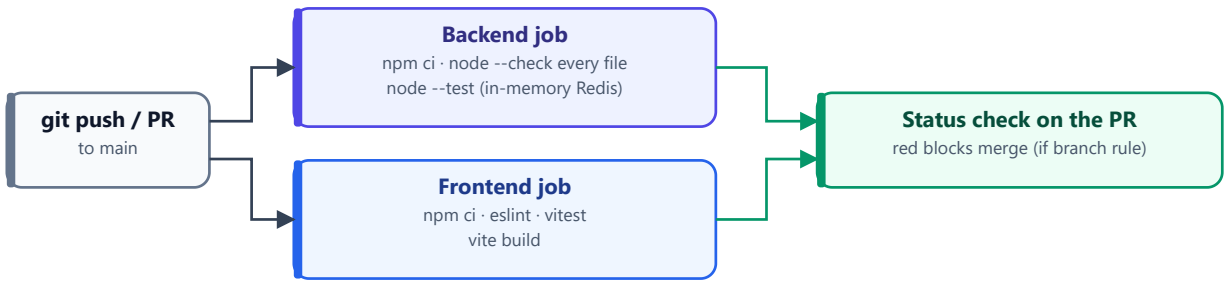


Figure 26. GitHub Actions: two parallel jobs on every push to main and every pull request.

- **Backend job:** `npm ci` (exact lockfile install), `node --check` on every source file (a syntax gate), `npm test`.
- **Frontend job:** `npm ci`, `eslint`, `vitest run`, `vite build` (which catches import errors and production-only failures).
- **Concurrency group** with `cancel-in-progress`: a new push cancels the outdated run for the same branch.
- **Caching:** `actions/setup-node` caches npm by lockfile path.

19.3 Testing gaps and what to add

- **Integration tests** for REST routes with `supertest` and an in-memory MongoDB (`mongodb-memory-server`), especially for authorisation rules.
- **Socket tests** with two `socket.io-client` instances against a test server: join, edit, converge, restore.
- **End-to-end tests** with Playwright: two browser contexts editing one room.
- **Load tests** with k6 or Artillery (which has Socket.IO support) to turn capacity claims into measurements.
- **Security in CI:** `npm audit`, Dependabot, secret scanning.

Interview kit · Testing

SIMPLE EXPLANATION

We wrote small programs that check the trickiest logic automatically (merging edits, version hashing, rate limits, login checks), and GitHub runs them every time code is pushed.

TECHNICAL EXPLANATION

Unit tests around the highest-risk pure logic (CRDT convergence, canonical hashing, limiter and bucket algorithms, JWT failure modes), hermetic through the in-memory Redis adapter, plus a CI gate on lint, tests and production build.

SAY IT LIKE THIS

> *"I focused tests where bugs would be silent and costly: CRDT convergence under concurrent edits, hash stability for version de-duplication, and the token bucket. One test exists because of a real bug: the Redis mock dropped multi-field HSETs, so the bucket never refilled and the code failed open, and the tests passed for the wrong reason. CI runs lint, tests and a production build on every push and PR."*

LIKELY FOLLOW-UPS

- **How would you test real-time sync end to end?** Spin up the server on a random port and connect two clients; or use Playwright with two browser contexts and assert both editors' text.
- **Coverage?** Not measured yet; I'd add `c8` or Vitest coverage and target the services layer.

MISTAKES TO AVOID

- Inflating the numbers: it is 16 backend + 7 frontend tests; say so.

CHAPTER 20

Deployment

20.1 Deployment architecture

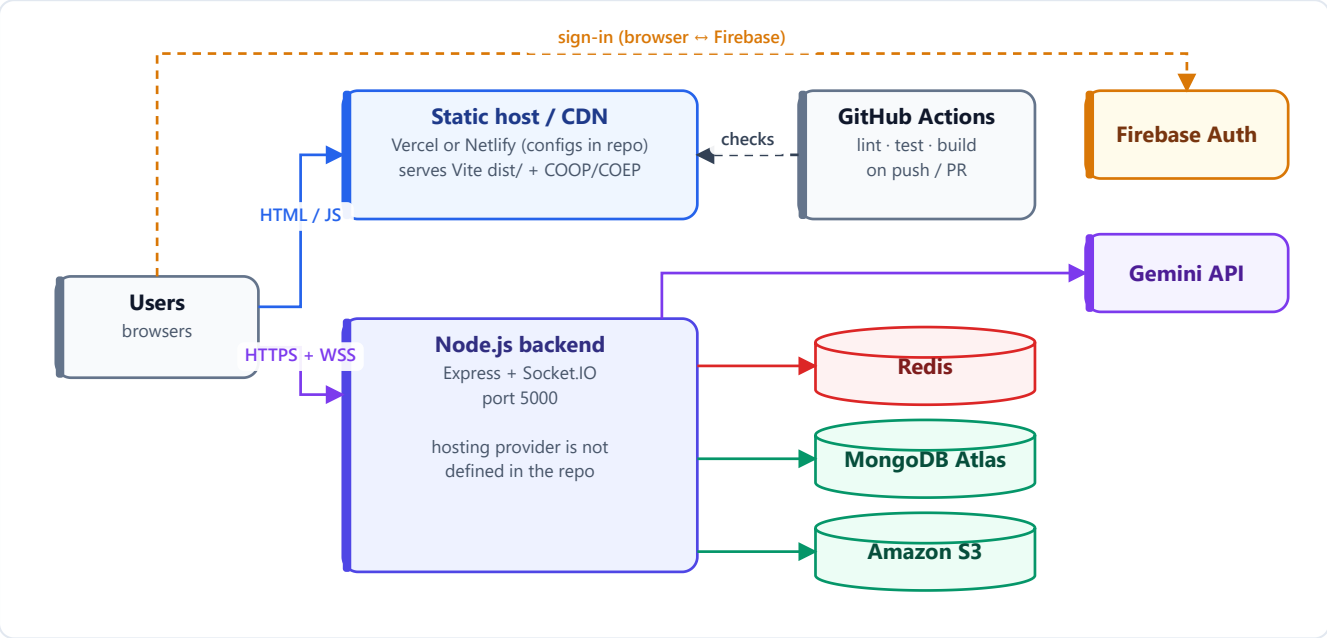


Figure 27. Deployment view. The static frontend host configs are in the repository; the backend's host is not.

- **Frontend:** `npm run build` produces static files in `dist/`. `vercel.json` adds the COOP/COEP headers and an SPA rewrite (all paths → `index.html`); `netlify.toml` does the same for Netlify. Both also send COEP/CORP for `/assets/*` so Monaco's worker scripts load on isolated pages. **Configs present**
- **Backend:** a long-running Node.js process (`npm start` → `node src/server.js`), because WebSockets need a persistent server, which rules out purely serverless functions. Suitable hosts include Render, Railway, Fly.io, AWS (EC2/ECS/App Runner) and Azure App Service. **Not defined in repo**
- **Managed data:** MongoDB Atlas (per the README), a hosted Redis (local instance or Upstash per the README), an S3 bucket, a Firebase project, and a Gemini API key.
- **Docker:** a Dockerfile per app plus a `docker-compose.yml` (backend, frontend dev server, `redis:7-alpine`) were added in commit `ccfdc28` and removed in `495f18d`. You can describe them as "previously containerised for local development", but they are not in the current tree.

20.2 Environment variables

Variable	Where	Purpose
`PORT`, `FRONTEND_URL`	Backend	Listen port; intended CORS origin
`MONGO_URI`	Backend	Atlas connection string (secret)
`REDIS_URL`	Backend	Redis connection (secret if it has a password)
`GEMINI_API_KEY`	Backend	Generative Language API key (secret)

Variable	Where	Purpose
`FIREBASE_PROJECT_ID`	Backend	Expected `aud` / `iss` for tokens (public)
`AWS_ACCESS_KEY_ID`, `AWS_SECRET_ACCESS_KEY`, `AWS_REGION`, `AWS_BUCKET_NAME`	Backend	S3 access (secrets; prefer an IAM role)
`VITE_FIREBASE_*`	Frontend	Firebase web config (public)
`VITE_BACKEND_URL`	Frontend	API and socket base URL

20.3 Production checklist and challenges

- ☐ Proxy/load balancer supports WebSocket upgrades and long idle timeouts (or Socket.IO falls back to polling).
- ☐ Single backend instance *or* sticky sessions + Socket.IO Redis adapter + shared CRDT strategy (Chapter 21).
- ☐ Real Redis available. Alert when the wrapper falls back to MockRedis (log line "Using in-memory fallback cache").
- ☐ CORS allow-list set from `FRONTEND_URL`.
- ☐ COOP/COEP on room routes and worker assets; verify `crossOriginIsolated === true` in production.
- ☐ Firebase authorised domains include the production domain.
- ☐ S3 bucket blocks public access; IAM scoped to the bucket.
- ☐ `/api/execute` removed or sandboxed; `maxHttpBufferSize` reduced.
- ☐ Graceful shutdown flushes dirty rooms before exit (to add).
- ☐ Health check wired to the platform; process restarts on crash.

Interview kit · Deployment

SIMPLE EXPLANATION

The website files go on a fast static host; the server that handles live editing runs somewhere that keeps a process alive; the databases are managed cloud services.

TECHNICAL EXPLANATION

Static SPA on a CDN with COOP/COEP and SPA rewrites; a stateful Node.js service (WebSockets and in-memory CRDT docs) on a long-running host; managed Atlas, Redis, S3, Firebase and Gemini; CI gates on lint, tests and build. Secrets are injected through environment variables.

SAY IT LIKE THIS

> *"The frontend is a static Vite build. Vercel and Netlify configs are in the repo, including the cross-origin isolation headers WebContainers need. The backend must be a long-running Node process because of WebSockets and the in-memory Yjs docs, so serverless functions don't fit. Data services are managed: Atlas, a hosted Redis, S3. The repo doesn't pin the backend host; I'd use a container platform, run a single instance at first, and add sticky sessions plus the Socket.IO Redis adapter to scale out."*

LIKELY FOLLOW-UPS

- **Why not deploy the backend on Vercel functions?** Functions are short-lived and stateless; they can't hold WebSocket connections or the room's Y.Doc.

- **Zero-downtime deploys?** Rolling deploy with connection draining: new instances come up, old ones stop accepting connections, clients reconnect (Socket.IO does this automatically), and dirty rooms are flushed on SIGTERM.

MISTAKES TO AVOID

- Claiming it is deployed on a specific platform if you can't show it.
- Saying Docker is used; it was removed from the tree.

CHAPTER 21

Scalability, Reliability and Monitoring

21.1 Where the current design stops scaling

Bottleneck	Why
Single Node process	Holds every socket and every active Y.Doc in its memory; one CPU core runs all JavaScript
In-memory CRDT per room	Two instances would each have their own Y.Doc for the same room, so edits on one wouldn't reach users on the other
Socket.IO broadcasts	`io.to(room)` only reaches sockets on the same process
Persist loop	Sequential S3 PUTs and Mongo upserts for *every* file on each flush
Background interval	Every instance would scan the same `dirty_rooms` set (the lock prevents double work, but the work isn't distributed)
`fetchSockets()` per presence change	O(room size) per event; fine for small rooms

21.2 Back-of-the-envelope capacity (estimates, not measurements)

Assumptions: 5 users per room, each user produces about 3 Yjs updates per second while typing (y-monaco batches), 30% of users typing at any moment, and an average update of about 50 bytes plus framing.

- **1,000 concurrent users (200 rooms):** about 900 updates/s arrive, and each is relayed to 4 peers, about 3,600 messages/s out. Node handles tens of thousands of small messages per second per core, so **one well-sized instance is likely sufficient**. Redis write-backs: at most 2.5/s per room × 200 = 500 small writes/s, trivial for Redis. S3: bounded by the bucket, at most about 12 flushes/min/room, but each flush uploads *every* file. That is the first thing to optimise (upload changed files only).
- **10,000 concurrent users (2,000 rooms):** about 9,000 updates/s in and 36,000/s out, plus 10,000 sockets' heartbeats and memory for 2,000 Y.Docs. This needs **horizontal scaling**.

SAY "ESTIMATE" OUT LOUD

No load test exists. Present the numbers as reasoning ("assuming..., the bottleneck would be...") and say you'd validate them with k6 or Artillery before committing to a capacity.

21.3 Horizontal scaling design

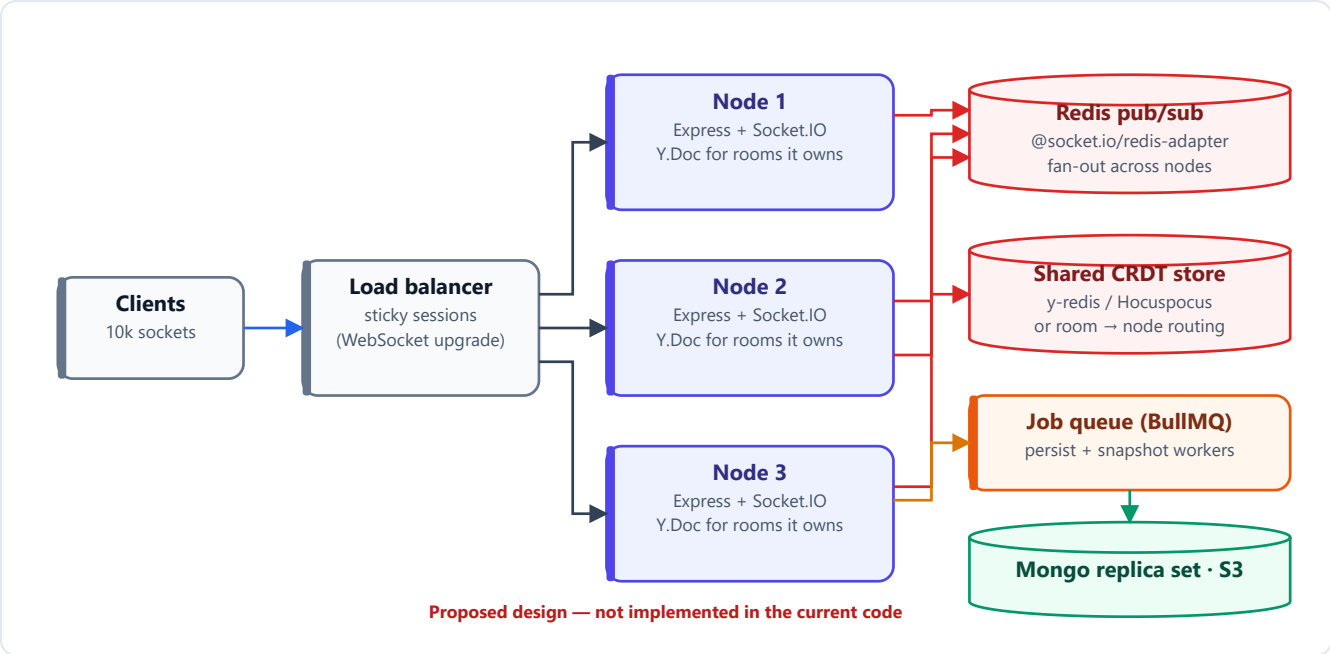


Figure 28. Proposed scaled-out architecture. None of this is implemented today.

1. **Sticky sessions at the load balancer** (cookie or IP hash). Socket.IO's polling fallback requires that a client's requests hit the same node.
2. **Room affinity:** route all sockets of one room to the same node (consistent hashing on `roomId`, for example via a room-aware gateway). Each Y.Doc then lives on exactly one node: the simplest model and the best fit for the current code.
3. **Or a shared CRDT layer:** `y-redis` or Hocuspocus with Redis distributes Yjs updates across nodes, so any node can serve any room.
4. **Socket.IO Redis adapter** (`@socket.io/redis-adapter`): `io.to(room).emit` publishes over Redis pub/sub so presence and cursors reach sockets on other nodes.
5. **Persistence workers:** move flushing out of the web process into a queue (BullMQ on Redis). A web node enqueues "flush room R" with a job ID of R (deduplicated); workers scale independently.
6. **Data tier:** a Mongo replica set (read scaling and failover); Redis with replication (Sentinel or a managed HA tier); S3 already scales.

21.4 Reliability and fault tolerance

Component fails	Current behaviour	Improvement
Backend process	All sockets drop; clients auto-reconnect (5 attempts); Y.Docs rebuilt from Redis/S3; ≤ 400 ms of edits may be lost	Process manager or orchestrator restarts; flush on SIGTERM; persist Yjs updates to a log (Redis stream)
Redis	Wrapper switches to in-memory mock: works, but not shared and not durable	Managed HA Redis; alert on fallback; refuse to start in production without Redis

Component fails	Current behaviour	Improvement
MongoDB	Start-up exits; runtime queries fail with 500	Atlas replica set (automatic failover); retryable writes
S3	Per-file upload skipped; snapshots stored inline	Retry with back-off; keep the room dirty on failure
Gemini	503 with a friendly message; editor unaffected	Circuit breaker; fallback model
Firebase JWKS	Cached keys keep working	None needed

21.5 Monitoring and observability

Today: `console.log` lines with prefixes (`[Socket]`, `[Workspace]`, `[History]`, `[CRDT]`, `[Redis]`) and a health endpoint. A production plan:

- **Structured logs** (pino) with roomId, userId and requestId fields, shipped to a log platform.
- **Metrics** (Prometheus/OpenTelemetry): connected sockets, active rooms and Y.Docs, Yjs updates/s, write-back latency, flush duration, flushes throttled by the bucket, lock contention, rate-limit 429s, Gemini latency and errors, event-loop lag, heap usage.
- **Alerts:** MockRedis fallback activated, persist failures, event-loop lag above 200 ms, error rate above a threshold, dirty-room count growing (flusher falling behind).
- **Tracing** across REST → service → Mongo/S3/Gemini to find slow paths.
- **Error tracking** (Sentry) on frontend and backend.

Interview kit · Scalability

SIMPLE EXPLANATION

Today one server keeps every live room in its memory, which is simple but has a ceiling. To grow, we'd run several servers, send everyone from the same room to the same server (or share room state through Redis), and move saving into background workers.

TECHNICAL EXPLANATION

The stateful parts are sockets and in-memory Y.Docs. Scale-out requires either room affinity (consistent hashing at a gateway) or a distributed Yjs provider (y-redis/Hocuspocus), plus the Socket.IO Redis adapter for cross-node broadcasts, sticky sessions for the polling fallback, and queue-based persistence workers. The data tier scales independently.

SAY IT LIKE THIS

> *"At a thousand users I'd expect one instance to cope, since that's a few thousand small messages a second, but I'd prove it with a load test first. The real limit is that each room's Yjs doc lives in one process's memory. To scale out I'd route all users of a room to the same node with consistent hashing, add the Socket.IO Redis adapter for cross-node events, and move S3 flushing into BullMQ workers. If rooms ever got too big for one node, I'd switch to a shared Yjs backend like y-redis."*

LIKELY FOLLOW-UPS

- **Why sticky sessions?** Socket.IO's HTTP long-polling sends several requests per session; they must reach the node holding that session.

- **What happens to rooms when a node dies under room affinity?** The hash ring reassigns its rooms; clients reconnect, and the new node rebuilds the Y.Doc from Redis/S3.
- **What would you measure first?** Event-loop lag and messages/s per node: they tell you when to add nodes.

MISTAKES TO AVOID

- "Just add more servers behind a load balancer." Without room affinity or a shared CRDT, users of the same room diverge.
- Presenting estimates as benchmarks.

CHAPTER 22

Troubleshooting, Failure Scenarios and Real Bugs

Real incidents from building CodeFusionAI. Each follows symptom → investigation → root cause → fix → lesson, which is exactly the shape interviewers want for "tell me about a hard bug".

22.1 Bug 1: concurrent edits overwrote each other

Symptom: two people typing in the same file; one person's text vanished. **Root cause:** every change sent the full file list and the server stored it whole: last-writer-wins. An intermediate fix (a Redis hash per file) solved *different*-file races but not same-file ones. **Fix:** Yjs CRDT with a server doc per room; structural changes reconciled so the CRDT stays authoritative for content. **Lesson:** concurrency bugs need a data-model fix, not more locking.

22.2 Bug 2: the token bucket never throttled

Symptom: flushes were never skipped, even under heavy edits. **Investigation:** logs showed the bucket "allowed" every time; adding a refill test reproduced it with the in-memory Redis. **Root cause:** the mock's `hset` ignored extra field/value pairs, so `lastRefill` wasn't saved, and `hgetall` returned `null`, which threw inside the check. The catch block fails open, so every check returned "allowed". **Fix:** make the mock follow Redis semantics; add a regression test. **Lesson:** fail-open paths hide bugs; test doubles must honour the real contract.

22.3 Bug 3: "crossOriginIsolated" error in the terminal

Symptom: the terminal printed "Error booting WebContainer" after entering a room from some pages but not others. **Investigation:** `window.crossOriginIsolated` was `false` after client-side navigation from `/` or `/docs`. **Root cause:** isolation is decided when a document loads, by its response headers. Client-side routing keeps the original document, which was served without COOP/COEP (deliberately, because they break Google sign-in popups). **Fix:** `IsolatedRoute` performs one guarded reload when the room is reached from a non-isolated document. **Lesson:** browser security state belongs to the document, not the route.

22.4 Bug 4: editor context-menu actions did nothing

Symptom: Go to Symbol, Change All Occurrences and Format Document silently failed. **Investigation:** the console showed a failing `Worker`; `self.MonacoEnvironment` was undefined. **Root cause:** (a) no worker factory was configured when importing Monaco's ESM build; (b) once added, worker scripts were blocked on the isolated room page because they lacked COEP headers; (c) a homemade formatter registered for JS/TS/JSON/CSS/HTML shadowed Monaco's real formatters. **Fix:** `lib/monacoSetup.js` with Vite `?worker` imports, isolation headers for worker files in dev and production configs, and custom formatters only for languages without built-ins. Verified in the browser: Format Document reformatted, Change All Occurrences selected both occurrences, and Go to Symbol listed the symbols.

22.5 Bug 5: AI answered about the wrong file

Symptom: the AI panel always discussed `App.jsx`. **Root cause:** stale React state copied from the editor; path-based models don't fire `onChange` when tabs switch. **Fix:** read text and selection from the live Monaco model at send time. **Lesson:** derive from the source of truth at the moment of use.

22.6 Bug 6: AI returned "failed to generate"

Symptom: every AI request failed although the key looked valid. **Investigation:** calling the API directly returned `403 API_KEY_SERVICE_BLOCKED`. **Root cause:** the Google Cloud key was restricted and did not allow the Generative Language API. **Fix:** allow the API on the key (or create a key in AI Studio); map upstream errors to specific 503 messages. An initial wrong guess blamed the key's format; the lesson is to reproduce against the upstream API before theorising.

22.7 Other bugs and edge cases worth knowing

- `EADDRINUSE :5000`: a previous backend still held the port. Find and stop it (`netstat -ano | findstr :5000` on Windows, `lsof -i :5000` on macOS/Linux), or change `PORT`. Also remember nodemon doesn't watch `.env`, so restart after changing secrets.
- **Relative-time bug:** `timeAgo` divided minutes by 24; a unit test now covers it.
- **Deleted files resurrecting:** `cacheWorkspace` only merges, so a deleted file stayed in the Redis hash. `replaceWorkspace` now deletes fields for missing paths.
- **Duplicate socket listeners** after navigating between rooms. Effect cleanups call `socket.off`.
- **Empty room:** `loadWorkspace` returns `[]`; snapshots are skipped for empty workspaces.
- **Closed room:** join returns an "Access Denied … reopen it from the dashboard" error, and the client navigates back.
- **Huge file pasted:** AI context is capped at 12,000 characters; the CRDT handles it, but WebContainer sync and flushes get slower.

22.8 Failure scenarios: what the system does

Scenario	Behaviour
Backend crashes while 3 users type	Sockets drop; clients show "Disconnected" and retry. On restart, a join rebuilds the doc from Redis (last write-back ≤ 400 ms before the crash). Edits typed while disconnected stay on screen but may not reach others (see the offline note in Chapter 10).
Backend crashes before cached data reaches S3	Data is still in Redis (2 h sliding TTL), and the room is still in `dirty_rooms`, so the restarted interval flushes it. Lost only if Redis also lost it (no persistence configured, or the in-memory fallback was active).
Redis restarts without persistence	Workspace hashes are gone; the next join rebuilds from S3, so unflushed edits since the last S3 flush (typically under ~40 s) are lost.
Two saves at once	The lock lets one through; the other returns false; the room stays dirty if needed.
A user's token expires mid-session	REST calls fetch a refreshed token automatically; the socket stays connected (verified at connect), and reconnections fetch a fresh token via the `auth` callback.

Scenario	Behaviour
Gemini quota exhausted	503 “over its quota… try again in a minute”; nothing else is affected.

22.9 Debugging toolkit

- **Browser DevTools:** Network → WS tab shows Socket.IO frames (look for `yjs-update` and `join-error`); the Console shows worker and COEP errors; `window.crossOriginIsolated` in the console.
- **Redis CLI:** `HGETALL room:<id>:token_bucket`, `SMEMBERS dirty_rooms`, `TTL room:<id>:persist:lock`.
- **Mongo:** `db.snapshots.find({roomId}).sort({version:-1}).limit(3)`; `.explain("executionStats")` to confirm index use.
- **Backend logs:** prefixed lines trace a flush (“Triggering S3 flush…”, “Persist complete”, “[History] Room → v12”).
- **Reproduce upstream:** call Gemini or S3 directly with the same credentials to separate “our bug” from “their error”.

Interview kit · “Tell me about a challenging bug”

SIMPLE EXPLANATION

Our save-rate limiter looked fine but never actually limited, because the fake database we used in tests behaved slightly differently from the real one, and the code quietly said “allow” whenever something went wrong.

TECHNICAL EXPLANATION

Contract drift in a test double (multi-pair `HSET` and the `HGETALL` empty-result shape), combined with a fail-open catch, produced an always-allow token bucket; a time-based refill test exposed it.

SAY IT LIKE THIS (STAR)

> *“Situation: I'd built a token bucket to cap S3 flushes per room. Task: verify it under load. Action: I wrote a test that drains the bucket, advances time and expects exactly two refills, and it failed. Tracing it, the in-memory Redis fallback only stored the first field of a multi-field HSET, so lastRefill never persisted, and HGETALL returned null instead of an empty object, which threw inside the check. The catch block fails open, so it always allowed. Result: I fixed the mock to match Redis semantics, kept the regression test, and learned to be suspicious of fail-open paths, because they turn bugs into silence.”*

LIKELY FOLLOW-UPS

- **Why fail open at all?** For a cost-control limiter, availability beats strictness, but it must be logged and alerted so it's never silent.
- **How do you prevent mock drift?** Run the same test suite against real Redis in CI (a service container), not only the mock.

MISTAKES TO AVOID

- Telling a bug story without the root cause and the lesson.

PART V

Interview Preparation

How to answer, what to say, and over 150 questions with model answers, from beginner to system design, plus deep follow-ups and full mock rounds.

CHAPTER 23

Answer Framework and Project Pitch

23.1 The 30-second, 2-minute and 5-minute pitches

30 SECONDS (HR / OPENING)

"CodeFusionAI is a real-time collaborative code editor with an AI assistant, like Google Docs for code. Several people edit the same files at the same time, run Node projects in the browser, and ask a Gemini-powered assistant about their code. I built it with React and Monaco on the front end, and Node, Socket.IO, Yjs, Redis, MongoDB and S3 on the back end."

2 MINUTES (TECHNICAL ROUND OPENER)

"The core problem is letting several people type in the same file without losing anyone's changes. My first version sent whole files and the last writer won, so I moved to Yjs, a CRDT: every keystroke becomes a small binary update that merges deterministically on every replica. Updates travel over Socket.IO; the server keeps one authoritative document per room and relays updates to the others. For storage, writing every keystroke to S3 would be slow and expensive, so the server writes the document back to Redis at most every 400 ms and marks the room dirty. A background job flushes dirty rooms to S3 for contents and MongoDB for metadata, throttled by a per-room token bucket and guarded by a Redis SET NX lock so only one save per room runs at a time. Every save records a de-duplicated snapshot, so users can compare and restore versions. Auth is Firebase on the client with JWT verification on the server. The AI panel sends the active file or selection through my backend to Gemini, and code runs inside the browser with WebContainers rather than on my server."

5 minutes: the 2-minute version plus one diagram on a whiteboard (the overall architecture, Chapter 4), one deep dive the interviewer chooses, and one honest limitation with its fix. Practise drawing the overall architecture from memory in under 90 seconds.

23.2 How to structure any technical answer

1. **Answer first** in one sentence ("We use a Redis SET NX lock with a 60-second expiry.").
2. **Why**: the problem it solves ("...so a manual save and the background flusher never write the same room at once.").
3. **How**: the mechanism, at the right depth ("SET with NX is atomic, so only one caller gets OK; the TTL prevents deadlock.").
4. **Trade-off or limitation** ("The release isn't owner-checked yet, so I'd add a token and a Lua compare-and-delete.").
5. **Stop** and let them pick the next thread. Don't monologue.

23.3 The STAR format for behavioural and project stories

Situation → **T**ask → **A**ction (most of the time, and specific) → **R**esult (measured or observable). Prepare three stories from Chapter 22: the last-writer-wins → CRDT redesign (impact), the token-bucket mock bug (debugging rigour), and the cross-origin isolation / Monaco worker issues (browser platform knowledge).

23.4 Phrases that signal seniority

- "For our constraints…" (choices depend on context).
- "The trade-off is…" (nothing is free).
- "I'd measure before optimising: specifically…"
- "Today it does X; the next step would be Y, because…" (honest about what exists).
- "The failure mode is…, and we handle it by…"

23.5 Red flags to avoid

- Claiming features or scale you can't demonstrate ("handles millions of users").
- Saying "we" for everything when asked what *you* did. Be precise about your role.
- Answering "why X" with "because it's popular / fast / modern".
- Getting defensive about weaknesses. Name them first; it builds trust.
- Reading code from memory wrong. Explain concepts, and quote only the lines you know.

CHAPTER 24

Question Bank: Beginner

Fundamentals and project basics, typically asked in the first 10 minutes or by a non-specialist interviewer. Answer briefly and clearly.

Q. What is CodeFusionAI, in one sentence? Beginner

A browser-based collaborative code editor where multiple people edit the same files in real time, run Node.js projects in the browser, and get help from an AI assistant.

Q. Why did you build this project? Beginner

To learn the engineering behind tools like Google Docs and VS Code Live Share: real-time sync, conflict resolution, caching and persistence trade-offs, authentication, and integrating an LLM. The result is something people instantly understand, and its hard problems are real distributed-systems problems.

Follow-up: Who is the target user? Students pair-programming, interview practice, small teams doing quick reviews, and teachers running live coding sessions.

Q. What is the difference between the frontend and the backend here? Beginner

The frontend (React in the browser) draws the UI, hosts the editor, holds a local copy of the shared document and runs code in WebContainers. The backend (Node.js) verifies identity, merges and relays edits, stores data in Redis, S3 and MongoDB, and calls the AI with a secret key.

Q. What is React and why use it? Beginner

React is a UI library where you describe the UI as a function of state; React updates the DOM efficiently when state changes. It suits a complex, stateful UI like an IDE, and its ecosystem had ready-made Monaco and markdown components.

Q. What is Node.js? Why is it good for this backend? Beginner

A JavaScript runtime built on V8 with an event loop and non-blocking I/O. It handles thousands of mostly idle WebSocket connections efficiently, and the same language (and the Yjs library) runs on both client and server.

Q. What is Express? Beginner

A minimal HTTP framework for Node. You register middleware functions that run in order for each request, and route handlers per URL and method. We use Express 5, which also forwards errors from async handlers automatically.

Q. What is an API? What is REST? Beginner

An API is a contract for how programs talk to each other. REST is a style of HTTP API built around resources (rooms, files) at URLs, manipulated with GET/POST/PUT/DELETE, where each request is self-contained (stateless). Example: `DELETE /api/rooms/:roomId`.

Q. What HTTP status codes does your API use? Beginner

200/201 for success; 400 for validation errors; 401 for missing or invalid tokens; 403 when you're not allowed (for example not the owner); 404 not found; 409 duplicate; 429 too many requests; 500 unexpected errors; 503 when the AI provider fails.

Q. What is a WebSocket? Beginner

A persistent, two-way connection between browser and server. It starts as an HTTP upgrade; afterwards either side can send messages any time with low overhead. It's what lets the server push someone's keystroke to you instantly.

Q. What is Socket.IO and how is it different from a WebSocket? Beginner

A library on top of WebSockets (with a long-polling fallback) that adds named events, rooms for group broadcasts, acknowledgements, automatic reconnection and binary support. It uses its own protocol, so it needs its own client library.

Q. What is a "room" in your app? Beginner

A shared workspace with an ID like `cfai-ab12-cd34`. In MongoDB it's a Room document (owner, name, template, status, privacy, participants). In Socket.IO it's a broadcast group (`socket.join(roomId)`) so edits go only to people in that room.

Q. What is MongoDB? What do you store in it? Beginner

A document database that stores JSON-like documents in collections. We store metadata: rooms, file records (path, language, size, version, S3 key), version snapshots and user profiles. We don't store file contents there.

Q. What is Mongoose? Beginner

An ODM (object–document mapper) for MongoDB: it adds schemas, validation (required, enum), defaults, timestamps, indexes and a model API like `Room.findOne({ roomId })`.

Q. What is Redis? Beginner

An in-memory data store with structures such as strings, hashes and sets. Commands are atomic and very fast. We use it as a write buffer for live workspaces, for locks, for rate-limit counters and for tracking which rooms have unsaved changes.

Q. What is Amazon S3? Beginner

Cloud object storage: you store bytes under a key in a bucket. It is extremely durable and cheap. We store every file's content and each version snapshot there.

Q. What is caching? Beginner

Keeping a fast copy of data that also lives somewhere slower. Our Redis hash per room is a cache of the workspace: a hit avoids reading from MongoDB and S3; a miss rebuilds it from them and fills the cache.

Q. What is authentication versus authorisation? Beginner

Authentication proves who you are (a valid Firebase token says you're uid X). Authorisation decides what you may do (only the room owner can delete it). They give different errors: 401 versus 403.

Q. How do users sign in? Beginner

With Firebase Authentication: Google sign-in or email and password, plus password reset. The browser receives an ID token, which the app sends to the backend on every request.

Q. What is a JWT? Beginner

A JSON Web Token: header, payload (claims such as user ID and expiry) and signature, base64url-encoded and joined by dots. The signature proves the token wasn't modified. It is signed, not encrypted, so anyone can read the payload.

Q. What is the Monaco editor? Beginner

The editor component behind VS Code, packaged for browsers. It gives syntax highlighting, IntelliSense, multi-cursor, a minimap and a diff view.

Q. What does the AI assistant do? Beginner

It answers questions about your code: explain, find bugs, optimise, write tests. It sees the open file or your selection, and you can apply its suggested code with one click.

Q. Which AI model do you use, and how? Beginner

Google's Gemini 2.5 Flash through the `@google/generative-ai` SDK, called from our backend so the API key stays secret.

Q. How do users run code? Beginner

With WebContainers: a Node.js runtime inside the browser tab. The Run button installs dependencies and starts the project in an in-browser terminal, and the app's preview opens in a panel.

Q. What is an environment variable and why use one? Beginner

A configuration value provided to the process by its environment rather than written in code, for example `MONGO_URI`. It keeps secrets out of the repository and lets the same code run in development and production with different settings.

Q. What is CORS? Beginner

Cross-Origin Resource Sharing: a browser rule that blocks a page from reading responses from another origin unless that server allows it via headers. It protects users' browsers; it doesn't stop non-browser clients.

Q. What is middleware in Express? Beginner

A function `(req, res, next)` that runs before the route handler. It can modify the request, end the response, or call `next()`. Ours do authentication, rate limiting, validation and error handling.

Q. What does "real-time" mean in your project? Beginner

Changes appear on collaborators' screens within milliseconds, with no refresh. It's achieved with WebSockets (Socket.IO) pushing small updates as they happen.

Q. What is version history? Beginner

Every save records a snapshot of the whole workspace (only if something changed). Users can see the list, compare a version with the current file, and restore it; the restore is saved as a new version.

Q. What testing did you do? Beginner

16 backend unit tests (CRDT convergence, history hashing, rate limiter, token bucket, authentication) with Node's test runner, and 7 frontend tests with Vitest. GitHub Actions runs lint, tests and a production build on every push and pull request.

Q. What was the most interesting thing you learned? Beginner

That correctness under concurrency is a data-model problem: moving from "send the whole file" to a CRDT fixed lost edits completely, where adding locks would only have made collaboration worse.

CHAPTER 25

Question Bank: Intermediate

Implementation and design questions. Expect the interviewer to push on the “why” and the mechanism.

Q. Walk me through what happens when a user types a character. Intermediate

Monaco's model changes; y-monaco converts that into a Y.Text operation in the local Y.Doc, which emits a binary update. The client emits `yjs-update` over Socket.IO. The server checks the socket is in that room, applies the update to the room's authoritative doc (rejecting malformed bytes), and relays it with `socket.to(room)`; the other clients apply it with origin “remote”, so they don't echo it back. Within 400 ms the server writes the doc's text into the room's Redis hash, adds the room to `dirty_rooms` and increments its edit counter. After 10 such batches, or 10 s idle, a flush, if the token bucket allows and the lock is free, writes S3 and MongoDB and records a snapshot.

Q. Why did you choose a CRDT over Operational Transformation? Intermediate

OT needs a central server to transform each operation against concurrent ones, and correct OT for rich editing is notoriously hard to implement. CRDTs make operations commutative by giving each character a unique ID, so any replica can merge in any order, with no transformation step and support for offline edits. Yjs is mature, fast for text and has a Monaco binding. The cost is extra metadata per character and in-memory docs on the server.

Follow-up: What do you lose? Some memory overhead (tombstones for deleted content) and a harder horizontal-scaling story, because the server holds docs in memory.

Q. Why does the server keep its own copy of the document? Intermediate

So a new joiner can get the full state instantly (`yjs-sync`) even if no other client is online; so persistence has an authoritative source to write back; and so the server can reject malformed updates and enforce membership. A pure relay couldn't do any of these.

Q. How do create, rename and delete work alongside the CRDT? Intermediate

The client sends the full file list on `code-change`. The server's `reconcileFiles` deletes Y.Texts for missing paths, seeds new ones, and builds the merged list using the CRDT's text for existing files, so a client's stale copy can't overwrite live typing. It broadcasts both the Yjs update and the new tree, then caches it with `replaceWorkspace`, which also removes deleted fields from the Redis hash.

Q. Explain your Redis key design. Intermediate

One hash per room (`room:{id}:files`, field = base64 path, value = file JSON) so files update independently; a set `dirty_rooms` for idempotent change tracking; a hash per room for the token bucket and edit counter; a string lock per room with `NX EX`; and fixed-window counters for rate limits. All key patterns live in one module (`redisKeys.js`), so there are no magic strings.

Q. Why a Redis hash per room instead of one JSON string? Intermediate

With one string, two concurrent writers each send the whole workspace and the last write wins for *all* files. With a hash, writing `App.jsx` doesn't touch `index.css`'s field, so writes to different files are independent, and each write is smaller.

Q. What is the token bucket for, and how does it work? Intermediate

It caps how often a room can flush to S3. Each room has up to 5 tokens, refilled at 1 per 5 s; a non-manual flush consumes one token or is skipped (the room stays dirty). The refill is computed lazily from the elapsed time on each check, so no timers are needed. This allows short bursts while bounding the long-run rate to about 12 flushes per minute per room.

Follow-up: Why not just a fixed interval? A fixed interval writes even when nothing changed and can't absorb bursts; the bucket plus the edit threshold reacts to activity while bounding cost.

Q. When exactly is a room flushed to S3? Intermediate

Three triggers: (1) a write-back pushes `editCount` to 10 or more, so an immediate attempt is made; (2) the 30-second background interval finds the room dirty with pending edits and idle for at least 10 s; (3) a manual save (Ctrl+S) or a restore, which bypasses the token bucket. Non-manual flushes need a token; all flushes need the lock.

Q. What does the persist lock protect against? Intermediate

Two persists of the same room running concurrently, for example a manual save during a background flush, or two backend instances. Without it, S3 writes could interleave, file version counters could double-increment, and snapshot numbering could race. `SET key 1 NX EX 60` is atomic, so exactly one caller wins; the TTL prevents a crashed holder from blocking the room forever.

Q. How do you load a room that isn't in Redis? Intermediate

Cache-aside: on a miss, query `File.find({ roomId })`, fetch each file's content from S3 (an empty string if a read fails), fall back to migrating the legacy `Room.files` JSON if there are no File documents, then fill the Redis hash with a 2-hour sliding TTL.

Q. How is version history de-duplicated? Intermediate

The workspace is canonicalised (sorted by path, normalised fields) and hashed with SHA-256. If the hash equals the latest snapshot's hash, no version is created. Sorting matters because the same files in a different order would otherwise hash differently.

Q. How does restore work without breaking collaborators' editors? Intermediate

The server rewrites the content of each existing Y.Text inside one transaction (`replaceContents`) and broadcasts the resulting update. Collaborators' editors stay bound to the same Y.Texts and change in place. The restored state is persisted as a new version with `restoredFrom` set, so the restore is undoable.

Q. How does the backend verify Firebase tokens without firebase-admin? Intermediate

With `jose`: `createRemoteJWKSet` on Google's securetoken JWKS URL (cached and rotated), then `jwtVerify(token, jwks, { issuer: "https://securetoken.google.com/<id>", audience: "<id>" })`, which checks the RS256 signature, issuer, audience and expiry; finally we require a `sub`. No service-account secret is needed.

Follow-up: Why does the audience matter? All Firebase projects share Google's signing keys; without `aud`, a token from another project would verify.

Q. How are WebSocket connections authenticated? Intermediate

The client passes the token in `handshake.auth` via a callback (so reconnections fetch a fresh token). `io.use(socketAuth)` verifies it before the connection is accepted and stores `socket.user`. Handlers take identity only from `socket.user`, and they ignore events for rooms the socket hasn't joined.

Q. Why are some events sent with `socket.to` and others with `io.to`? Intermediate

`socket.to(room)` broadcasts to everyone in the room *except* the sender, used for relaying a client's own update or cursor. `io.to(room)` includes the sender, used for server-originated changes (reconcile updates, restores, "workspace-saved") that everyone, including the requester, must apply.

Q. How do live cursors work? Intermediate

Monaco's cursor-position event emits `cursor-change` with the position; the server relays `cursor-update` with the user's active file. Other clients draw a coloured decoration and a name-tag content widget (hidden after 3 s idle), but only if the cursor is in the file they're viewing. Leaving users' cursors are removed when `room-users` changes.

Q. How do you prevent duplicate socket listeners in React? Intermediate

Every `socket.on` in an effect has a matching `socket.off` in the cleanup. Otherwise, re-running the effect (dependency change, StrictMode double-invoke, navigating between rooms) stacks handlers, and each event is processed several times.

Q. How does the AI panel know what code to send? Intermediate

At send time it calls `getContext()`, which reads the live Monaco model's value and the current selection. The selection is preferred, and the snippet is capped at 12,000 characters. An earlier version used a React state copy that went stale on tab switches, so the AI kept answering about App.jsx.

Q. Why call Gemini from the backend instead of the browser? Intermediate

The API key would be visible in browser network traffic and could be abused. Going through the backend also enables per-user rate limits (30/min), consistent error mapping, and a single place to swap models or add logging.

Q. What does your rate limiter do, and what are its weaknesses? Intermediate

A Redis fixed window: `INCR ratelimit:name:uid:window`, `EXPIRE` on first hit, 429 with `Retry-After` above the limit. It's cheap and shared across instances. Weaknesses: bursts of up to 2× the limit across a window boundary, `INCR` and `EXPIRE` aren't atomic together, and it fails open on Redis errors (a deliberate choice).

Q. Explain the MongoDB indexes. Intermediate

Rooms: `{ownerId, lastActive}` and the multikey `{participants, lastActive}` serve "my rooms, newest first"; `{accessType, lastActive}` serves the public hub. Files: unique `{roomId, path}`, which enforces one record per path and serves loads. Snapshots: unique `{roomId, version}`, which makes "latest version" the first index entry and prevents duplicate version numbers.

Q. What is a multikey index? Intermediate

An index on an array field. MongoDB creates an index entry per array element, so `{ participants: uid }` finds every room containing that UID through the index.

Q. How does the app handle errors consistently? Intermediate

Services throw `AppError(message, status)`; `asyncHandler` forwards rejections to `next`; the last middleware, `errorHandler`, returns `{ success:false, message }`. In production it hides unknown errors behind a generic 500 and maps Mongo errors (CastError/ValidationError → 400, duplicate key → 409). Socket handlers use try/catch and emit error events or acknowledgement errors.

Q. What is an operational error versus a programmer error? Intermediate

Operational errors are expected runtime conditions (not found, forbidden, rate limited, upstream down) with messages safe to show. Programmer errors are bugs (undefined property access), whose details must be logged but hidden from users.

Q. Why do the room pages need COOP and COEP headers? Intermediate

WebContainers need `SharedArrayBuffer`, which browsers only enable on cross-origin-isolated pages. That requires `Cross-Origin-Opener-Policy: same-origin` and `Cross-Origin-Embedder-Policy: require-corp`. We send them only on routes that need them, because COOP breaks OAuth popups.

Q. What does `IsolatedRoute` do? Intermediate

If the user reaches a room by client-side navigation from a non-isolated page, the document still lacks the headers. The component detects `!window.crossOriginIsolated` and reloads once (guarded by a sessionStorage flag to avoid loops), so the browser fetches the room page with COOP/COEP.

Q. How does the WebContainer file sync avoid being slow? Intermediate

It's debounced by 1 s; it compares against a map of the last synced contents and writes only changed, new or deleted paths, unless more than 10 paths changed, in which case it mounts a fresh tree. It also re-runs `npm install` only when `package.json` changed or `node_modules` is missing.

Q. Where are AI chats stored? Intermediate

In the browser: the full-page Assistant uses `localStorage`, and the room panel uses `sessionStorage` per room. They're not synced across devices, which is a privacy plus and a portability minus. Server-side storage with per-user encryption would be the upgrade.

Q. What happens when the owner closes a room? Intermediate

The service verifies ownership and sets `status = "closed"`; the socket handler broadcasts `room-closed` so connected users are sent to the dashboard; subsequent joins get “Access Denied … reopen it from the dashboard”. Only the owner can resume it.

Q. What does deleting a room clean up? Intermediate

After the owner check: S3 objects under the room's prefix, all snapshots (metadata and S3 JSON), File documents, the Redis workspace cache and the dirty flag, all in `Promise.allSettled`; then the Room document. Rate-limited to 20 per hour.

Follow-up: What if one cleanup fails? `allSettled` lets the others proceed, but orphans can remain. A periodic reconciliation job, or recording a “deleting” status and retrying, would make it robust.

Q. Why a 400 ms write-back instead of writing Redis on every update? Intermediate

Fast typists produce many updates per second; serialising the whole doc into Redis each time would waste CPU and Redis bandwidth. Coalescing into one write per 400 ms per room bounds the work, at the cost of at most ~400 ms of edits existing only in process memory.

Q. What is the in-memory Redis fallback, and is it safe? Intermediate

`MockRedis` implements the same commands in a JavaScript Map with timers. The wrapper switches to it if Redis is unreachable. It's great for local development and tests, but in production it silently loses sharing (multiple instances) and durability (process restarts), so it should trigger an alert or be disabled.

Q. What is a presigned URL and when would you use one? Intermediate

A URL signed with the server's AWS credentials that grants one operation (PUT or GET) on one key for a limited time (300 s here). The browser can upload or download directly to S3 without credentials and without streaming bytes through Node. The `/api/files/upload-url` and `/download/:id` routes generate them.

Q. Why is Room.jsx so large, and how would you refactor it? Intermediate

It grew as the integration point for sockets, the CRDT, cursors, the WebContainer, panels and shortcuts. I'd extract custom hooks (`useRoomSocket`, `useYjsBinding`, `useRemoteCursors`, `useWebContainerSync`, `useSaveShortcut`), each with focused tests, leaving Room as layout and wiring.

Q. How does the app behave on a slow or flaky network? Intermediate

Local edits apply instantly (the local CRDT replica), so typing never waits on the network. The status badge shows connecting or disconnected; Socket.IO retries 5 times, 1 s apart; on reconnect the client re-joins and merges the server state. The known gap is that offline-buffered updates may be dropped if they arrive before the re-join (Chapter 10).

Q. How do you choose the numbers: 400 ms, 10 edits, 10 s idle, 5 tokens, 5 s refill? Intermediate

They're tuning knobs balancing durability window, S3 cost and load. 400 ms keeps the in-memory loss window below human perception; 10 s idle means a pause saves soon after; 5 tokens refilling every 5 s caps a room near 12 flushes per minute. They live in one config object so they can be tuned from metrics. I'd tune them with real usage data.

Q. What does the background interval do differently from threshold flushes? Intermediate

Threshold flushes happen inline after a write-back when editCount ≥ 10. The interval (every 30 s) catches rooms that never reach 10, the “typed a little and stopped” case, by flushing those idle for at least 10 s with pending edits. Together they guarantee every change is eventually flushed.

Q. How is the user's display name shown to collaborators? Intermediate

Sign-up stores it on the Firebase profile and in our UserProfile collection via `POST /api/users/profile`. When joining, the Room page fetches the profile and sends the name and photo in `join-room`; the server attaches them to the socket and includes them in `room-users` and cursor name tags.

Q. What would you put in a code review of this project? Intermediate

Room-membership middleware for history and file routes; private-room enforcement; `textContent` instead of `innerHTML` for name tags; remove or sandbox `/api/execute`; a CORS allow-list; owner-checked lock release; atomic token-bucket and rate-limit scripts; flush on shutdown; upload only changed files; smaller body limits; Zod validation.

Q. How does the dashboard know which rooms to show? Intermediate

It calls `/api/rooms/user/:uid` (protected by `requireSelf`) and `/api/rooms/public` in parallel. The first returns rooms where you're the owner or a participant, sorted by `lastActive` through compound indexes; the second returns active public rooms with a limited field projection.

Q. What did you do for accessibility and UX quality? Intermediate

Native `<dialog>` modals (focus handling), labelled inputs, keyboard shortcuts (Ctrl+S, Ctrl+K palette), toasts and confirm dialogs instead of blocking alerts, empty, loading and error states, and responsive drawers for narrow screens.

CHAPTER 26

Question Bank: Advanced and System Design

Questions from senior engineers and system-design rounds. They test whether you understand failure modes, consistency, concurrency and scale, and whether you can reason honestly about limits.

Q. Is your CRDT setup strongly consistent or eventually consistent? Advanced

Eventually consistent with **strong eventual consistency**: any two replicas that have received the same set of updates are in the same state, without coordination. There is no global order of operations; a user may briefly see a state another user never saw. For collaborative text that's the right trade-off: availability and low latency beat linearizability (CAP: we choose AP for editing).

Q. How does Yjs order two concurrent inserts at the same position? Advanced

Each item stores its *origin* (left neighbour) and *right origin*. When two items share an origin, YATA's integration rules order them deterministically using their IDs (client ID comparison as the tiebreak), so every replica picks the same order. Positions are relative to items, never to absolute indexes.

Q. What are tombstones and why do they matter? Advanced

Deleted items stay as markers so concurrent operations that reference them can still be placed. Yjs garbage-collects deleted *content* but keeps small structural markers, so a document's memory grows with its edit history. For long-lived rooms, our periodic doc drop and rebuild from plain text (after 5 minutes idle) effectively compacts the history.

Follow-up: Any downside of rebuilding from plain text? Clients that still hold the old doc and reconnect have a different history; their updates won't merge cleanly with the new doc. That's acceptable only because the doc is dropped when everyone has left.

Q. What happens if a client reconnects after the server dropped and rebuilt the room's doc? Advanced

The rebuilt doc has new item IDs. If the client applies `yjs-sync` into its old doc, Yjs sees two independent histories and merges them, which can duplicate text. In practice the drop happens only 5 minutes after the room becomes empty, but a robust design would persist the Yjs binary state (`encodeStateAsUpdate`) to Redis/S3 instead of plain text, or have clients discard their local doc when the server's doc ID changes (a doc GUID or epoch in `yjs-sync`).

Q. Why not persist the Yjs binary state instead of plain text? Advanced

Plain text keeps S3 and history human-readable and decoupled from Yjs, and the original pipeline already stored text. The trade-off is losing CRDT history across doc rebuilds (the previous question). A strong design stores both: binary state for continuity (updates appended to a Redis stream, compacted periodically) and text snapshots for history and export.

Q. Is there any scenario where your system loses acknowledged edits? Advanced

Yes. (1) A process crash within the 400 ms write-back window. (2) Redis data loss before an S3 flush: no persistence configured, failover to a replica that lagged, or the MockRedis fallback being active. (3) The offline-buffer race on reconnect, where updates arrive before the re-join and are ignored. (4) An S3 upload failure followed by the room being marked clean. Each has a known fix: a Yjs update log, Redis AOF/HA, two-step state-vector sync, and keeping the room dirty on failure.

Q. Make the persist lock correct under process pauses and expiry. Advanced

Three changes. (1) Unique value: `SET lock <uuid> NX PX ttl`, and release with a Lua script: `if redis.call("get",KEYS[1])==ARGV[1] then return redis.call("del",KEYS[1]) end`. (2) Lease renewal (a watchdog extends the TTL while work progresses) or a TTL longer than the worst-case persist. (3) Fencing: an `INCR`-based token issued with each acquisition; storage writes carry it, and stale tokens are rejected, for example by snapshot inserts conditioned on version, or a `lastFence` field compared in a Mongo update filter.

Q. Is Redlock the answer? Advanced

Redlock acquires the lock on a majority of N independent Redis nodes, protecting against a single node failing over. But Martin Kleppmann's critique stands: without fencing tokens, no lease-based lock is safe under GC pauses or clock jumps. For our use (avoiding duplicate S3 work, where the result is idempotent-ish), a single-node lock plus fencing on the snapshot version is enough; for correctness-critical locks, use a consensus system (etcd, ZooKeeper) or fencing.

Q. Make the token bucket atomic across instances. Advanced

Move the read-refill-consume-write sequence into one Lua script executed with `EVAL` (or as a Redis Function): it reads `s3Tokens` and `lastRefill` with `HMGET`, computes the refill using a time passed in as an argument (or `redis.call("TIME")`), decrements if at least 1, writes back with `HSET` and `PEXPIRE`, and returns allowed or denied. Redis runs scripts atomically, so there are no interleavings.

Q. How would you scale to multiple backend instances? Advanced

Sockets and Y.Docs are the state. Option A, room affinity: a gateway or load balancer routes by consistent hash of `roomId`, so each room's doc lives on one node; add the Socket.IO Redis adapter for anything cross-room and sticky sessions for polling. Option B, a shared CRDT: y-redis or Hocuspocus with Redis distributes updates, so any node can serve any room. Either way, move persistence into a queue (BullMQ, deduplicated job per room) and run the flusher in workers.

Q. Design consistent-hash routing for rooms. Advanced

Nodes register in a ring (with virtual nodes for balance) stored in Redis or a service registry. The gateway hashes `roomId`, picks the owning node, and proxies the WebSocket there (the room ID goes in the connection URL or handshake). When a node leaves, only its rooms move; clients reconnect and the new owner rebuilds the doc. A brief ownership lease in Redis prevents two nodes believing they own a room during membership changes.

Q. What does the Socket.IO Redis adapter actually do? Advanced

It replaces the in-memory adapter: broadcasts (`io.to(room).emit`) are published to Redis pub/sub channels, and every node delivers them to its local sockets in that room. `fetchSockets()` becomes a cross-node request. It does not share Y.Docs; it only fans out messages.

Q. Where would back-pressure appear, and how do you handle it? Advanced

Slow clients: Socket.IO buffers messages per socket, so memory grows. Mitigations: cap the buffer, disconnect sockets with a large backlog, and for cursors use volatile emits (droppable). The flusher: if persists take longer than they arrive, the dirty set grows, so monitor its size. Gemini: rate limits plus upstream 429s.

Q. How would you make the persist step faster? Advanced

Upload only changed files (compare a per-file content hash stored in the File document or Redis); parallelise uploads and Mongo writes with a concurrency limit; batch Mongo with `bulkWrite`; move the work to queue workers; and for snapshots, store per-file content-addressed blobs so unchanged files aren't copied for every version.

Q. How do you guarantee version numbers are unique and monotonic? Advanced

Numbers are assigned inside the persist lock as latest + 1, and the unique index on `{roomId, version}` rejects a duplicate if the lock ever fails. A stronger approach is an atomic counter per room (`findOneAndUpdate` with `$inc` on a counters document, or Redis `INCR`) so numbering doesn't depend on the lock.

Q. What are the consistency issues between S3 and MongoDB? Advanced

There is no distributed transaction. A crash after the S3 PUT but before the Mongo update leaves new content with old metadata (size, version), which is harmless because the next flush fixes it. Deleting a room can orphan objects if some deletes fail. Patterns: write S3 first, then metadata (so metadata never points to missing content); make operations idempotent; and run a periodic reconciler that compares Mongo against an S3 listing. S3 Lifecycle can also expire orphaned prefixes.

Q. How would you implement private rooms properly? Advanced

Add `members` (with roles: owner, editor, viewer) and an `invites` collection with signed, expiring tokens. `join-room` and a shared `requireRoomMember(role)` middleware check membership for private rooms; public rooms allow anyone to join as an editor or viewer. Viewers' `yjs-update` events are rejected server-side. Use `crypto.randomUUID()` for room IDs and invite tokens.

Q. How would you add read-only viewers? Advanced

Store the role on the socket at join (`socket.role`). In `yjs-update` and `code-change`, reject when the role is viewer; on the client, set Monaco to `readOnly`. Enforcement must be server-side, because the client flag is only UX.

Q. What's the attack surface of the WebSocket layer? Advanced

Message size (1 GB buffer: reduce it), message rate (no per-socket limit: add a token bucket per socket), malformed payloads (Yjs decode is guarded; other payloads should be schema-validated), room hopping (membership checks exist except the `create-room` edge case), Cross-Site WebSocket Hijacking (mitigated because auth is a token in the handshake, not a cookie), and long-lived sessions past token expiry (re-verify periodically).

Q. How do you protect against a user crashing the server with a huge CRDT update? Advanced

Cap `maxHttpBufferSize` (for example 1–5 MB), reject updates above a byte limit before decoding, cap document size per room (reject updates that would exceed N MB), and rate-limit updates per socket. Decoding is synchronous on the event loop, so huge updates also cause lag for everyone.

Q. How would you support 100 people in one room? Advanced

Fan-out becomes N^2 for cursors: throttle cursor events (for example 10/s per user), send them as volatile, and batch them server-side. Use Yjs awareness, whose updates are small and merge. Replace `fetchSockets()` per presence change with an incremental user map. Text updates are fine: they're tiny and relayed once each.

Q. The event loop is blocked: how would you find out and what could cause it? Advanced

Measure with `perf_hooks.monitorEventLoopDelay` and alert on p99. Likely culprits: decoding or encoding very large Yjs docs, `JSON.stringify` of huge workspaces in write-back or hashing, SHA-256 of large snapshots, and sync file-system calls in the execute service. Fixes: size caps, worker threads for hashing and serialisation, and incremental write-back (only changed files).

Q. How would you implement presence correctly across nodes? Advanced

Keep presence in Redis (a hash per room: socketId → {user, file, lastSeen}) with heartbeats and TTL-based expiry, published via the adapter. Or use Yjs awareness, which already handles timeouts (30 s) and propagates through the CRDT channel.

Q. How does your design behave during a Redis failover? Advanced

ioredis reconnects; with our wrapper, a second failure switches permanently to MockRedis, which splits state between instances and loses it on restart. For production I'd remove the permanent switch (retry with back-off instead), use a managed HA Redis with AOF, accept that a failover may lose the last second of writes (async replication), and rely on S3 and history as the durable layer.

Q. How would you do zero-downtime deploys with stateful sockets? Advanced

On SIGTERM: stop accepting connections, flush every dirty room and persist Y.Doc state, then close sockets with a "server restarting" reason. Clients auto-reconnect to new instances, which rebuild docs from storage. With room affinity, drain rooms node by node; keep the old version alive until its sockets are gone.

Q. How would you add GitHub integration (import, commit, push)? Advanced

A GitHub App with per-user OAuth; import uses the Contents or Git Trees API into a new room; commit builds a tree from the current workspace (blobs, tree, commit, update ref) via the Git Data API, or runs isomorphic-git inside the WebContainer. Store tokens encrypted; scope permissions minimally. Proposed

Q. How would you add multi-file AI context ("explain this project")? Advanced

Index the room: chunk files by function or class, embed them with an embedding model, store vectors (pgvector, Atlas Vector Search) keyed by room, path and hash; re-embed changed files on flush. At query time, retrieve the top-k chunks plus the active file within a token budget and cite paths in the answer. Add caching per (question, context hash). Proposed

Q. How would you add an undo that's per-user in a shared document? Advanced

Yjs `UndoManager` scoped to the user's own origin (`trackedOrigins`) undoes only that user's changes, even if others edited since. y-monaco integrates with it; Monaco's own undo stack would otherwise mix in remote edits.

Q. How would you observe whether collaboration is "healthy" in production? Advanced

SLIs: update relay latency (server receive → peer apply, measured with client timestamps), reconnect rate, divergence checks (periodically compare a hash of the client doc with the server doc), write-back lag, flush lag (time from dirty to clean), persist error rate. SLOs on p95 relay latency and maximum flush lag.

Q. Design the whole system for 1 million registered and 50,000 concurrent users. Advanced

Edge: CDN for the SPA; a WebSocket gateway tier with consistent-hash routing by room. Collaboration tier: stateless-ish Node workers owning room shards (Yjs docs in memory with an update log in Redis Streams or Kafka and periodic binary snapshots to S3). Persistence workers consume the log and write text snapshots and metadata. Data: a sharded Mongo (hashed `roomId`) or Postgres with partitioning; S3 for blobs; Redis Cluster for presence, rate limits and queues. AI: a separate service with a queue, per-tenant quotas and streaming. Identity: Firebase or an OIDC provider; per-room ACLs. Observability: metrics, tracing, SLOs. Capacity: roughly 10k sockets and a few thousand active rooms per node means about 5–10 collaboration nodes plus headroom, to be validated by load tests.

Q. What is the CAP theorem and where does your design sit? Advanced

Under a network partition, a distributed system must choose between consistency (every read sees the latest write) and availability (every request gets a response). Editing is AP: each client keeps working on its replica and merges later. Room metadata in Mongo with majority writes leans CP. The persist lock is a CP-style mechanism, which is why it should fail closed.

Q. Why is fail-open right for the rate limiter but fail-closed right for the lock? Advanced

The limiter protects cost and fairness; refusing all traffic during a Redis blip would cause an outage worse than a burst of unthrottled requests. The lock protects correctness; proceeding without it risks interleaved writes and corrupted versions, whereas skipping a save just delays it (the room stays dirty).

Q. How do you test distributed behaviour like the lock and the token bucket? Advanced

Unit-test the algorithm with an injectable clock; integration-test against a real Redis in CI (a service container) with concurrent promises racing `acquire`; fault-inject delays (a persist that exceeds the TTL) to prove the owner-checked release; and property-test CRDT convergence with random operation sequences and orders.

Q. If you rebuilt it today, what would you change architecturally? Advanced

Use the standard Yjs sync protocol and awareness from day one (y-websocket or Hocuspocus semantics over Socket.IO), persist Yjs binary state alongside text, put a room-membership and role model in the core, run persistence as queue workers, and use Postgres for relational membership and ACLs (or keep Mongo but with a members collection). Keep WebContainers, and drop server-side execution.

CHAPTER 27

Tough Follow-Ups, Answered in Depth

The questions interviewers use to separate “built it” from “understands it”. Each has a model answer, follow-ups and traps.

Q. Why did you use Socket.IO instead of ordinary HTTP polling?

Answer. Collaboration needs the server to push changes the moment they happen, many small messages per second per user, in both directions. Polling either adds latency (up to the interval) or wastes requests (most return nothing), and each request repeats headers and the TLS/HTTP overhead. A WebSocket is one long-lived connection with tiny frames in both directions. I chose Socket.IO over raw WebSockets because I needed rooms for per-room broadcast, automatic reconnection, acknowledgements (used by restore-version), binary support for Yjs updates, and an auth middleware hook.

Numbers to quote: with 1-second polling and 5 users, that's 5 requests/s per room even when idle; with sockets, zero messages when idle and about 10–50 ms delivery when active.

Follow-ups: *Long polling?* Better latency than polling but still a request per message and awkward bidirectionality. Socket.IO actually uses it as a fallback. *SSE?* One-way only. *Downsides of Socket.IO?* A proprietary protocol, sticky sessions for multiple nodes, and per-connection memory.

Q. Why did you choose MongoDB for metadata?

Answer. The metadata is naturally document-shaped (a room with an array of participants; a snapshot with an array of file summaries), the schema evolved during development, and every query is a single-collection lookup that a compound index serves: rooms by owner or participant sorted by activity, files by room and path, snapshots by room and version. Atlas gave a managed, replicated cluster for free. Mongoose added validation and enums.

Honest trade-off. PostgreSQL would also work well and would add foreign keys and transactions, useful for room deletion and membership. If membership and permissions grow (roles, invites, organisations), relational modelling becomes more attractive.

Follow-ups: *How do you keep data consistent without foreign keys?* In the service layer (deleteRoom cleans files, snapshots, S3 and cache) plus unique indexes as invariants. *When would you pick Postgres?* Complex relationships, reporting and joins, or strict transactional requirements.

Q. Why use Redis for caching and locking?

Answer. Three needs: sub-millisecond reads and writes for live workspaces (RAM), atomic operations visible to every backend process (single-threaded command execution makes `SET NX`, `INCR` and `HINCRBY` atomic), and automatic expiry (TTL) so locks and caches clean themselves up. Rich types matter: hashes for per-file storage, sets for idempotent dirty tracking, counters for rate limits and buckets. One tool covers caching, locking, rate limiting and change tracking.

Follow-ups: *Why not an in-process Map?* Not shared across instances and lost on restart; that's exactly what the dev fallback is. *Why not Mongo for locks?* Possible (a unique index plus TTL index), but slower, and TTL indexes are swept only about every 60 s. *What if Redis is down?* Today it falls back to memory, which I'd replace with HA Redis plus alerts.

Q. Why store files in Amazon S3 instead of MongoDB?

Answer. File contents are blobs read and written whole. They need durability and cheap storage, not queries. S3 is designed for eleven nines of durability, scales without limits, costs far less per GB than database storage, offers presigned URLs for direct browser transfer, and keeps MongoDB documents small, which matters given the 16 MB BSON limit and because big documents bloat the working set and replication. MongoDB holds the metadata that *is* queried (path, size, version, s3Key).

Follow-ups: *GridFS?* Avoids a second service, but puts blob traffic on the database. *Latency of S3?* Tens of milliseconds per request, which is why Redis sits in front. *Cost control?* The token bucket caps PUTs per room; uploading only changed files is the next optimisation.

Q. How do you prevent two users from editing the same file simultaneously?

Answer. We deliberately *don't* prevent it, because that's the point of a collaborative editor. Instead we make simultaneous editing safe. Each file is a Yjs Y.Text, a CRDT, so concurrent edits from different users merge deterministically and every replica converges. No edit is lost, and nobody waits for a lock. The only lock in the system is the Redis persist lock, which serialises *saving* a room, not *editing*.

History worth telling. The first version sent whole files and the last writer won, which lost edits. A per-file Redis hash fixed different-file races; the CRDT fixed same-file races. A per-file editing lock was the alternative: correct, but it destroys real-time collaboration and needs lock expiry and waiting UX.

Follow-ups: *What if two people rename the same file?* Structural changes are full-list messages; the last one processed wins for the tree, while contents stay in the CRDT. *Could you offer optional locking?* Yes, as a soft "someone is editing this function" indicator via awareness, or a read-only role.

Q. What happens if the backend crashes before cached data is saved?

Answer, layer by layer.

- **Edits within the last ~400 ms** exist only in the server's in-memory Y.Doc (and in the clients' docs). They are lost on the server; clients still show them, and they reach the server again only if sent after reconnection (see the offline race).
- **Edits already written back** are in Redis, and the room is in `dirty_rooms`. On restart, the background interval flushes them to S3 and MongoDB. Redis has a 2-hour sliding TTL, so there's ample time.
- **A lock held during the crash** expires within 60 s; the next flush proceeds.
- **If Redis also lost the data** (no AOF/RDB, a failover, or the in-memory fallback), edits since the last S3 flush are lost, typically under ~40 s of work given the 10 s idle and 30 s interval.

Improvements: flush dirty rooms on SIGTERM; append Yjs updates to a Redis stream (a durable log) and replay it on restart; Redis AOF with `everysec`; keep rooms dirty when an S3 upload fails.

Follow-ups: *What's your RPO?* About 400 ms for a server crash with durable Redis; up to roughly the flush interval if Redis loses data. *RTO?* Process restart plus reconnect, typically seconds; docs rebuild lazily on join.

Q. How would you handle 1,000 or 10,000 concurrent users?

1,000: estimate first. About 200 rooms × 5 users; about 900 Yjs updates/s in and about 3,600/s relayed; Redis write-backs at most 500/s. One Node instance with enough memory should cope, with the S3 flush (which uploads every file) as the first hotspot to fix. Then prove it with a k6 or Artillery load test that tracks event-loop lag, p95 relay latency and memory.

10,000: scale out. Room-affinity routing (consistent hashing on roomId) so each Y.Doc lives on one node, the Socket.IO Redis adapter for cross-node events, sticky sessions for the polling fallback, persistence moved to BullMQ workers, Redis HA, a Mongo replica set, and per-socket rate limits and payload caps. Alternatively, y-redis or Hocuspocus for a shared CRDT layer if rooms must be served by any node.

Follow-ups: *What breaks first?* Memory and CPU on the single node (docs, sockets, JSON serialisation in write-back), then S3 request volume from full-workspace flushes. *How would you know?* Event-loop delay p99, heap, dirty-set size and flush lag metrics.

Q. How do you handle authentication, unauthorised access and malicious input?

Authentication: Firebase ID tokens verified on every REST call and every socket handshake: signature via cached JWKS, issuer, audience, expiry, subject.

Unauthorised access: identity is always taken from the token; owner checks for close, reopen and delete; `requireSelf` on user-scoped routes; socket events require room membership; public listings project only safe fields. I'm upfront about the gaps: private rooms aren't enforced on join, history and file routes lack membership checks (IDOR), and `create-room` with an existing ID joins the broadcast group. The fix is a shared `requireRoomMember` check plus roles.

Malicious input: JSON parsing and type validation, a path-traversal guard on the disk route, malformed CRDT updates rejected without crashing, rate limits on expensive routes, and safe error messages in production. Gaps: the 1 GB socket buffer, the 50 MB body limit, `innerHTML` for cursor names (XSS), and the unsandboxed `/api/execute`.

Follow-ups: *XSS fix?* `textContent` plus a CSP. *Token theft?* Short expiry, revocation checks for sensitive actions, no tokens in URLs, and XSS hygiene.

Q. What are the weaknesses of the current architecture, and how would you improve it?

Weakness	Improvement
Single process holds sockets and docs (SPOF, vertical-only scaling)	Room-affinity routing or y-redis; Redis adapter; HA
Room authorisation gaps (private join, history/files IDOR, create-room edge)	`requireRoomMember`, roles, invites, secure IDs
Unsandboxed `/api/execute`	Remove, or a sandboxed execution service
Lock release not owner-checked; no fencing	Token value + Lua release; fencing via version
Non-atomic bucket and limiter updates	Lua scripts
Full-workspace uploads per flush; sequential I/O	Changed-files-only, parallel with limits, bulkWrite, queue workers
Text-only persistence loses CRDT continuity	Persist Yjs binary state and an update log
No flush on shutdown; MockRedis in production	Flush on SIGTERM; fail hard or alert on fallback
Custom cursor protocol	Yjs awareness
Reconnect sync race	State-vector two-step sync
Logging only; no metrics	pino + OpenTelemetry + alerts

Follow-up: *Which one first?* Security first (authorisation gaps, execute endpoint, XSS), because they're cheap and high-impact; then durability (flush on shutdown, update log); then scale.

Q. Why not just use Google Docs-style OT, or a library like Liveblocks?

OT needs a central transformation server and is hard to get correct; CRDTs are simpler to reason about and offline-friendly. Liveblocks or other hosted services would remove infrastructure work, but the goal was to own and understand the sync and persistence layers, avoid per-connection pricing, and keep data under our control. For a startup shipping fast, a hosted service could be the right call; I'd say so.

Q. Your token bucket and edit threshold both control flushing. Isn't that redundant?

They answer different questions. The edit threshold and idle timeout decide *when a flush is desirable* (enough change has accumulated, or the user paused). The token bucket decides *whether a flush is permitted right now* (cost cap, burst control). Without the bucket, a large room crossing the threshold every second would flush every second; without the threshold, we'd flush on a timer regardless of activity.

Q. What if the same user opens the room in two tabs?

Two sockets, two Y.Doc replicas; they merge like any two collaborators. Presence shows the user twice (keyed by socket ID); cursors show both. `$addToSet` keeps participants unique. A nicer UX would group presence by UID.

Q. How do you know your CRDT integration is correct and not just "seems to work"?

A convergence test applies concurrent edits from two simulated clients in different orders and asserts identical text on all three docs; reconcile tests prove a stale file list can't overwrite live text; and a malformed-update test proves robustness. Next I'd add property-based tests (random operations, random delivery orders) and a production divergence check that compares doc hashes periodically.

CHAPTER 28

Mock Interview Rounds

Three realistic rounds. Cover the answers, say yours out loud with a timer, then compare. Interviewer lines are in bold.

28.1 Round 1: Project walkthrough (30 minutes, campus or first technical round)

1. **"Introduce yourself and your best project."** Use the 2-minute pitch. End with a hook ("the interesting part is how concurrent edits merge…").
2. **"Draw the architecture."** Browser (React, Monaco, Y.Doc, WebContainer, Firebase SDK) → REST and Socket.IO → Node (middleware, sockets, CRDT, workspace, history) → Redis, Mongo, S3, Gemini, Firebase. Narrate the two channels.
3. **"What did you personally build?"** Be specific: the CRDT migration, the persistence pipeline (token bucket, lock, snapshots), auth hardening, the AI context fix, the tests and CI.
4. **"Why Socket.IO?"** Push, rooms, reconnection, binary, acks; polling's waste and latency.
5. **"What's stored where?"** Contents in S3, metadata in Mongo, hot state in Redis, chats in browser storage.
6. **"How do you authenticate users?"** Firebase tokens verified with jose: iss, aud, exp; the socket handshake too.
7. **"What's one thing you'd improve?"** Room-level authorisation (private rooms, IDOR on history and files), with the concrete middleware.
8. **"Any questions for us?"** Ask about their real-time or collaboration stack, how they handle on-call, and what a new engineer ships in the first month.

28.2 Round 2: Deep dive (45 minutes, product-company technical round)

1. **"Walk me through a keystroke end to end."** The Chapter 17 C-section script.
2. **"Two users type in the same line. What happens?"** CRDT item IDs, origins, deterministic ordering, convergence; the test that proves it.
3. **"Your server crashes. What's lost?"** The layered answer: 400 ms in memory; Redis plus the dirty set survive; lock TTL; Redis durability caveat; the improvements.
4. **"Explain your token bucket. Write it."** Write the lazy-refill function on the board, then mention the Lua script for atomicity.
5. **"Is your lock safe?"** SET NX EX semantics, TTL rationale, unsafe DEL, fencing tokens, Redlock debate, fail-closed.
6. **"Why S3 and Mongo, not just Mongo?"** The 16 MB limit, cost, durability, working set, presigned URLs.
7. **"How does version history avoid duplicates?"** Canonical JSON, SHA-256, compare with the latest, prune to 50, restore as a new version.
8. **"A bug you're proud of fixing?"** The token-bucket mock story (STAR).

28.3 Round 3: System design (45–60 minutes, senior or SDE-2 style)

Prompt: "Design a collaborative code editor for 50,000 concurrent users."

1. **Clarify (3 min):** users per room (2–10 typical), languages, execution needs, history depth, latency target (p95 relay under 100 ms), durability target (RPO under 1 s), AI features, private rooms and roles.
2. **Estimate (3 min):** 10k rooms, about 45k updates/s in and about 180k/s fan-out at peak; Yjs docs about 100 KB–1 MB each, so 1–10 GB of RAM across the fleet; S3 at about 12 flushes/min/room, so about 2k PUT/s if only changed files are uploaded.
3. **High-level design (10 min):** CDN; WebSocket gateway with consistent hashing by room; collaboration nodes owning room shards; Redis (adapter, presence, rate limits, update streams); persistence workers; S3; Mongo/Postgres; an AI service; Firebase/OIDC.
4. **Deep dive (15 min):** the CRDT sync protocol (state vectors), the durability log (Redis Streams → compaction to S3), node failure and rehoming, fencing for persistence.
5. **Bottlenecks (5 min):** hot rooms (cursor fan-out → throttle/awareness), big docs (caps, worker threads), Gemini quotas (queue, per-tenant limits).
6. **Security (5 min):** per-room ACLs and roles, invite tokens, payload caps, per-socket rate limits, sandboxed execution.
7. **Observability (3 min):** SLIs and SLOs, divergence checks, flush lag, event-loop delay.
8. **Tie back:** "CodeFusionAI implements the single-node version of this: here's exactly which pieces exist and which I'd add."

28.4 Rapid-fire round (answer each in one breath)

Question	One-breath answer
Default port of the backend?	5000 (`PORT` env)
Which model?	Gemini 2.5 Flash
Write-back interval?	At most every 400 ms per room
Flush triggers?	10 edit batches, 10 s idle (checked every 30 s), manual save, restore
Token bucket?	Capacity 5, +1 every 5 s, per room, lazy refill
Lock command?	`SET room:{id}:persist:lock 1 NX EX 60`
Workspace cache TTL?	2 hours, sliding
Versions kept?	50 per room
AI rate limit?	30 per minute per user
AI context cap?	12,000 characters; selection preferred
Idle doc drop?	5 minutes after the last member leaves
Token lifetime?	Firebase ID token about 1 hour, auto-refreshed
Why COOP/COEP?	SharedArrayBuffer for WebContainers

Question	One-breath answer
Test counts?	16 backend, 7 frontend; CI on push and PR

PART VI

Revision

Cheat sheets, comparison tables, a glossary, the final checklist and references. Read this the night before.

CHAPTER 29

Quick Reference and Cheat Sheets

29.1 Architecture in ten lines

1. React 19 + Vite SPA; Monaco editor; Tailwind tokens; light UI.
2. Firebase Auth on the client → ID token (JWT) on every request and socket handshake.
3. Express 5 REST: rooms, users, AI, files, history; middleware: auth → rate limit → validate; errors via AppError → errorHandler.
4. Socket.IO: join/create, yjs-update, code-change (structure), cursors, presence, save, restore (ack).
5. Yjs: client Y.Doc bound to Monaco via y-monaco; server Y.Doc per room, Map<path, Y.Text>.
6. Write-back ≤ 400 ms → Redis hash per room → dirty set → edit tokens.
7. Flush: ≥ 10 edits or 10 s idle (interval 30 s) or manual → token bucket (5, +1/5 s) → `SET NX EX 60` lock.
8. Persist: S3 (contents, `users/{owner}/{room}/{path}`) + Mongo (File metadata) + Snapshot (SHA-256 dedupe, keep 50).
9. AI: live file or selection (≤ 12k chars) → backend (30/min) → Gemini 2.5 Flash → markdown with Apply.
10. Run: WebContainers (Node in the browser) with COOP/COEP; xterm terminal; preview iframe.

29.2 Numbers to remember

Thing	Value	Where defined
CRDT write-back throttle	400 ms per room	`socketController.js`
Flush edit threshold	10	`TOKEN_BUCKET_CONFIG`
Idle flush timeout	10 s	`TOKEN_BUCKET_CONFIG`
Background interval	30 s	`server.js`
Bucket capacity / refill	5 / 1 per 5 s	`TOKEN_BUCKET_CONFIG`
Workspace TTL / lock TTL / bucket TTL	2 h / 60 s / 24 h	`REDIS_TTL`
Versions kept	50	`history.service.js`
Idle Y.Doc drop	5 min	`crdt.service.js`
Presigned URL expiry	300 s	`s3.service.js`
Rate limits	AI 30/min · files 120/min · execute 10/min · profile 20/min · history 60/min · delete 20/h	routes
AI context cap	12,000 characters	`AIPanel.jsx`
Socket reconnection	5 attempts, 1 s delay	`socket.js`
Body limit / socket buffer	50 MB / 1 GB (too large)	`server.js`
Execute timeout	10 s	`execute.service.js`

Thing	Value	Where defined
Restore ack timeout (client)	20 s	`Room.jsx`

29.3 Comparison tables

Real-time transports

Option	Direction	Latency	Overhead	Use when
Short polling	Client pull	Up to the interval	High	Rare updates, simple infrastructure
Long polling	Pull, held open	Low	Medium	Fallback when WebSockets are blocked
SSE	Server → client	Low	Low	Feeds and notifications
WebSocket	Both	Lowest	Lowest	Interactive, bidirectional
Socket.IO	Both	Lowest	Low	WebSockets + rooms, acks, reconnection

Conflict resolution strategies

Strategy	Correct?	Collaboration UX	Complexity
Last-writer-wins (whole file)	Loses edits	Bad	Trivial
Pessimistic file lock	Yes	One editor at a time	Low (needs expiry)
Operational Transformation	Yes (if implemented right)	Great	High; central server
CRDT (Yjs)	Yes (convergent)	Great, offline-tolerant	Library-managed

Storage roles

Store	Holds	Strength	Weakness
Redis	Live workspaces, locks, counters	µs latency, atomic ops, TTL	RAM cost; durability depends on config
MongoDB	Rooms, file metadata, snapshots, profiles	Flexible documents, indexes	No FKs; 16 MB doc limit
S3	File contents, snapshot JSON	Durability, cost, scale	Per-request latency and cost
Browser storage	Template seed (IndexedDB), AI chats	Instant, private	Per device, not synced

Rate-limiting algorithms

Algorithm	Burst behaviour	Cost	In CodeFusionAI
Fixed window	Up to 2× at boundaries	1 INCR	API rate limits
Sliding log	Exact	Sorted set per user	—
Sliding counter	Approximate, smooth	2 counters	—

Algorithm	Burst behaviour	Cost	In CodeFusionAI
Token bucket	Bursts up to capacity, steady refill	1 hash	S3 flushes
Leaky bucket	Smooth output, queue	Queue	—

29.4 What is implemented versus proposed (memorise)

Implemented

- Yjs CRDT co-editing, live cursors, presence
- Firebase auth + jose verification (REST and sockets)
- Owner checks, requireSelf, membership on socket events
- Redis cache, dirty set, edit tokens, token bucket, persist lock
- S3 + Mongo persistence, legacy migration
- Snapshots (SHA-256), compare, restore
- AI panel + Assistant (Gemini 2.5 Flash), Apply
- WebContainers run + terminal + preview
- Rate limiting, validation, central errors
- 16 + 7 tests, GitHub Actions CI

Partial / **Proposed**

- Private-room enforcement; room-membership checks on history and files
- Server-side execution (exists, unsandboxed, unused)
- AI review/explain/optimise endpoints (no UI)
- Presigned upload API (not in the main flow)
- Horizontal scaling, Redis adapter, queue workers
- Yjs awareness, two-step sync, binary state persistence
- Metrics, tracing, alerts; load tests
- Docker / backend hosting config (removed / absent)

29.5 FAQ: the twelve most likely questions

Question	Key points
Explain your project.	Real-time collaborative editor + AI; CRDT; Redis → S3/Mongo pipeline; Firebase auth; WebContainers
Why Socket.IO?	Push, rooms, reconnection, binary, acks vs polling's latency and waste
How do concurrent edits work?	Yjs Y.Text, unique item IDs, commutative merges, convergence test
Why S3 and Mongo?	Blobs vs queryable metadata; 16 MB; cost; durability
Why Redis?	Speed, atomic ops across processes, TTL, rich types
What if the server crashes?	≤ 400 ms in memory; Redis + dirty set; lock TTL; Redis durability caveat; fixes
How is it secured?	JWT verify (iss/aud/exp), identity from token, owner checks, rate limits; known gaps + fixes
How does it scale?	Estimate; single node now; room affinity / y-redis + Redis adapter + workers
Hardest bug?	Token bucket mock (STAR) or CRDT migration
What would you improve?	AuthZ gaps → durability → scale; be specific

Question	Key points
How does the AI know the code?	Live model + selection at send time, 12k cap, backend proxy, rate limit
How do you run code safely?	WebContainers in the user's browser; COOP/COEP; execute endpoint caveat

CHAPTER 30

Glossary

Term	Definition
Acknowledgement (ack)	A Socket.IO callback the receiver invokes to reply to a specific emit, giving request/response semantics over a socket.
AppError	Our Error subclass carrying an HTTP status and `isOperational = true`.
Atomic operation	One that happens entirely or not at all, with no observable intermediate state; Redis commands are atomic.
Awareness (Yjs)	A lightweight protocol for ephemeral state such as cursors and user names, with timeouts.
Cache-aside	The application reads the cache, falls back to the store on a miss, and fills the cache.
CAP theorem	Under a partition, choose consistency or availability.
COOP / COEP	Headers that make a page cross-origin isolated, enabling SharedArrayBuffer.
CORS	A browser policy controlling which origins can read cross-origin responses.
CRDT	A replicated data type whose concurrent updates merge deterministically without coordination.
Dirty set	The Redis set of room IDs with unsaved changes.
Distributed lock	Mutual exclusion across processes via a shared store (Redis `SET NX EX`).
Event loop	Node's single-threaded scheduler running callbacks when I/O completes.
Fencing token	A monotonically increasing number issued with a lock, so storage can reject writes from stale holders.
Fixed-window limiter	Counts requests per fixed time window; simple, with boundary bursts.
Idempotent	Applying an operation multiple times has the same effect as once (`$addToSet`, Yjs updates).
IDOR / BOLA	Insecure Direct Object Reference / Broken Object-Level Authorisation: accessing objects by ID without an ownership check.
JWKS	JSON Web Key Set: the published public keys used to verify JWT signatures.
JWT	A signed token with header, payload and signature; claims include sub, iss, aud, exp.
Lazy refill	Computing token-bucket refills from elapsed time on each check instead of using timers.
Middleware	An Express function in the request pipeline: `(req, res, next)`.
Multikey index	A MongoDB index over array elements.
Operational Transformation	A conflict-resolution technique that transforms concurrent operations against each other.
Presigned URL	A time-limited signed URL granting one S3 operation without credentials.
Rate limiting	Capping how many operations an identity may perform per unit of time.
Redis adapter	A Socket.IO component broadcasting across nodes via Redis pub/sub.

Term	Definition
RPO / RTO	Recovery Point Objective (max data loss) / Recovery Time Objective (max downtime).
SharedArrayBuffer	Memory shared between threads; required by WebContainers.
Snapshot	An immutable saved version of a room's workspace.
State vector	Yjs summary of which updates a replica has, used to compute missing updates.
Sticky session	Load-balancer affinity routing a client's requests to the same backend.
Strong eventual consistency	Replicas that received the same updates are in the same state.
Token bucket	A rate-limit algorithm allowing bursts up to capacity with a steady refill.
Tombstone	A marker for deleted CRDT content kept for merge correctness.
TTL	Time to live: automatic key expiry.
WebContainer	StackBlitz's in-browser Node.js runtime.
Write-behind cache	Writes go to the cache first and are persisted to the store asynchronously.
Y.Doc / Y.Text / Y.Map	Yjs document and its shared text and map types.

CHAPTER 31

Final Checklist and References

31.1 The night before

- ☐ I can give the 30-second and 2-minute pitches without notes.
- ☐ I can draw the overall architecture in under 90 seconds and narrate both channels.
- ☐ I can trace a keystroke end to end, naming every hop.
- ☐ I can explain CRDT versus OT versus locking, with one trade-off each.
- ☐ I can write the lazy-refill token bucket on a whiteboard.
- ☐ I can explain `SET NX EX`, why the TTL exists, and the unsafe-release fix.
- ☐ I know the layered answer to "the server crashes before saving".
- ☐ I can justify S3 versus MongoDB versus Redis for each piece of data.
- ☐ I can explain Firebase token verification, including `aud` / `iss`.
- ☐ I can list the known weaknesses and the order I'd fix them in.
- ☐ I have two STAR bug stories ready (token bucket, CRDT migration or isolation).
- ☐ I know the numbers table (400 ms, 10, 10 s, 30 s, 5/5 s, 60 s, 2 h, 50, 30/min, 12k).
- ☐ I can estimate capacity for 1k and 10k users and name what breaks first.
- ☐ I will say "estimate" for capacity and "proposed" for unbuilt features.
- ☐ The demo works: two browser windows, the same room, live typing, AI panel, Run, history restore.

31.2 Demo script (5 minutes)

1. Sign in; show the dashboard (stats, recent rooms, public hub).
2. Create a React room; open the same room in a second window as another user.
3. Type in both windows on the same line: show the live cursors and the merge.
4. Click Run: the terminal runs `npm install` and `npm run dev`, and the preview appears.
5. Select a function and ask the AI to "find bugs"; Apply the fix; undo with Ctrl+Z.
6. Ctrl+S, open History, Compare with a previous version, Restore; the other window updates live.

31.3 References

Official documentation and primary sources for the technical facts in this guide. Project-specific facts come from the CodeFusionAI repository itself.

1. Yjs documentation: shared types, document updates, awareness, UndoManager. https://docs.yjs.dev
2. Nicolaescu, P. et al. "Near Real-Time Peer-to-Peer Shared Editing on Extensible Data Types" (YATA), GROUP 2016.
3. Shapiro, M., Preguiça, N., Baquero, C., Zawirski, M. "Conflict-free Replicated Data Types", SSS 2011.
4. y-monaco binding. https://github.com/yjs/y-monaco
5. Socket.IO documentation: how it works, rooms, adapters, Redis adapter, sticky sessions, connection-state behaviour. https://socket.io/docs/v4/
6. RFC 6455: The WebSocket Protocol. https://www.rfc-editor.org/rfc/rfc6455

7. Express 5 documentation and migration guide (promise rejection handling). https://expressjs.com/en/guide/migrating-5.html
8. Node.js documentation: event loop, test runner (`node:test`), `perf_hooks.monitorEventLoopDelay`. https://nodejs.org/docs/latest/api/
9. MongoDB manual: compound indexes, multikey indexes, BSON document size limit (16 MB), GridFS. https://www.mongodb.com/docs/manual/
10. Mongoose documentation: schemas, validation, indexes, timestamps. https://mongoosejs.com/docs/
11. Redis documentation: SET with NX/EX/PX, INCR rate-limiter pattern, hashes, sets, Lua scripting (EVAL), persistence (RDB/AOF), distributed locks. https://redis.io/docs/
12. Kleppmann, M. "How to do distributed locking" (fencing tokens; Redlock critique), 2016. https://martin.kleppmann.com/2016/02/08/how-to-do-distributed-locking.html
13. Amazon S3 user guide: data durability, strong read-after-write consistency (Dec 2020), request-rate performance per prefix, presigned URLs, object size limits. https://docs.aws.amazon.com/AmazonS3/latest/userguide/
14. AWS SDK for JavaScript v3: S3 client and s3-request-presigner. https://docs.aws.amazon.com/AWSJavaScriptSDK/v3/latest/
15. Firebase Authentication: verifying ID tokens with a third-party JWT library (issuer, audience, signing keys); token lifetime. https://firebase.google.com/docs/auth/admin/verify-id-tokens
16. `jose` library: `createRemoteJWKSet`, `jwtVerify`. https://github.com/panva/jose
17. RFC 7519: JSON Web Token. https://www.rfc-editor.org/rfc/rfc7519
18. Google AI for Developers: Gemini API, models (Gemini 2.5 Flash), JavaScript SDKs. https://ai.google.dev/gemini-api/docs
19. WebContainers documentation: browser support, cross-origin isolation requirements, licensing. https://webcontainers.io/guides/introduction
20. MDN: Cross-Origin-Opener-Policy, Cross-Origin-Embedder-Policy, SharedArrayBuffer security requirements, CORS, Web Workers. https://developer.mozilla.org/
21. Monaco Editor: API reference and ESM integration (MonacoEnvironment.getWorker). https://microsoft.github.io/monaco-editor/
22. Vite documentation: web workers (`?worker` imports), server middleware hooks, build. https://vite.dev/guide/
23. OWASP API Security Top 10 (2023): API1 Broken Object Level Authorization. https://owasp.org/API-Security/
24. OWASP Cheat Sheets: Cross-Site Scripting Prevention, REST Security. https://cheatsheetseries.owasp.org/
25. GitHub Actions documentation: workflow syntax, concurrency, setup-node caching. https://docs.github.com/actions

LAST WORD

You built something real with genuinely hard parts. Explain the mechanism, own the trade-offs, name the gaps before they're found, and always connect each choice to the problem it solves. That is what interviewers hire for.